Команда «Ультратех», задача № 9

Тренажёр диспетчера ДДС

Сопроводительная документация

Учебное программное обеспечение для подготовки диспетчеров дежурно-диспетчерских служб города Москвы с использованием искусственного интеллекта.

Хакатон «Лидеры цифровой трансформации 2026»
Москва, редакция от 20.10.2026

Содержание

1. Пояснительная записка **6**

1. Основание и назначение 6
2. Кто работает в системе 6
3. Что умеет решение 6
4. Состав решения 7
5. Как проходит занятие 8
6. Исходные данные заказчика 8
7. Оценка действий и прогноз 9
8. Чем решение отличается 9
9. Что проверено 9
10. Соответствие техническому заданию 10
11. Границы решения 10
12. Состав поставки 10

2. Маршрут проверки решения **11**

Как открыть 11
Основной маршрут 11
Дополнительные проверки 13
Где ответ по каждому критерию 13
Если что-то не открылось 14

3. Архитектура решения **15**

Функциональная архитектура 15
Компонентная архитектура 16
Режимы ответа собеседников 18
Голосовой тракт 18
Данные 19
Контур без интернета 19
Масштабирование 20
Надёжность 20
Безопасность 21

4. Методы обработки данных **22**

1. Данные заказчика 22
2. Сценарий и карточка 22
3. Задания формирует ИИ 23
4. Генерация сценариев для библиотеки 24
5. Звонки диспетчера ДДС 24

6. Разговоры и режимы ответа 25
7. Речь 25
8. Оценка действий 26
9. Аналитика и прогноз 28

5. Установка, настройка и восстановление 29

1. Требования 29
2. Сервер с интернетом 29
3. Сервер без интернета (закрытый контур) 30
4. Первый вход и проверка 32
5. Настройки 33
6. Модели искусственного интеллекта 34
7. Телефония 35
8. Облачные режимы (только вне закрытого контура) 36
9. Резервное копирование и восстановление 37
10. Обновление 38
11. Частые проблемы 39

6. Руководство пользователя 40

1. Роли 40
2. Вход 40
3. Режимы занятия 41
4. Обучаемый 42
5. Преподаватель 44
6. Администратор 45
7. Если что-то не работает 45

7. Данные заказчика 47

1. Классификатор происшествий (xlsx) 47
2. Билеты (PDF-скан) 51
3. Памятка «Работа на АРМ-112» 52
4. Улицы Москвы 54
5. Учебные сценарии в поставке 54
6. Заложенные ошибки оператора 112 55
7. Звонки и собеседники ДДС 55
8. Озвучка 56
9. Наши тестовые наборы 56

8. Условия и ограничения решения 58

1. Назначение 58
2. Работа без внешних сетей 58

3. Ограничения искусственного интеллекта 58
4. Ограничения телефонии 59
5. Ограничения облачных режимов 60
6. Ограничения данных 60
7. Что не реализовано 60
8. Условия эксплуатации 61

9. Производительность и нагрузка 63
1. Сводка 63
2. Ответ собеседника на локальных моделях 64
3. Голос в линии и в браузере 66
4. Облачный голос (Vapi) 67
5. Написание карточек и сценариев моделью 67
6. Нагрузка по цифрам ТЗ 68
7. Голос под нагрузкой и память (оценка) 70

10. Проверка подсчёта оценки на живых звонках 71
1. Набор живых звонков приёма вызова 71
2. Что изменилось в оценке приёма вызова после прогона 74
3. Звонки диспетчера ДДС: что проверяется сейчас 74
4. Как запустить набор живых звонков 76

11. Использованные библиотеки и компоненты 78
Бэкенд (Python 3.12) 78
Распознавание речи (сервис `stt`) 80
Телефония (сервисы `asterisk`, `asterisk-cloud`) 80
Фронтенд (TypeScript) 80
Инфраструктура (образы) 81
Модели 81
Облачные сервисы (только при `ALLOW_EXTERNAL_AI=true`) 83
Данные 83

12. Соответствие техническому заданию 84
1. Ключевые компоненты решения (раздел «Описание работы компании») 84
2. Предназначение сервиса 85
3. Постановка задачи 86
4. Границы решения 87
5. Общие требования к сервису 88
6. Опциональные требования 89
7. Требования к производительности 90
8. Пользовательские требования: интерфейс 91

9. Роль: администратор 92
10. Роль: преподаватель 94
11. Роль: обучающийся 97
12. Нефункциональные требования 99
13. Сценарии из ТЗ и ответы заказчика 101
14. Требования к решению и сдаче 102
15. Рекомендации заказчика к финалу (16.10.2026) 103
Что не закрыто 105

1. Пояснительная записка

1. Основание и назначение

Работа выполнена по техническому заданию задачи № 9 хакатона «Лидеры цифровой трансформации 2026». Постановщик: Департамент по делам гражданской обороны, чрезвычайным ситуациям и пожарной безопасности города Москвы, ГБУ «Система 112». Команда «Ультратех».

Назначение решения: учебная программа для подготовки диспетчеров дежурно-диспетчерских служб (ДДС) города Москвы, которые работают с карточками происшествий системы-112. Тренажёр повторяет рабочее место АРМ-112 (журнал происшествий и карточку), выдаёт обучаемым учебные карточки, играет по телефону собеседников диспетчера, сам оценивает действия и даёт советы для дальнейшего обучения. Преподаватель получает наблюдение за занятием, отчёты, аналитику группы и прогноз готовности к аттестации.

Схема работы, как её задал заказчик: **преподаватель и ИИ → обучаемый ДДС → результат → рекомендации для дальнейшего обучения**.

Решение может работать в закрытом контуре: распознавание речи, языковые модели, синтез голоса и проверка грамотности выполняются на сервере заказчика, на процессоре, без видеокарты и без обращений во внешние сети.

2. Кто работает в системе

Роль	Что делает в тренажёре
Обучаемый, диспетчер ДДС (основная роль)	принимает карточку от оператора 112 в норматив 30 секунд («Принята» или «Не принята»), проверяет данные карточки и отмечает ошибки оператора 112, по виртуальному телефону высылает аварийную бригаду и докладывает руководству, принимает доклады бригады, ведёт ход работ статусами с номером наряда и комментарием, закрывает карточку
Преподаватель	создаёт группы и занятия, выбирает сложность, наблюдает за обучаемыми во время занятия, смотрит отчёты и разборы, при необходимости исправляет оценку с причиной, ведёт справочник телефонов, учебные сценарии и методические материалы, смотрит аналитику и прогноз
Администратор	пользователи и роли, службы, состояние системы, журнал аудита, резервные копии, настройки
ИИ	пишет каждому обучаемому свои карточки с учётом его ошибок, играет собеседников по телефону, распознаёт речь обучаемого

Дополнительно в программе есть режим оператора 112: приём звонка заявителя, роль которого играет ИИ, и заполнение карточки. По рекомендации заказчика он скрыт и открывается администратором отдельным пользователям (колонка «Режим 112» в разделе «Пользователи»).

3. Что умеет решение

Рабочее место обучаемого. Журнал происшествий и карточка сделаны по скриншотам живого АРМ-112 и памятке «Работа на АРМ-112 для ДДС»: те же колонки, названия статусов, расположение полей и полоса служб. Поля карточки диспетчер не меняет, но может отметить ошибку оператора 112 в адресе, описании, пострадавших, типе происшествия или службах. На первичный статус даётся

норматив (по умолчанию 30 секунд), опоздание отмечается как «Не оповещено». Служебная панель «Тренажёр» у правого края экрана свёрнута и не мешает копии рабочего места; в ней подсказки, если преподаватель их включил.

Виртуальный телефон. Зелёный кружок в правом нижнем углу карточки открывает телефон с клавиатурой и справочником номеров своей организации. Номера задаёт преподаватель, до 10 на службу. Собеседники: руководитель организации, начальник отдела реагирования, начальник аварийной бригады, специалист аварийной бригады. Трубки на вкладках других служб в карточке звонят дежурным этих служб. Начальник бригады выезжает и называет номер наряда, как только узнал адрес и что случилось, а потом бригада сама звонит с докладами. Распоряжение на выезд диспетчер должен дать сам: без него в оценке будет ошибка «Не дано распоряжение на выезд». С заявителем диспетчер не общается; позвонить ему можно, только если преподаватель включил это в занятии. Обучаемый говорит в гарнитуру или пишет текстом, собеседники отвечают голосом.

Сложность. Три уровня. На первом в карточке нет ошибок оператора 112 и собеседники понятливые. На втором в карточке одна ошибка, которую нужно найти. На третьем — чрезвычайная ситуация, две ошибки в карточке, собеседники торопят, у бригады может не оказаться машин или на месте случается осложнение. На третьем уровне карточки идут валом: первая сразу, дальше по одной каждые 40–50 секунд, всего три; в вале бывает повторный вызов о том же происшествии.

Задания формирует ИИ. Если преподаватель отметил это в форме занятия, ИИ сам пишет каждому обучаемому свои карточки по службе ДДС, которую выбрал преподаватель: на сложности 1 одну, на сложности 2 две по очереди, на сложности 3 вал из трёх. Следующая карточка подбирается под ошибки: не зачтена карточка или пропущена ошибка оператора 112 — придёт похожая из той же группы происшествий; всё верно — из группы, где рейтинг обучаемого ниже. У разных обучаемых разные происшествия, преподаватель только наблюдает. Если модель не ответила вовремя, карточка собирается по шаблону, и занятие не останавливается.

Оценка и советы. После каждой карточки обучаемый видит балл, разложенный на составляющие, найденные ошибки словами памятки заказчика со ссылкой на страницу, сравнение своего решения с эталоном, проверку данных карточки, звонки с расшифровкой и до пяти советов, что делать иначе. Если карточку написал ИИ, в разборе объяснено, почему выдана именно она. После занятия в отчёте и в «Моём прогрессе» появляются советы по занятию, после вала — отдельная сводка.

Кабинет преподавателя. Группы, занятия, наблюдение за каждым обучаемым во время занятия, отчёт с выгрузкой в PDF, Excel и CSV, исправление оценки с причиной, аналитика группы, прогноз готовности к аттестации, справочник телефонов, библиотека сценариев с генерацией по фразе, методические материалы.

Кабинет администратора. Пользователи и роли, службы, состояние сервисов с оповещениями, журнал аудита с проверкой целостности, резервные копии, настройки.

Справочник. Памятка «Работа на АРМ-112», методические материалы преподавателя и классификатор происшествий с поиском по словам.

Вход для демонстрации. На демонстрационном сервере на странице входа показаны логины и пароли по ролям и кнопки «Войти». У заказчика это выключается настройками.

4. Состав решения

- **Веб-интерфейс** (React): эмулятор АРМ-112 с виртуальным телефоном, рабочее место оператора 112, кабинеты обучаемого, преподавателя и администратора.

- **Сервер приложения** (Python, FastAPI): занятия и карточки, оценка, разговоры, события занятия, клиент телефонии. Фоновая работа идёт внутри него же: выдача карточек вала и докладов бригады, написание карточек и сценариев, озвучка реплик. Отдельный сервис `worker` запущен из того же образа и оставлен под вынос фоновых задач, сейчас работы не выполняет.
- **Хранилища**: PostgreSQL 16 (данные, история, сценарии, журнал аудита), Redis (рассылка событий занятия и состояние задач генерации).
- **Модели внутри контура**: языковые модели Qwen2.5 через llama.cpp (собеседники и написание карточек), распознавание речи faster-whisper, синтез голоса Silero или Piper и заранее записанные реплики, сравнение смысла комментариев (multilingual-e5), проверка грамотности LanguageTool.
- **Телефония** (Asterisk): своя телефонная станция в поставке. Звонки работают в браузере сразу после установки; к станции подключаются настольные IP-телефоны и линия оператора связи или АТС заказчика по SIP — тогда вызов занятия приходит на настоящий телефон обучаемого, разговор записывается. Подключение — настройками, без доработки программы.
- **Резервные копии**: отдельный контейнер, ежедневная копия базы.

Всё запускается в контейнерах Docker одной командой `./start.sh`. Для сервера без интернета готовится поставка на носителе (`scripts/offline_bundle.sh`), на сервере она ставится командой `sudo ./install.sh` и ничего не скачивает. На демонстрационном сервере для скорости часть функций отдана облачным сервисам (написание карточек и живой голос собеседников); это включается только явной настройкой `ALLOW_EXTERNAL_AI=true`, без неё система с внешним адресом модели не запустится. Подробности — в «Общем техническом описании», разделы 4–7, и в INSTALL.md.

5. Как проходит занятие

1. Преподаватель собирает обучаемых в группу и создаёт занятие: группа, способ подачи карточек (из утверждённых сценариев или «Задания формирует ИИ»), служба ДДС, сложность, норматив, порог зачёта, подсказки, разрешение звонить заявителю. Нажимает «Начать занятие».
2. Карточки приходят в журнал обучаемого. С появления карточки идёт норматив.
3. Обучаемый ставит «Принята» или «Не принята», проверяет данные, звонит бригаде и руководству, принимает доклады, ведёт статусы и закрывает карточку.
4. Система записывает каждое действие с отметкой времени и сравнивает с эталоном.
5. Сразу после закрытия карточки — разбор с баллом, ошибками и советами. В режиме ИИ следующая карточка пишется с учётом этого результата.
6. Преподаватель видит всё это на плитках обучаемых во время занятия, после — в отчёте, аналитике и прогнозе готовности.

6. Исходные данные заказчика

- **Классификатор происшествий** v046: 1283 итоговых типа, 24 группы, 64 службы; по признакам карточки подбираются службы.
- **Памятка «Работа на АРМ-112 для ДДС»**: статусы реагирования, статусы карточки и типичные нарушения. Названия ошибок в разборе взяты оттуда, со ссылкой на страницу.
- **Учебные билеты**: 96 ситуаций распознаны и превращены в учебные сценарии приёма вызова; по ним же сделана часть готовых карточек ДДС. Всего в поставке 96 сценариев приёма вызова и

18 готовых карточек ДДС; остальные карточки ДДС пишет ИИ во время занятия.

- **Скриншоты живой системы и рекомендации заказчика к финалу**: по ним сделаны журнал и карточка ДДС, виртуальный телефон, вход с логинами по ролям, скрытие режима оператора 112.

Разбор данных и способ загрузки — в DATASET.md.

7. Оценка действий и прогноз

Оценка складывается из независимых составляющих: решение по карточке, время принятия относительно норматива, цепочка статусов, комментарии и номер наряда, типичные ошибки, грамотность, проверка данных карточки, звонки в службы. Неприменимые к карточке составляющие отбрасываются, остальные нормируются к 100. Оценка считается по правилам памятки заказчика, поэтому каждый балл можно объяснить и пересчитать; смысл комментариев сравнивается с эталоном моделью multilingual-e5, грамотность проверяет LanguageTool. Советы собираются из найденных ошибок, от самой важной. Преподаватель вправе изменить итог с указанием причины: прежняя оценка сохраняется, изменение попадает в журнал аудита.

Прогноз готовности к аттестации считается по истории попыток обучаемого и показывается вместе с проверкой достоверности: качество модели на отложенной выборке, калибровка обещанной вероятности против фактической и вес признаков. Реальных аттестаций у системы пока нет, поэтому модель обучена на виртуальной когорте обучаемых; об этом прямо написано на странице «Достоверность прогноза», модель переобучается на настоящих данных по мере их накопления. Методика — в METHODS.md.

8. Чем решение отличается

- **Рабочее место как в оригинале.** Обучаемый тренируется на той же раскладке журнала и карточки, что в живом АРМ-112, и на тех же статусах, что в памятке.
- **Телефон и собеседники.** Диспетчер действительно звонит: набирает номер или выбирает его в справочнике, говорит голосом, получает номер наряда и доклады бригады. Ошибки разговора («не дано распоряжение на выезд», «позвонил не тому», «доклад бригады не принят») входят в оценку.
- **ИИ выдаёт разные задания и учит на ошибках.** Каждому обучаемому свои карточки, подобранные под его промахи, без участия преподавателя.
- **Объяснимая оценка.** Балл раскладывается на составляющие, ошибки названы словами памятки со ссылкой на страницу, преподаватель может исправить итог с причиной.
- **Работа без интернета и без видеокарты.** Все модели ставятся на сервер заказчика; адреса моделей задаются настройками, поэтому их можно вынести на отдельный сервер.

9. Что проверено

Проверка	Результат
Подсчёт баллов на живых звонках (режим приёма вызова)	12 звонков, 408 сверок итога и составляющих с независимым пересчётом, расхождений нет
Отклик интерфейса при 100 пользователях	худшее значение 1,8 с при требовании ТЗ не более 2 с
Одновременные занятия	20 обучаемых одновременно ведут карточки, отклик 1,1 с

Проверка	Результат
Запись в базу	825 событий в секунду при требовании не менее 100
Задержка сети в голосовом звонке	2 мс туда и обратно в локальной сети при требовании не более 150 мс
Автоматические тесты	91 файл тестов сервера, 18 наборов тестов интерфейса и 15 сценариев проверки в браузере, в том числе виртуальный телефон, режим «Задания формирует ИИ» и вал карточек

Цифры нагрузки замерены на сервере с 6 ядрами и 12 ГБ памяти; способ замера — в PERFORMANCE.md, живые звонки — в VOICE_TESTS.md. Голосовые разговоры с моделями на процессоре тяжелее веб-нагрузки: на 6 ядрах рассчитывайте на 3–4 разговора одновременно.

10. Соответствие техническому заданию

Требования ТЗ разобраны построчно в REQUIREMENTS_MATRIX.md: где реализован каждый пункт и как его проверить. Не сделаны вход через внешний каталог учётных записей (LDAP) и второй фактор при входе.

11. Границы решения

Тренажёр учебный: не принимает настоящие вызовы, не связан с реальными службами и не хранит персональные данные граждан, все происшествия учебные. Собеседник на процессоре в основном режиме (предзаписанные реплики) отвечает через 1,5–3 секунды; распознавание речи может ошибаться на номерах домов и именах при плохом микрофоне, поэтому преподаватель видит расшифровку и может исправить оценку. Полный перечень ограничений — в LIMITATIONS.md.

12. Состав поставки

- исходный код без обфускации, запуск описан в README;
- документация: «Общее техническое описание», руководства администратора, преподавателя и обучаемого (PDF в `docs/submission`), настоящая записка, маршрут проверки, архитектура, методы, установка, ограничения, данные;
- учебные данные: классификатор, памятка, сценарии из билетов, готовые карточки ДДС, записи реплик;
- файлы развёртывания: Docker Compose, `start.sh`, сборка поставки без интернета, пример настроек `.env.example`, скрипты резервного копирования и проверки восстановления;
- презентация и видеоролик работы тренажёра (ссылки в README).

2. Маршрут проверки решения

Короткий маршрут для эксперта: где открыть, под кем войти, что нажать и на что смотреть, и где лежит ответ по каждому критерию оценки. Основной маршрут занимает 10–15 минут. Подробные описания — в четырёх руководствах (`docs/submission`) и остальных документах `docs/` .

Что это за система. Учебная программа для подготовки диспетчеров ДДС города Москвы: копия рабочего места АРМ-112 (журнал происшествий и карточка), виртуальный телефон с собеседниками, карточки, которые ИИ пишет каждому обучаемому под его ошибки, автоматическая оценка с советами, кабинет преподавателя с отчётами, аналитикой и прогнозом готовности. Может работать в закрытом контуре без интернета, на процессоре, без видеокарты.

Как открыть

Способ	Что делать
Готовый сервер	https://155-212-186-2.sslip.io — преподаватель `teacher` , обучающийся `student` , пароль у обоих `112112112`
У себя	Linux с Docker: `git clone` , затем `./start.sh` — скачает нейросети, соберёт и запустит; вход `teacher` / `student` / `admin` , пароль `112112112` . Сервер без интернета — готовая поставка `offline-bundle` . Подробно в README и INSTALL.md

На странице входа показана таблица логинов и паролей по ролям и кнопки «Войти» — вход в один клик. Кроме преподавателя и обучающегося там есть администратор `admin` (пароль тот же). Для проверки двух обучаемых в установке есть второй обучающийся `student2` с тем же паролем.

Браузер: Chrome, Яндекс.Браузер или Firefox. Для звонков нужны микрофон и наушники; когда браузер спросит доступ к микрофону, нажмите «Разрешить». Без микрофона в звонке можно писать текстом. Преподавателя и обучающегося удобно открыть в разных браузерах или в обычном окне и окне инкогнито.

Основной маршрут

1. Преподаватель: подготовить занятие (3 минуты)

Войдите преподавателем. Меню: «Занятия», «Группы», «Аналитика», «Сценарии», «Справочник телефонов».

1. «Группы»: если группы ещё нет, «Создать группу», отметьте `student` (и `student2` , если проверяете двух обучаемых), «Создать группу».
2. «Справочник телефонов»: номера людей организации по службам, до 10 на службу, четыре должности. Эти номера обучаемый увидит в своём телефоне.
3. «Занятия» → «Создать занятие»:
   - выберите группу;
   - поставьте галочку **«Задания формирует ИИ»** и отметьте «ДДС какой службы»;
   - «Сложность»: 1 — одна карточка без ошибок оператора 112; 2 — две карточки по очереди, в карточке одна ошибка оператора 112; 3 — вал из трёх карточек, чрезвычайная ситуация, по

две ошибки в карточке;

- по желанию «Разрешить звонок заявителю (по номеру из карточки)»;
- «Создать занятие», затем **«Начать занятие»**.

На плитке обучаемого появится «ИИ пишет карточку». Облачная модель на демонстрационном сервере пишет карточку за несколько секунд, модель на сервере без интернета — до двух минут.

2. Обучающийся: отработать карточку (5 – 7 минут)

Войдите обучающимся в другом окне.

1. «Мои задания» → «Открыть журнал АРМ-112». Журнал повторяет живой АРМ-112, слева у строки идёт таймер норматива 30 секунд.
2. Откройте карточку щелчком по строке. Нажмите оранжевый карандаш на вкладке своей службы, выберите «Принята» и сохраните (или `Alt+A`) — до конца норматива, иначе «Не оповещено».
3. Сверьте поля с описанием со слов заявителя. На сложности 2 и 3 оператор 112 ошибся: кнопка с флажком рядом с полем отмечает ошибку и верное значение.
4. Зелёный кружок в правом нижнем углу — виртуальный телефон. Позвоните начальнику аварийной бригады (трубка на вкладке своей службы или номер из «Справочника»), назовите адрес и что случилось, скажите «Выезжайте». Начальник назовёт номер наряда.
5. Поставьте «Начало реагирования» с этим номером наряда.
6. Бригада сама звонит с докладами: кружок загорается красным, нажмите «Ответить». После каждого доклада — свой статус: «Прибытие», «Проведение работ», «Работы завершены» с комментарием.
7. Панель «Тренажёр» покажет балл и ссылку **«Открыть разбор»**.

В разборе: итог и составляющие, советы для дальнейшего обучения, почему ИИ выдал именно эту карточку, эталон против решения, типичные ошибки со ссылкой на страницу памятки, проверка данных карточки, звонки с расшифровкой и записью. На сложности 2 следующая карточка пишется после оценки первой и подбирается под ошибки.

3. Преподаватель: наблюдение и результат (3 минуты)

1. Страница идущего занятия: сводка и плитка на каждого обучаемого — что делает, таймер норматива, баллы. Щелчок по плитке открывает его текущую карточку.
2. «Отчёт»: таблица по обучаемым, раскрытие до отдельных карточек и колонка «Действия» с шагами и временем, советы по занятию, выгрузка PDF, XLSX, CSV.
3. В разборе карточки — «Изменить оценку» с обязательной причиной и «Комментарии преподавателя».
4. «Аналитика»: выжимка, тепловая карта «обучающиеся × группы происшествий», динамика по неделям, типичные ошибки группы, прогноз готовности к аттестации. Ссылка **«Достоверность прогноза»**: метрики на отложенной выборке, калибровка, вес признаков и откуда взяты данные.

4. Администратор (1 минута)

Войдите `admin`. «Пользователи» (ФИО скрыто до «Показать», колонка «Режим 112»), «Состояние» (плитки сервисов и нагрузка), «Аудит» (кнопка «Проверить целостность»), «Копии», «Настройки».

Дополнительные проверки

Что проверить	Как
Два обучаемых получают разные задания	В группе `student` и `student2`, занятие с «Задания формирует ИИ». Откройте обоих в разных браузерах: происшествия разные
Вал карточек	Занятие сложности 3: первая карточка сразу, дальше по одной каждые 40–50 секунд, всего три. После вала в «Моём прогрессе» и в отчёте — сводка «Вал карточек»
Трудные собеседники	На сложности 3 начальник бригады торопит, может ответить «машин нет» (нужно доложить руководителю организации), или специалист бригады сообщит об осложнении (нужно оповестить другую службу)
Звонок заявителю	С галочкой «Разрешить звонок заявителю» в справочнике телефона появляется «Заявитель — номер из карточки», трубка у «АОН» активна; без галочки она серая
Готовые сценарии	Снимите «Задания формирует ИИ»: карточки берутся из утверждённых сценариев, под формой «Доступно карточек: N». «Сценарии» → «Сгенерировать по фразе» создаёт черновик, «Утвердить целиком» отправляет его в занятия
Режим оператора 112	Администратор ставит галочку «Режим 112» преподавателю и обучающемуся. В форме занятия появляется режим «Приём вызова (оператор 112)»: заявителя играет ИИ, обучающийся заполняет адрес, опросную карту и службы

Где ответ по каждому критерию

Критерий экспертизы	Где смотреть
Идея и способ решения, используемые технологии	EXPLANATORY_NOTE.md, ARCHITECTURE.md, LIBRARIES.md, «Общее техническое описание»
Качество кода	`backend/app` и `frontend/src`; тесты: 91 файл тестов сервера (`backend/tests`), 18 наборов тестов интерфейса, 15 сценариев проверки в браузере (`frontend/e2e`)
Возможность интеграции в другие системы	HTTP API с описанием OpenAPI на `/api/openapi.json` (страница `/api/docs` включается вне боевого режима), метрики для Prometheus, телефония по SIP и WebRTC, адреса моделей задаются настройками, см. ARCHITECTURE.md
Скорость работы решения	PERFORMANCE.md: отклик при 100 пользователях, 20 одновременных занятий, запись в базу, задержка голоса, время ответа модели на процессоре
Проверка работы расчётов	Разбор любой карточки: балл разложен на составляющие; VOICE_TESTS.md: 12 живых звонков, 408 проверок подсчёта баллов без расхождений с независимым пересчётом; формулы в METHODS.md
Проверка прогнозирования и достоверности	Страница «Достоверность прогноза» в кабинете преподавателя; методика в METHODS.md
Объективность диаграмм	«Аналитика»: у каждого блока указан период и число оценённых попыток, под тепловой картой — как её читать
Соответствие техническому заданию	REQUIREMENTS_MATRIX.md: пункты ТЗ, где реализованы и как проверить

Критерий экспертизы	Где смотреть
Соответствие рекомендациям заказчика к финалу	Логины по ролям на странице входа; четыре PDF в `docs/submission`; журнал и карточка как в оригинале; режим оператора 112 скрыт и включается администратором; виртуальный телефон; «Задания формирует ИИ» с советами
Полнота сопроводительной документации	`docs/submission`: «Общее техническое описание», руководства администратора, преподавателя, обучаемого; `docs/`: архитектура, методы, установка, ограничения, данные, библиотеки
Честные границы решения	LIMITATIONS.md: что система не делает и почему
Работа с данными заказчика	DATASET.md: классификатор происшествий, памятка, учебные билеты, что загружено и как

Если что-то не открылось

Что видно	Что сделать
Браузер пишет «Подключение не защищено» (своя установка)	«Дополнительные» → «Перейти на сайт»
Собеседник не слышит	Разрешите микрофон в адресной строке браузера или пишите текстом в поле звонка
Обучающийся не видит карточек	Проверьте, что он в группе занятия и занятие запущено; в режиме ИИ подождите, пока на плитке идёт «ИИ пишет карточку»
Страница не открывается сразу после установки	Подождите 2–3 минуты: загружаются модели. Проверка: `https://<адрес>/health` → `"status":"ok"`

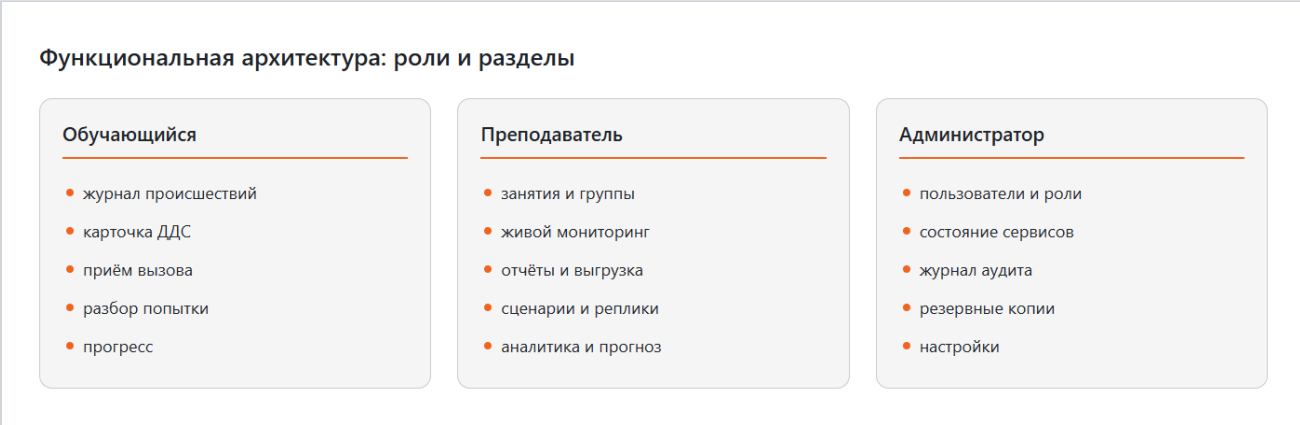
Запасной путь — видеоролик работы тренажёра, ссылка в README. Установка с нуля, перенос без интернета и проверка восстановления из копии — в INSTALL.md.

3. Архитектура решения

Тренажёр диспетчера ДДС ставится на один сервер учебного класса. Рабочие места преподавателя и обучающихся — компьютеры с браузером в той же сети. По умолчанию система работает в закрытом контуре: модели ИИ, распознавание и синтез речи, проверка грамотности и телефония находятся на сервере и не обращаются во внешние сети. Облачные части (модель для написания карточек, облачный голос собеседников, SIP-линия оператора связи) подключаются только явной настройкой, см. раздел «Контур без интернета».

Функциональная архитектура

Система обслуживает три роли. Четвёртый участник — ИИ: он пишет карточки, играет собеседников по телефону и оценивает действия. Схема работы, как её задал заказчик: **преподаватель и ИИ → обучающийся → результат → рекомендации для дальнейшего обучения**.



Роли и разделы системы

Обучающийся (диспетчер ДДС) работает на копии АРМ-112: журнал происшествий, карточка ДДС с виртуальным телефоном, разбор каждой карточки с советами, «Мои задания» и «Мой прогресс», справочник (памятка, методические материалы, классификатор).

Преподаватель ведёт группы и занятия, наблюдает за обучающимися во время занятия, смотрит отчёты с выгрузкой в PDF, Excel и CSV, исправляет оценку с указанием причины, ведёт справочник телефонов, библиотеку сценариев (с генерацией по фразе) и методические материалы, смотрит аналитику группы и прогноз готовности.

Администратор ведёт пользователей и роли, службы, видит состояние сервисов с оповещениями, журнал аудита с проверкой целостности, резервные копии и настройки (в том числе телефонии).

Режимы занятий

Реагирование на карточку (основной). Карточка от оператора 112 приходит в журнал службы обучающегося. Он принимает её («Принята») или отклоняет («Не принята» с причиной) в пределах норматива, отмечает ошибки оператора 112 в данных карточки, по виртуальному телефону звонит начальнику аварийной бригады, руководству и дежурным других служб, принимает доклады бригады, ведёт ход работ статусами с номером наряда и комментарием и закрывает карточку.

Приём вызова (оператор 112). Обучающийся принимает звонок заявителя, роль которого играет ИИ, и заполняет карточку: опросную карту признаков, адрес, флаги, описание. Службы для оповещения подставляются по классификатору. По рекомендации заказчика режим скрыт: администратор открывает его отдельным пользователям, сервер проверяет право доступа при создании занятий и сценариев этого режима.

Как подаются карточки

- **Из утверждённых сценариев.** Карточки берутся из библиотеки по сложности занятия, при включённом адаптивном подборе — из групп происшествий, где рейтинг обучающегося ниже.
- **«Задания формирует ИИ».** Каждому обучающемуся модель пишет свои карточки во время занятия по службе ДДС, выбранной в занятии: на сложности 1 — одну, на сложности 2 — две по очереди, на сложности 3 — вал из трёх. Следующая карточка пишется после оценки предыдущей с учётом её ошибок. Если модель не ответила вовремя, карточка собирается по шаблону.
- **Вал на сложности 3.** Первая карточка сразу, следующие по одной через 40–50 секунд, всего три; в обычном занятии среди них бывает повторный вызов о том же происшествии.

Ход занятия

1. Преподаватель создаёт занятие (группа, способ подачи карточек, служба, сложность, норматив, порог зачёта, режим ответа собеседников, разрешение звонить заявителю) и начинает его.
2. Сервер выдаёт карточки в журнал обучающегося и рассылает события занятия.
3. Каждое действие обучающегося (статус, отметка ошибки, звонок, фраза разговора) пишется в базу с отметкой времени.
4. Фоновый цикл сервера следит за нормативом («Не оповещено»), выдаёт карточки вала и запускает доклады бригады.
5. После закрытия карточки движок оценки сравнивает действия с эталоном сценария и выдаёт разбор с баллом, ошибками и советами. Результат обновляет рейтинг навыка обучающегося.
6. Преподаватель видит ход занятия на плитках обучающихся, после занятия — отчёт, аналитику и прогноз.

Компонентная архитектура

Все части запускаются в контейнерах Docker Compose.

Браузер. Интерфейс на React: эмулятор АРМ-112 с виртуальным телефоном, рабочее место оператора 112, кабинеты. С сервером работает по HTTPS и WebSocket через nginx.

nginx завершает TLS (свой центр сертификации создаётся при первом запуске), отдаёт собранный интерфейс, проксирует `/api` и `/ws` к бэкенду и `/ws/sip` к Asterisk. Задаёт заголовки безопасности (CSP, HSTS).

backend (Python, FastAPI, один процесс uvicorn) содержит всю логику:

Модуль	Что делает
`training`	занятия, выдача карточек, вал, доклады бригады, задания от ИИ, справочник телефонов, отчёты, события занятия
`dialog`	ход разговора с заявителем и с собеседниками диспетчера ДДС

Модуль	Что делает
`telephony`	клиент Asterisk (ARI), звук звонка, звонки в службы, облачный голос
`scenarios`	библиотека сценариев, генерация, утверждение, озвучка реплик
`domain/evaluation`	оценка действий, типичные ошибки, советы
`domain/analytics`, `analytics`	рейтинг навыка, адаптивный подбор, прогноз готовности
`providers`	доступ к моделям: языковая модель, распознавание и синтез речи, смысловая близость, грамотность, поиск по материалам
`admin`, `auth`, `reports`	пользователи, состояние, аудит, копии, вход, выгрузка отчётов

Фоновая работа идёт внутри процесса бэкенда:

- цикл раз в 5 секунд выдаёт карточки вала, ставит «Не оповещено» и «Не завершено», запускает доклады бригады; расписание считается по отметкам времени в базе, а записи блокируются (`SKIP LOCKED`), поэтому перезапуск ничего не теряет;
- генерация сценариев, написание карточек для режима ИИ и озвучка реплик идут асинхронными задачами; состояние задач генерации хранится в Redis;
- монитор состояния сервисов раз в минуту открывает оповещения администратору.

Сервис `worker` (ARQ) запущен из того же образа и зарезервирован под вынос фоновых задач; сейчас в нём только служебная проверка.

Хранилища. PostgreSQL 16 — пользователи, занятия, попытки, оценки, сценарии и их версии, события занятия, журнал аудита. Redis — рассылка событий занятия между подключениями и состояние задач генерации.

Модели и речь (профиль `ai`): языковые модели Qwen2.5 через сервер llama.cpp (`llm-dialog` — собеседники, `llm-gen` — написание сценариев и карточек), распознавание речи faster-whisper (`stt`). Синтез речи (Silero, Piper), смысловая близость (multilingual-e5) и определитель фраз в звонке (Silero VAD) работают внутри бэкенда. LanguageTool проверяет грамотность.

Телефония (профиль `telephony`): Asterisk с PJSIP. Программный телефон в браузере регистрируется по WebRTC через `/ws/sip`, настольный IP-телефон — по SIP на порт 5060. Бэкенд управляет звонками через ARI: соединяет обучающегося с моделью, снимает его голос, проигрывает ответы и пишет разговор.

Сервисы Docker Compose

Сервис	Профиль	Назначение
`nginx`	всегда	TLS, интерфейс, прокси к `/api`, `/ws`, `/ws/sip`
`backend`	всегда	API, WebSocket, занятия, разговоры, оценка, клиент телефонии, фоновый цикл
`worker`	всегда	ARQ, зарезервирован под фоновые задачи
`postgres`	всегда	база данных (образ с pgvector, векторные колонки не используются)
`redis`	всегда	события занятия, состояние задач генерации

Сервис	Профиль	Назначение
`languagetool`	всегда	проверка грамотности текстов обучающегося
`backup`	всегда	ежесуточная копия базы данных
`llm-dialog`	`ai`	языковая модель собеседников
`llm-gen`	`ai`	языковая модель для сценариев и карточек
`stt`	`ai`	распознавание речи
`asterisk`	`telephony`	SIP, WebRTC, запись разговоров, ARI, SIP-транк оператора связи (по настройке)
`asterisk-cloud`	`cloud`	Asterisk с SIP-транком в облачный голосовой сервис; вместо `asterisk`, вместе не поднимаются

Профили `live` и `monitoring` объявлены как заготовка под узел с видеокартой и под Prometheus с Grafana и пока не содержат сервисов. Бэкенд уже отдаёт метрики по адресу `/api/metrics`.

Поставка заказчику (`start.sh` и установщик без интернета) поднимает профили `ai` и `telephony` с настройками демонстрационного стенда: одна модель Qwen2.5-3B в `llm-dialog` ведёт и разговоры, и написание карточек (сервис `llm-gen` не запускается), распознавание — `faster-whisper-small`.

Режимы ответа собеседников

Режим выбирается в занятии («Как отвечает заявитель» или «Как отвечают служба и бригада»):

Режим	Где работает	Как устроен
Предзаписанные реплики	сервер	модель выбирает номер утверждённой реплики, текст не сочиняет; реплики озвучены заранее
Свободная генерация речи (без интернета)	сервер	модель отвечает своими словами по листу фактов, ответ озвучивается на сервере
Свободная генерация речи (с интернетом)	облако	весь разговор ведёт облачный голосовой сервис; сервер даёт ему факты и роль, после звонка читает запись и анкету

Облачный режим предлагается только при включённом облачном голосе. Если облако не ответило за 15 секунд или звонок оборвался, разговор продолжает локальная модель в режиме предзаписанных реплик. Без модели диалога ответы подбираются по ключевым словам, в приёме вызова появляются кнопки тем. Подробности — METHODS.md, раздел 6.

Голосовой тракт

Браузер без телефонии (основной путь). Обучающийся держит кнопку «Удерживайте и говорите» (или включает микрофон в приёме вызова), браузер записывает фразу и отправляет её по HTTPS. Сервер распознаёт речь, получает ответ собеседника и отдаёт его звуковым файлом. Можно отвечать и текстом.

Через Asterisk. Телефония включается не для всего сервера, а для занятия: галочкой «Живой звонок на телефон» с номером телефона. Галочка доступна, только если включена телефония и настроена SIP-линия оператора связи (в форме — для приёма вызова с облачным голосом). Тогда вызовы занятия приходят на указанный обычный телефон. Без номера занятия Asterisk звонит на программный телефон в браузере (WebRTC) и на настольный IP-телефон обучающегося (SIP). Для локальных моделей Asterisk передаёт голос обучающегося на сервер по RTP, сервер режет поток на фразы (Silero VAD, при его отсутствии — по громкости), распознаёт каждую фразу, выбирает ответ и проигрывает его в звонок. Разговор записывается целиком. В остальных занятиях голос идёт через браузер, как описано выше.

Облачный голос. Браузер соединяется с облачным сервисом напрямую через его библиотеку, или звонок идёт по SIP через `asterisk-cloud`. Сервер передаёт облаку роль и факты карточки (факты не попадают в браузер), принимает события звонка по защищённому адресу с секретом и сохраняет запись разговора у себя.

Данные

Что	Где
Пользователи, группы, занятия, попытки, действия, оценки, исправления оценок	PostgreSQL
Справочники заказчика: классификатор, службы, статусы, причины отказа, улицы Москвы	PostgreSQL, загружаются при первом запуске из `data/seed` и файлов заказчика
Сценарии и их версии	PostgreSQL, тело сценария в JSONB
Справочник телефонов служб	PostgreSQL
События занятия	PostgreSQL (с порядковым номером) и рассылка через Redis
Журнал аудита	PostgreSQL, цепочка контрольных сумм, у роли приложения нет права изменять и удалять записи
Методические материалы для поиска при генерации	текст в томе `storage`, индекс строится в памяти бэкенда при запуске
Озвученные реплики, студийные записи, записи разговоров	том `storage`
Резервные копии базы	том `backups`
Модели ИИ	папка `models` (не в git), подключается только на чтение

Отчёты не хранятся: PDF, Excel и CSV собираются по запросу.

Контур без интернета

- Модели скачиваются один раз (`scripts/fetch_models.sh`) и проверяются по контрольным суммам из `scripts/models.manifest`.
- Для сервера без интернета `scripts/offline_bundle.sh --with-models` собирает поставку: образы Docker, исходный код, модели, файлы заказчика, установщик и контрольные суммы. Установщик проверяет суммы, загружает образы и запускает систему, ничего не скачивая.

- При запуске бэкенд проверяет адреса моделей (`LLM_DIALOG_URL` , `LLM_GEN_URL` , `STT_URL`) и облачный голос: если адрес не локальный или облачный голос включён, а `ALLOW_EXTERNAL_AI` не равен `true` , система не запускается и сообщает причину.
- SIP-линия оператора связи для звонков на обычный телефон по умолчанию выключена и включается только заполнением её настроек.
- Шрифты и библиотеки интерфейса входят в сборку, внешних запросов из браузера нет. Только при включённом облачном голосе nginx разрешает в CSP адреса облачного сервиса.
- Служебные порты (база, Redis, модели, ARI) открыты только на самом сервере. Наружу смотрят HTTPS (443), HTTP (80) и, при телефонии, SIP и диапазон голосовых портов.

На демонстрационном стенде включены облачная модель для написания карточек и облачный голос; код при этом тот же.

Масштабирование

Вертикальное. Больше ядер и памяти ускоряют модели и позволяют вести больше одновременных голосовых разговоров. Замеры — PERFORMANCE.md.

Вынос моделей. Модели и распознавание речи выносятся на отдельный узел той же сети: их адреса задаются переменными окружения, код не меняется. Распознавание совместимо с OpenAI-интерфейсом транскрипции, поэтому его можно заменить сервером на видеокарте.

Несколько экземпляров бэкенда. Часть состояния уже вынесена из процесса: события занятия идут через базу и Redis и догоняются по номеру, расписание вала и докладов считается по базе с блокировкой записей, задачи генерации хранят состояние в Redis. Но активные звонки Asterisk и облачного голоса, а также идущие задачи генерации живут в памяти одного процесса, поэтому при телефонии бэкенд запускается в одном экземпляре.

Надёжность

- У сервисов есть проверки работоспособности, контейнеры перезапускаются при падении (`restart: unless-stopped`). Администратор видит состояние сервисов и получает оповещения.
- При недоступной модели занятие не останавливается: карточка ИИ собирается по шаблону, собеседник отвечает по ключевым словам, облачный звонок переходит на локальную модель, оценка без LanguageTool пересчитывается без грамотности.
- Копия базы делается ежесуточно в заданное время, хранится заданное число копий, копию можно создать вручную. Восстановление проверяется скриптом `scripts/restore_test.sh` : он разворачивает копию в отдельную базу и проверяет вход, чтение таблиц и цепочку аудита. Записи разговоров и озвучка (том `storage`) в копию базы не входят и копируются вместе с сервером.
- Обрыв связи не приводит к потере данных: события занятия догоняются по порядковому номеру, черновик карточки приёма вызова хранится в браузере и на сервере, отправка карточки и смена статусов идемпотентны; черновик статуса в карточке ДДС хранится в браузере.
- Журнал аудита связан цепочкой контрольных сумм, проверка цепочки запускается из кабинета администратора.

Безопасность

- Доступ по ролям с проверкой владельца объекта на стороне сервера; режим оператора 112 — отдельное право пользователя.
- Пароли хранятся в виде хешей Argon2, после 5 неудачных попыток вход закрывается на 15 минут. Короткоживущий токен доступа передаётся в заголовке, токен продления — в защищённой cookie.
- Трафик браузера идёт по HTTPS, программный телефон — по защищённому WebRTC. Настольные IP-телефоны работают по обычному SIP внутри локальной сети.
- SIP-пароли обучающихся хранятся в базе в зашифрованном виде.
- Персональные данные минимальны: в списках показываются инициалы, показ ФИО — отдельное действие, которое пишется в журнал аудита. Все происшествия и заявители учебные.
- Внешние обращения запрещены конфигурацией: без явного разрешения `ALLOW_EXTERNAL_AI` система с внешним адресом модели или облачным голосом не запускается.
- Вход в один клик и таблица логинов на странице входа нужны только для демонстрации и выключаются настройками `QUICK_LOGIN` и `SHOW_DEMO_CREDENTIALS`.

4. Методы обработки данных

Документ описывает, как система обрабатывает данные заказчика, как строятся учебные карточки и сценарии, как ведутся разговоры, как распознаётся и синтезируется речь, как оценивается работа обучающегося и как строится прогноз готовности.

1. Данные заказчика

Классификатор происшествий. Таблица заказчика (v046) разбирается программно: три уровня признаков («что», «где», уточнение) образуют дерево опросной карты, итоговый тип происшествия и набор служб берутся из тех же строк. Всего 24 группы, 1283 итоговых типа, 64 службы. У части служб есть колонки под дополнительные признаки («Пострадавшие / погибшие», «Нет доступа», «Угроза людям», «Правонарушение» и другие, всего 16 флагов), поэтому список оповещаемых служб зависит не только от типа, но и от выбранных флагов.

Правило выбора служб хранится списком условий. Службы — это объединение всех правил, условия которых выполнены (правило без условий выполняется всегда), за вычетом служб из выполненных правил «нет реагирования». Функция подбора служб покрыта табличным тестом на строках исходного файла.

Памятка «Работа на АРМ-112». Текст берётся из PDF без распознавания, постранично. Из неё взяты 9 статусов реагирования, 7 статусов карточки, 7 причин отказа, правила простановки статусов и 29 типичных ошибок со ссылками на страницы. Тексты используются в оценке, в советах и в справочнике обучающегося.

Билеты. Учебные ситуации заказчика пришли сканом. Страницы распознаются по ячейкам таблицы (Tesseract, русский язык, 300 dpi), ячейки с уверенностью ниже порога сверены вручную, правки хранятся отдельным файлом. Из 96 ситуаций сделаны 96 сценариев приёма вызова и часть готовых карточек ДДС (в поставке их 18). Каждая ситуация превращена в сценарий: лист фактов для заявителя, реплики, эталонная карточка и эталонные действия.

Улицы Москвы. Список улиц с округом и районом собран из OpenStreetMap: улица привязывается к району по геометрии границ. Он нужен для подсказок распознаванию речи, проверки адресов и настоящих адресов в карточках ИИ.

2. Сценарий и карточка

Сценарий — это данные, а не код. В нём хранятся: карточка в том виде, как её видит обучающийся; правда о происшествии; ошибки оператора 112, заложенные в карточку (подвохи); лист фактов и утверждённые реплики собеседников; эталонные решение, статусы, комментарии и звонки в службы; расписание докладов бригады. Сценарии хранятся с версиями. Перед утверждением тело сценария сверяется со справочниками: тип происшествия, флаги, службы, статусы, причины отказа, темы вопросов, голоса и заложенные ошибки должны существовать. Сценарий с ошибками не утверждается.

Сложность задаёт число подвохов и поведение собеседников:

Сложность	Подвохи в карточке	Собеседники
1	нет	понятливые
2	одна ошибка оператора 112	понятливые
3	две ошибки, карточка помечена как ЧС	начальник бригады торопит и перебивает длинные фразы; у бригады нет машин или на месте осложнение; карточки идут валом

В сгенерированную карточку подвохи закладываются программно, в готовых сценариях они записаны в данных: соседний дом, похожая улица, соседний тип происшествия из той же группы, неверная отметка о пострадавших или карточка, отправленная не той службе. Описание при этом всегда говорит правду, поэтому ошибку можно найти, сверив поля с описанием. Фоновый шум в голосе собеседника растёт с уровнем сложности.

Вал карточек. На сложности 3 в режиме реагирования карточки приходят по расписанию: первая сразу, каждая следующая через случайные 40–50 секунд, всего три. Расписание своё у каждого обучающегося и воспроизводимо. Повторный вызов о том же происшествии приходит через 1–2 минуты после исходной карточки и входит в те же три; в режиме ИИ повторных вызовов нет.

«Машин нет» и осложнение. На сложности 3 карточки обучающегося по очереди получают одно из двух усложнений, первым — «машин нет». В первом случае бригада не выезжает, пока диспетчер не доложит руководителю организации и тот не снимет машину с планового выезда. Во втором через 25 секунд после прибытия звонит специалист бригады и просит вызвать другую службу из карточки; доклад о завершении работ ждёт, пока диспетчер не позвонит этой службе.

3. Задания формирует ИИ

В этом режиме преподаватель не выбирает сценарии: каждому обучающемуся модель пишет свои карточки во время занятия.

1. **Служба.** Каждому обучающемуся назначается служба ДДС из выбранных в занятии (ЖКХ, МОЭК, Мосгаз, Мосводоканал, Гормост), без выбора — его собственная. Группы происшествий берутся только те, о которых классификатор оповещает эту службу.
2. **Что писать.** Если предыдущая карточка не зачтена или в ней пропущена ошибка оператора 112, следующая берётся из той же группы происшествий (другой тип), и пропущенный вид ошибки закладывается первым. Иначе берётся группа с самым низким рейтингом навыка среди ещё не пройденных. Число оповещаемых служб растёт со сложностью. Карточка «не той службе» появляется третьей начиная со сложности 2 и повторяется, если её пропустили. Типы, которые обучающийся уже получал, не повторяются.
3. **Как пишется.** Модель получает тип, адрес, признаки и подходящие фрагменты памятки и пишет по строгой схеме описание, правду о происшествии и эталон. Адрес — настоящая улица из списка улиц, ФИО и телефон заявителя (АОН) подбираются программно, службы и признаки берутся из классификатора, а не у модели. Потом закладываются подвохи по сложности.
4. **Сколько ждать.** Облачной модели даётся 30 секунд, модели на сервере — 120 секунд. Обучающийся видит надпись «ИИ пишет для вас карточку». Если модель не уложилась, ответ не прошёл проверку или модель недоступна, карточка собирается по шаблону, и занятие не останавливается.

5. **Когда.** Первая карточка пишется при начале занятия (в вале — две вперёд), следующая — после оценки предыдущей. В разборе написано, почему выдана именно эта карточка.

Карточки ИИ сохраняются как утверждённые сценарии с отметкой источника и не попадают в библиотеку и в другие занятия.

4. Генерация сценариев для библиотеки

Преподаватель описывает ситуацию фразой. Система подбирает кандидатов в типы происшествия по словам и синонимам (до 12), находит 5 подходящих фрагментов памятки и 3 похожих билета. Поиск идёт по смысловым векторам multilingual-e5, без модели — по частотному методу; индекс строится в памяти. Модель формирует сценарий по строгой JSON-схеме: локальная модель llama.cpp получает схему как грамматику и не может выйти за неё, облачная — как схему ответа. Ответ проверяется схемой и списком кандидатов; при ошибке модели возвращается её ответ с текстом ошибки, всего до трёх попыток, после чего сценарий собирается по шаблону. Признаки, службы, адрес и заявитель берутся из классификатора и запроса, а не у модели. Затем сценарий сверяется со справочниками (раздел 2).

Преподаватель правит сценарий, проверяет грамотность и утверждает его целиком или по частям. Утверждённые части нельзя изменить правкой, а при переделке они переносятся в новую версию без изменений. Сценарии приёма вызова из билетов заказчика подготовлены заранее этим же механизмом (отдельный скрипт).

5. Звонки диспетчера ДДС

В карточке ДДС собеседники такие:

Как дозвониться	Собеседник	Что делает
трубка на вкладке своей службы или номер из справочника	начальник аварийной бригады	выезжает, называет номер наряда, отвечает на вопросы о ходе работ
трубка на вкладке другой службы	дежурный этой службы	принимает информацию, номер наряда не называет
номер из справочника	руководитель организации, начальник отдела реагирования, специалист аварийной бригады	отвечают по своей роли; распоряжение о выезде у них не принимается
номер из карточки, если разрешено в занятии	заявитель	называет правду о том, что оператор 112 записал неверно
входящий звонок	бригада	докладывает о выезде, прибытии, начале и завершении работ

Справочник телефонов ведёт преподаватель: до 10 номеров на службу для пяти служб ДДС. Номер, которого нет в справочнике, не существует.

Ответ собеседника строится правилами вокруг модели. Сначала проверяются особые случаи (торопит, «машин нет», не тот собеседник), затем вопросы о ходе работ, на которые ответ берётся из состояния бригады, затем ответ модели в режиме занятия. После этого правила не дают переспрашивать уже переданные факты и следят за выездом.

Выезд бригады. Начальник бригады выезжает, когда услышал адрес и что случилось: «Принял, выезжаем. Наряд N». До этого номера наряда нет, а на «выезжайте» он спрашивает, чего не хватает. Адрес засчитывается не один в один: улица в любом падеже и порядке слов, допуск на ошибку распознавания, адрес может прозвучать в нескольких фразах подряд. Бригада выезжает и по неверному адресу из карточки, но это стоит баллов. Отсутствие распоряжения на выезд — ошибка в оценке, а не запрет выезда. Номер наряда принадлежит службе: он задан в сценарии или вычисляется из карточки, поэтому один и тот же при каждом звонке.

Доклады бригады запускает сервер по расписанию сценария (по умолчанию, считая от выезда: доклад о выезде через 30 секунд, о прибытии ещё через 45, о начале работ через 30, о завершении через 60). Если начальник бригады уже назвал наряд в звонке, отдельного доклада о выезде нет. Доклад ждёт, пока обучающийся ответит; без ответа за 90 секунд он считается непринятым.

6. Разговоры и режимы ответа

Режим ответа задаётся в занятии и действует и на заявителя, и на собеседников диспетчера ДДС.

- **Предзаписанные реплики** (основной). Модель получает фразу обучающегося и пронумерованный список утверждённых реплик и возвращает только номер; ответ ограничен схемой, где допустимы только номера списка или «нет подходящей». Выбор проверяется ключевыми словами: если они однозначно указывают на другую тему, побеждают они. Текст не генерируется, поэтому собеседник не скажет того, чего преподаватель не утверждал.
- **Свободная генерация речи (без интернета).** Модель на сервере отвечает своими словами (не длиннее 15 слов), опираясь только на лист фактов. Ответ проходит фильтры: попытки вывести модель из роли, повтор фразы обучающегося, повтор своей прошлой реплики. Новые реплики сохраняются в сценарий как черновики на утверждение преподавателю.
- **Свободная генерация речи (с интернетом).** Разговор целиком ведёт облачный голосовой сервис (Vapi): распознавание, языковая модель и голос. Сервер передаёт ему роль, факты и номер наряда и сам запускает звонки с докладами. Во время звонка сервер в разговор не вмешивается: после звонка он читает расшифровку, а для звонка начальнику бригады — анкету, которую облако заполняет по итогам (назван ли адрес и дом, что случилось, пострадавшие, доступ, дано ли распоряжение, исправлен ли дом). По ним сервер решает, выехала ли бригада, и оценивает звонок. Если облако недоступно, звонок продолжает локальная модель в режиме предзаписанных реплик.

Если модель диалога недоступна, ответ подбирается по ключевым словам; без модели вообще в приёме вызова появляются кнопки тем.

Темы вопросов в приёме вызова определяются двумя способами сразу: ключевыми словами во фразе оператора и темой реплики, которую выбрала модель. Это защищает оценку обязательных вопросов от ошибок модели. В облачном звонке темы считаются только по словам оператора, сказанное заявителем без вопроса в зачёт не идёт.

7. Речь

Распознавание. faster-whisper на процессоре (int8), модель `base` или `small`. Вместе с фразой передаётся подсказка из слов карточки: улица, район, город и частые термины, — это снижает ошибки на адресах. В браузере фраза записывается целиком по кнопке. В звонке через Asterisk

поток режется на фразы детектором речи Silero VAD (без его файла — по громкости), каждая фраза распознаётся отдельно.

Синтез. Голоса берутся в таком порядке:

1. **Студийные записи.** Реплики эталонных сценариев и постоянные фразы собеседников ДДС (начальник бригады, дежурные служб, бригада, специалист, руководство, заявитель) записаны заранее сервисом ElevenLabs и входят в поставку готовыми файлами. Фраза ДДС находится по своему тексту, поэтому совпавшая фраза звучит студийным голосом и в живом разговоре. Во время работы ElevenLabs не вызывается.
2. **Silero** (основной синтез на сервере, 5 голосов). Числа перед синтезом переводятся в слова, длинные номера читаются по цифрам.
3. **Piper**, если модели Silero нет.
4. Только текст, если нет ни одного движка.

Утверждённые реплики озвучиваются фоновой задачей при утверждении сценария. Всё, чего нет в готовых файлах (сгенерированные ответы, фразы с номером наряда и адресом, доклады), синтезируется во время разговора и кэшируется. Голос собеседника выбирается по полу и возрасту из сценария; у каждой роли ДДС свой голос.

8. Оценка действий

Оценка построена как набор независимых составляющих. Каждая даёт баллы и объяснение, итог приводится к ста баллам. Зачёт — итог не ниже порога занятия (по умолчанию 70) и нет критической ошибки. Норматив времени берётся из занятия (по умолчанию 30 секунд для карточки ДДС и 60 для приёма вызова).

Режим «Реагирование на карточку»:

Составляющая	Вес	Что проверяет
Решение	30	принял или не принял карточку, совпадает ли причина отказа с эталонной
Время принятия	20	время от поступления карточки до первичного статуса
Цепочка статусов	20	поставлены ли эталонные статусы и в правильном ли порядке
Комментарии и наряд	15	обязательные комментарии, их близость к эталону по смыслу, номер наряда
Типичные ошибки	10	срабатывание правил памятки
Грамотность	5	ошибки в комментариях по LanguageTool
Проверка данных	20	найдены ли ошибки оператора 112 в карточке
Звонки в службы	15	дозвонился ли до нужных людей, передал ли факты, дал ли распоряжение, уложился ли в норматив

Проверка данных учитывается, только если в карточку заложены подвохи, звонки — только если они есть в эталоне. Применимые веса пересчитываются к 100.

Режим «Приём вызова»: опросная карта (25), признаки и службы (10), адрес (15), обязательные вопросы (15), описание (10), время (10), типичные ошибки (5), грамотность (10).

Как считаются отдельные вещи.

- *Время.* Полный балл, если уложился в норматив; ноль, если превысил вдвое; между ними линейно. Опоздание к тому же меняет статус карточки на «Не оповещено» и даёт штраф по типичным ошибкам.
- *Решение.* Неверная причина отказа — половина балла. Ошибочное «Принята», исправленное отказом с комментарием, теряет часть балла, как в памятке.
- *Цепочка статусов.* Доля эталонной цепочки, которая есть у обучающегося в том же порядке (наибольшая общая подпоследовательность).
- *Проверка данных.* Каждая заложенная ошибка — равная доля: найдена — полная, найдена с неверным исправлением — половина, пропущена — ноль, каждая ложная отметка снимает половину доли. Карточка «не той службе» считается найденной, только если её отклонили с причиной «не в компетенции».
- *Звонки.* Для каждого эталонного звонка: дозвонился, передал обязательные факты (адрес, что случилось, пострадавшие), уложился в норматив звонка (120 секунд); для начальника бригады ещё распоряжение и номер наряда. Звонок без единого обязательного факта — ноль. Факты берутся из расшифровки, а в облачном режиме — из анкеты после звонка; адрес засчитывается, только если названы улица и дом карточки.
- *Текст.* Ответ сравнивается с эталоном по смыслу: тексты переводятся в векторы моделью multilingual-e5, считается косинусная близость (пороги 0,85 и 0,88 подобраны замером). Если модель недоступна, используется частотный метод с порогами 0,30 и 0,70. В результате указано, каким способом считали.
- *Адрес и названия.* Приводятся к общему виду: регистр, буква «ё», типы улиц и сокращения, числительные словами; затем сравниваются с допуском на опечатку (улица — от 90 из 100).
- *Типичные ошибки.* Каждое правило — отдельная функция со штрафом и ссылкой на страницу памятки: 24 правила для карточки ДДС (например, первичный статус позже норматива, отказ без комментария, пропущенные статусы хода работ, отказ от профильного происшествия, не передано осложнение, не доложено об отсутствии машин, распоряжение не дано) и 5 для приёма вызова. Каждое правило покрыто тестом.
- *Недоступный сервис.* Если LanguageTool не отвечает, грамотность помечается «не проверено», а итог пересчитывается по остальным составляющим, чтобы обучающийся не терял баллы из-за инфраструктуры.
- *Блокирующие правила.* Две ошибки в приёме вызова критические: при них итог «незачёт» при любом балле. **Обязательные вопросы не заданы** — обучающийся сам задал меньше половины обязательных вопросов (доля задаётся в сценарии полем `min_questions_share`, по умолчанию 0,5). **Карточка не заполнена** — нет ни типа происшествия, ни признаков, ни адреса, ни описания. В разборе причина видна текстом: «Незачёт: задано 0 из 7 обязательных вопросов (нужно не меньше 4)». В карточке ДДС критической правило становится, если так отмечено в эталоне сценария.
- *Пустой ввод.* Составляющая, которой нечего оценивать, получает 0, а не максимум: описание короче двух слов даёт 0 по описанию и по грамотности; незаполненная карточка даёт 0 ещё по признакам и службам и по времени. Пустая карточка после полноценного разговора даёт около 15 баллов и незачёт.

Советы. После каждой карточки разбор даёт до пяти советов: сначала по пропущенным ошибкам оператора 112, затем по найденным ошибкам от критических и самых дорогих, со ссылкой на

страницу памятки. После занятия — до пяти советов по занятию: повторяющиеся ошибки, доля найденных подвохов, динамика балла. Советы собираются по правилам, без модели, поэтому каждый можно объяснить.

Сводка по валу. После вала отдельно показывается, сколько карточек пришло, сколько закрыто, сколько принято с опозданием и без реакции, взята ли первой карточка ЧС. Баллы карточек сводка не меняет.

Исправление оценки. Преподаватель может изменить итог закрытой попытки, указав причину. Зачёт пересчитывается по порогу занятия, прежняя оценка сохраняется, изменение пишется в журнал аудита и видно обучающемуся.

Оценка не зависит от базы данных и внешних вызовов: это чистые функции, поэтому её поведение воспроизводимо и проверяется тестами. Сквозная проверка на живых звонках с облачным заявителем описана в VOICE_TESTS.md.

9. Аналитика и прогноз

Рейтинг навыка считается по каждой группе происшествий и режиму по схеме Эло: результат попытки сравнивается с ожидаемым для текущего рейтинга и сложности карточки, рейтинг растёт сильнее за успех на сложной карточке.

Адаптивный подбор в обычном занятии ставит первыми непройденные карточки, затем из группы с самым низким рейтингом и по сложности, ближайшей к уровню обучающегося. В режиме ИИ рейтинг выбирает группу для следующей карточки (раздел 3).

Прогноз готовности к аттестации — вероятность сдать аттестацию. Модель — логистическая регрессия с калибровкой по восьми признакам: средний балл последних попыток, направление изменения, доля времени от норматива, неверные решения, пропущенные обязательные вопросы, частота типичных ошибок, самый низкий рейтинг по группам, число попыток. Модель простая и объяснимая, поэтому для каждого прогноза названы два признака, которые сильнее всего на него влияют.

Проверка достоверности. Реальных аттестаций у системы пока нет, поэтому модель обучена на виртуальной когорте из 300 обучающихся с разной историей; итог аттестации каждого — средний результат скрытых попыток. Когорта делится на обучающую и проверочную части по обучающимся. На проверочной части считаются ROC AUC (0,93), ошибка Брайера (0,11 против 0,25 у постоянного прогноза) и калибровка по десяти интервалам. Все числа и оговорка про виртуальную когорту публикуются на странице «Достоверность прогноза». Когорту можно пересоздать и переобучить модель скриптом; когда накопятся настоящие аттестации, модель нужно переобучить на них.

5. Установка, настройка и восстановление

Документ описывает установку на сервер с интернетом и в закрытом контуре без интернета, первый вход, настройки, телефонию, облачные режимы, резервное копирование, восстановление и обновление. Все команды вводятся в терминале сервера в папке установки.

1. Требования

Сервер:

- процессор от 6 ядер (Intel Core i7, Xeon или аналог), видеокарта не нужна;
- оперативная память от 16 ГБ, рекомендуется 32 ГБ (минимум по техническому заданию);
- диск SSD, свободно от 100 ГБ: программа с моделями занимает около 40 ГБ, остальное — база, записи разговоров и резервные копии; техническое задание рекомендует NVMe от 512 ГБ;
- Linux: Ubuntu 22.04 или 24.04;
- Docker Engine и docker compose версии 2.

Скрипты установки (`start.sh` , `install.sh`) рассчитаны на Linux. На Windows и macOS они не запускаются; систему можно поднять вручную через Docker Desktop для разработки и показа, но как сервер учебного класса этот вариант не проверялся.

Без моделей искусственного интеллекта (профиль `ai` выключен) система работает и на машине с 8 ГБ памяти: собеседники отвечают по ключевым словам заранее записанными репликами, речь обучающегося не распознаётся, он пишет текстом.

Рабочие места: браузер Chrome, Яндекс.Браузер или Firefox, гарнитура с микрофоном. Ставить на рабочие места ничего не нужно. Настольный IP-телефон подключается дополнительно (раздел 7).

Сеть: рабочие места должны видеть сервер по HTTPS (порт 443; порт 80 только перенаправляет на 443). Для телефонии дополнительно открываются порт SIP (5060, UDP и TCP) и диапазон голосовых портов (по умолчанию 10000–10100, UDP). Служебные порты (база, Redis, модели, распознавание, управление телефонией) по умолчанию слушают только сам сервер (`INTERNAL_BIND=127.0.0.1`) и снаружи не видны.

2. Сервер с интернетом

```
git clone https://github.com/Vadik145-beep/lct-112-simulator.git
cd lct-112-simulator
./start.sh                      # или ./start.sh --host 192.168.1.50
```

Что делает `start.sh` :

1. Проверяет, что Docker установлен, запущен и есть docker compose v2.
2. Скачивает модели (около 9 ГБ) одним архивом с Google Диска команды и сверяет контрольную сумму. Если связь оборвалась, повторный запуск докачивает с того же места. Если архив недоступен или повреждён, модели скачиваются по одной с официальных источников (`scripts/fetch_models.sh` : Hugging Face и models.silero.ai). Если модели уже на месте, шаг пропускается.

3. Если файла `.env` ещё нет, создаёт его из `.env.example` с настройками демонстрационного стенда: свой случайный `SECRET_KEY`, адрес сервера в сертификате (`TLS_HOSTS`) и в телефонии (`TELEPHONY_EXTERNAL_IP`), `APP_ENV=production`, модель собеседников Qwen2.5-3B с контекстом 16384, написание карточек той же моделью (`LLM_GEN_URL=http://llm-dialog:8080`), распознавание `faster-whisper-small`, телефония включена, логины и вход в один клик на странице входа (`QUICK_LOGIN=true`, `SHOW_DEMO_CREDENTIALS=true`). Уже существующий `.env` скрипт не трогает.
4. Собирает и запускает систему с профилями `ai` и `telephony`; отдельный сервер модели 7B (`llm-gen`) не запускается (`--scale llm-gen=0`), его работу делает модель собеседников.
5. Ждёт, пока сервер приложения станет «healthy», и печатает адрес, логины и пароль.

Адрес сервера скрипт определяет сам (`hostname -I`); если у сервера несколько адресов, укажите нужный через `--host`. Первый запуск занимает от 30 минут до часа (скачивание моделей и сборка образов), повторный — несколько минут.

Без `start.sh` (свои настройки с самого начала):

```
cp .env.example .env            # поменять SECRET_KEY, пароли и остальное (раздел 5)
./scripts/fetch_models.sh       # модели, около 9 ГБ
docker compose --profile ai --profile telephony up -d --build
```

Значения по умолчанию в `.env.example` отличаются от стенда: модель собеседников Qwen2.5-1.5B, карточки пишет отдельный сервер с моделью Qwen2.5-7B (`llm-gen`), распознавание `faster-whisper-base`, телефония выключена, логины на странице входа не показываются.

3. Сервер без интернета (закрытый контур)

Сборка образов требует интернета (пакеты Python и npm, базовые образы), поэтому поставка собирается на машине с интернетом и переносится в контур готовой. В контуре ничего не скачивается и не собирается.

На машине с интернетом (Linux, Docker, Git, около 25 ГБ свободного места):

```
git clone https://github.com/Vadik145-beep/lct-112-simulator.git
cd lct-112-simulator
./scripts/fetch_models.sh                       # или models.tar с Google Диска, как в start.sh
./scripts/offline_bundle.sh --with-models       # результат — папка ./offline-bundle (около 11 ГБ)
```

Что попадает в поставку:

Файл	Что внутри
`images.tar`	все образы всех профилей (`ai`, `telephony`, `cloud`), собранные и скачанные на этой машине
`source.tar`	исходники репозитория без истории git
`models.tar`	папка моделей (только с `--with-models`)

Файл	Что внутри
`organizers.tar`	материалы заказчика из `data/organizers`, если папка есть
`install.sh`	установщик
`SHA256SUMS`, `VERSION`	контрольные суммы и версия сборки

Без `--with-models` поставка занимает около 2 ГБ и ставится без профиля `ai` (раздел 1, работа без моделей). Другая папка вывода задаётся `--out`.

На сервере в контуре нужны только Docker Engine и docker compose v2. Их ставят заранее из пакетов дистрибутива: в поставку Docker не входит. Папка `offline-bundle` переносится на сервер флешкой, диском или по локальной сети.

```
cd offline-bundle
sudo ./install.sh                                  # в /opt/lct/app, как на
стенде
sudo ./install.sh --dir /srv/trainer --host 10.0.0.5
sudo ./install.sh --profile telephony              # только выбранные профили
```

Установщик:

1. проверяет контрольные суммы всех файлов поставки;
2. загружает образы (`docker load`);
3. распаковывает исходники, материалы заказчика и модели в папку установки (по умолчанию `/opt/lct/app`);
4. если `.env` ещё нет, создаёт его так же, как `start.sh` (раздел 2, пункт 3); телефония включается, только если поднимается профиль `telephony`;
5. поднимает систему с `--pull never --no-build`: любая попытка что-то скачать завершилась бы ошибкой, а не тихим обращением в сеть. Без `--profile` поднимаются `ai` (если в поставке есть модели) и `telephony`;
6. ждёт готовности и печатает адрес, логины и пароль.

При первом запуске сервер приложения сам применяет миграции базы, загружает классификатор, памятку, учебные билеты и справочник улиц, создаёт пользователей и раскладывает записи голосов.

Образы собранных сервисов имеют постоянные имена (`lct112/backend`, `lct112/nginx`, `lct112/backup`, `lct112/stt`, `lct112/asterisk`, `lct112/asterisk-cloud`; тег `APP_VERSION`, по умолчанию `latest`), поэтому установка не зависит от имени папки.

Проверено 22.09.2026: поставка без моделей (1,8 ГБ образов) развёрнута в чистую папку на сервере с удалёнными образами и `--pull never`; журнал Docker за время установки не содержит ни одного обращения к реестру; `/health` отвечает ok, страница входа открывается, сквозной тест «полный проход» ДДС проходит. Установка поставки с моделями на чистую машину без сети целиком не прогонялась.

4. Первый вход и проверка

Откройте адрес, который напечатал скрипт, например `https://192.168.1.50`. Браузер предупредит о сертификате: у системы свой центр сертификации. Нажмите «Дополнительные» → «Перейти на сайт». Чтобы предупреждения не было, установите на рабочие места корневой сертификат:

```
docker compose cp nginx:/etc/nginx/certs/ca.crt .
```

Сертификат выпускается при первом запуске на адреса из `TLS_HOSTS` и действует 825 дней. Если адрес сервера сменился, поправьте `TLS_HOSTS` и выпустите сертификат заново (корневой сертификат при этом не меняется):

```
docker compose exec nginx rm /etc/nginx/certs/server.crt
/etc/nginx/certs/server.key
docker compose restart nginx
```

Пользователи после установки (`DEMO_MODE=false`):

Роль	Логин	Пароль
Администратор	`admin`	`112112112`
Преподаватель	`teacher`	`112112112`
Обучающийся	`student`	`112112112`
Обучающийся	`student2`	`112112112`

Пароль задаётся настройкой `SEED_PASSWORD` и действует только при первом запуске: пользователи создаются один раз, и смена `SEED_PASSWORD` потом их пароли не меняет. Менять пароль при первом входе не требуется. Обучающимся назначена служба «Гор. хозяйство» (ЕДЦ ЖКХ). Групп и занятий после установки нет: их создаёт преподаватель. Режим оператора 112 у преподавателя и обучающихся скрыт, администратор включает его отдельным пользователям галочкой «Режим 112».

Скрипты установки включают на странице входа таблицу логинов с паролем и кнопки «Войти» в один клик, как на демонстрационном стенде. Пока таблица включена, показанные в ней пароли сменить нельзя. На рабочем сервере выключите оба параметра и перезапустите систему (раздел 5), затем смените пароли в разделе администратора «Пользователи»:

```
QUICK_LOGIN=false
SHOW_DEMO_CREDENTIALS=false
```

`DEMO_MODE=true` (до первого запуска) вместо этого создаёт демонстрационный набор: `teacher1`, `teacher2`, `student1` … `student12`, две группы и демо-занятия. Если строки `DEMO_MODE` в `.env` нет совсем, docker compose подставит `true`, поэтому строку не удаляйте.

Проверка после установки:

1. `https://<адрес>/health` отвечает `"status":"ok"`.
2. `docker compose ps` — все сервисы `running` или `healthy`.
3. Администратор → «Состояние»: все плитки сервисов «работает».

4. Преподаватель создаёт группу, добавляет в неё `student` и создаёт занятие.
5. Обучающийся в «Моих заданиях» нажимает «Открыть журнал АРМ-112» — в журнале первая карточка.

5. Настройки

Почти всё задаётся в файле `.env` в папке установки; образец с пояснениями — `.env.example`. После правки систему перезапускают той же командой, что и запускали, например:

```
docker compose --profile ai --profile telephony up -d --scale llm-gen=0
```

Часть настроек администратор меняет в интерфейсе, в разделе «Настройки»: ожидание ответа на вызов, запись разговоров, телефоны стажёров, расписание и число копий, порог «Не завершено», уровень журнала. Изменения записываются в журнал аудита.

5.1. Общие и безопасность

Настройка	Что задаёт
`SECRET_KEY`	Секрет для подписи входа, не короче 32 символов. Скрипты установки генерируют свой; вручную: `openssl rand -hex 32`
`POSTGRES_PASSWORD`, `APP_DB_PASSWORD`	Пароли владельца базы и роли приложения. Поменять **до первого запуска**: потом они уже записаны в базу. Вместе с ними поправить пароли в `DATABASE_URL` и `DATABASE_ADMIN_URL`
`SEED_PASSWORD`	Пароль пользователей, создаваемых при первом запуске, по умолчанию `112112112`
`APP_ENV`	`production` для рабочей установки (без подробных ошибок)
`DEMO_MODE`	Демонстрационный набор пользователей и занятий, по умолчанию `false`
`QUICK_LOGIN`, `SHOW_DEMO_CREDENTIALS`	Кнопки «Войти» без пароля и таблица логинов на странице входа. На рабочем сервере `false`
`ALLOW_EXTERNAL_AI`	Разрешить внешние ИИ-сервисы (раздел 8). В закрытом контуре `false`
`TLS_HOSTS`	Адреса и имена сервера для сертификата, через запятую
`HTTP_PORT`, `HTTPS_PORT`	Порты веб-интерфейса, по умолчанию 80 и 443
`INTERNAL_BIND`	На каком адресе слушают служебные порты. `127.0.0.1` — только сам сервер
`ACCESS_TOKEN_MINUTES`, `REFRESH_TOKEN_DAYS`	Время жизни входа: 15 минут с автоматическим продлением до 7 дней
`LOGIN_MAX_ATTEMPTS`, `LOGIN_LOCK_MINUTES`	Блокировка входа после неудачных попыток: 5 попыток, 15 минут
`TZ`	Часовой пояс интерфейса и отчётов, по умолчанию `Europe/Moscow`
`LOG_LEVEL`	Уровень журнала сервера, по умолчанию `INFO`

Настройка	Что задаёт
`BACKUP_TIME`, `BACKUP_KEEP`	Время ежедневной копии (03:00) и сколько копий хранить (14)

5.2. Модели ИИ

Настройка	Что задаёт
`LLM_DIALOG_MODEL`	Модель собеседников: `qwen2.5-1.5b-...` (по умолчанию) или `qwen2.5-3b-...` (стенд, скрипты установки)
`LLM_DIALOG_CTX`, `LLM_DIALOG_SLOTS`	Контекст и число одновременных разговоров сервера модели: 8192 и 4. Если карточки пишет модель собеседников, нужен контекст 16384
`LLM_GEN_URL`	Кто пишет карточки и сценарии: `http://llm-gen:8080` (отдельная 7B), `http://llm-dialog:8080` (модель собеседников) или облачная модель (раздел 8)
`LLM_GEN_MODEL`, `LLM_GEN_CTX`	Модель и контекст отдельного сервера генерации
`LLM_GEN_TIMEOUT_SECONDS`	Сколько ждать сценарий от модели в библиотеке преподавателя, по умолчанию 900 с
`DIALOG_MODE`	Режим разговора по умолчанию: `select` — модель выбирает утверждённую реплику
`STT_MODEL`	Распознавание речи: `faster-whisper-base` быстрее, `faster-whisper-small` точнее
`TTS_PROVIDER`	Голос собеседников для фраз без готовой записи: `auto` — Silero, без него Piper, без голосов — только текст
`LLM_THREADS`	Сколько ядер процессора отдать каждому серверу модели и распознаванию, по умолчанию 4
`LLM_LOAD_MODE`	`auto` на Linux; `mlock` на Docker Desktop под Windows
`MODELS_DIR`	Папка с моделями на сервере, по умолчанию `./models`

6. Модели искусственного интеллекта

Модели скачиваются один раз: `start.sh` делает это сам, вручную — `./scripts/fetch_models.sh`. Скрипт сверяет размер и контрольную сумму каждого файла и докачивает оборванные; отдельную часть можно скачать ключом `--only llm|stt|tts|embeddings`. Перечень файлов — в `scripts/models.manifest`, лицензии — в `docs/LIBRARIES.md`.

Что	Файлы	Для чего
Qwen2.5-1.5B, Qwen2.5-3B, Qwen2.5-7B (GGUF Q4_K_M)	`models/llm`	собеседники по телефону, написание карточек и сценариев
faster-whisper base и small	`models/stt`	распознавание речи обучающегося
Silero TTS v4, Piper (три голоса)	`models/tts`	голос собеседников для фраз без готовой записи

Что	Файлы	Для чего
multilingual-e5-small	`models/embeddings`	смысловое сравнение ответов с эталоном при оценке
Silero VAD	`models/vad`	границы фраз в телефонном звонке

Голос Silero скачивается с models.silero.ai. Если этот адрес закрыт, скачайте файл на другой машине и положите в `models/tts/silero_v4_ru.pt`.

Фразы учебных сценариев, дежурных служб и бригады из поставки уже озвучены заранее и поставляются записями вместе с данными; синтез на сервере нужен только для новых и изменённых фраз и для режима свободной генерации.

Лицензии: Qwen2.5-3B и Silero TTS распространяются по некоммерческим лицензиям. Для учебного контура заказчика это допустимо; при коммерческом использовании модель собеседников меняется на 1.5B или 7B (`LLM_DIALOG_MODEL`), голос — на Piper (`TTS_PROVIDER=piper`).

7. Телефония

Звонки работают в браузере сразу после установки: обучающийся говорит в гарнитуру, речь распознаётся на сервере, собеседник отвечает голосом. Ставить на рабочие места ничего не нужно.

Для настоящих телефонов в поставку входит своя станция Asterisk (профиль `telephony`); она подключает настольные IP-телефоны и линию оператора связи или АТС заказчика. В занятиях с галочкой «Живой звонок на телефон» вызовы звонят одновременно в софтфон браузера, на настольный IP-телефон обучающегося и на мобильный через линию оператора связи. Галочка видна в форме занятия в режиме оператора 112 с облачным голосом (раздел 8.2), когда включены телефония и линия оператора связи (транк ниже или МультиФон облачного контура).

```
docker compose --profile telephony up -d
```

В `.env` задать `TELEPHONY_ENABLED=true`, внешний адрес сервера `TELEPHONY_EXTERNAL_IP` (без него звук не пойдёт в одну сторону), диапазон голосовых портов `TELEPHONY_RTP_START` и `TELEPHONY_RTP_END`, порт SIP `SIP_PORT`, пароль управления `ARI_PASSWORD`. Скрипты установки включают телефонию и подставляют адрес сервера сами; без линии оператора связи она просто не используется.

На межсетевом экране открыть порт SIP (по умолчанию 5060, UDP и TCP) и диапазон голосовых портов (UDP).

Софтфон в браузере настраивается автоматически: учётная запись создаётся вместе с пользователем, данные приходят при входе.

Настольный IP-телефон (например, РТУ Т16Р): в настройках аппарата указать адрес сервера, логин и пароль из кабинета администратора (меню «⋯» у обучающегося → «SIP-учётка»), кодеки G.722 и PCMA, регистрация по UDP или TCP. Шифрование SIP (TLS, SRTP) не поддерживается.

Если телефония выключена (`TELEPHONY_ENABLED=false`), занятия работают полностью, кроме живого звонка на телефон.

Внешний SIP-транк оператора связи (необязательно)

Транк подключают, чтобы вызовы занятия звонили на настоящие телефоны и чтобы обучающийся мог сам позвонить на номер линии. Подробности — `docs/TRUNK.md`.

В `.env` задать данные учётной записи оператора связи:

```
TELEPHONY_TRUNK_USER=       # номер или логин у оператора
TELEPHONY_TRUNK_PASSWORD=   # пароль SIP
TELEPHONY_TRUNK_DOMAIN=     # домен регистрации, например multifon.ru
TELEPHONY_TRUNK_PROXY=      # узел оператора, например sbc.megafon.ru:5060
TELEPHONY_TRUNK_CODECS=alaw,ulaw
TELEPHONY_TRUNK_INBOUND=    # логин обучающегося, которому идут все входящие;
пусто = входящие разбирает сервер
```

Перезапустить телефонию и сервер приложения: `docker compose --profile telephony up -d --force-recreate asterisk backend`.

Проверка: `docker compose exec asterisk asterisk -rx "pjsip show registrations"` должна показать состояние `Registered`. Номер обучающегося для звонков задаётся в форме занятия, в профиле обучающегося или администратором в «Настройки → Телефония» строками `логин = телефон`.

Если поля пустые, транк не создаётся. Ограничения оператора учитывать отдельно: у «МультиФон Бизнес» МегаФона не больше трёх одновременных регистраций на учётную запись и голосовые порты 10000–27999 на стороне оператора.

8. Облачные режимы (только вне закрытого контура)

По умолчанию облачные режимы выключены, а все ключи в `.env.example` пустые: система не обращается в интернет. Облако включается только явно, на сервере с интернетом, своими ключами. Если внешний адрес модели или облачный голос указаны без `ALLOW_EXTERNAL_AI=true`, сервер приложения не запустится и напишет в журнале, какая настройка внешняя.

8.1. Облачная модель для карточек

Карточки режима «Задания формирует ИИ» и сценарии в библиотеке может писать облачная модель с API, совместимым с OpenAI. Облачная модель пишет карточку за несколько секунд, локальная на процессоре — около минуты (раздел «Производительность»).

```
ALLOW_EXTERNAL_AI=true
LLM_GEN_URL=https://<адрес API без /v1>
LLM_GEN_API_KEY=<ключ сервиса>
LLM_GEN_API_MODEL=<имя модели у сервиса>
```

Ключ без имени модели — ошибка запуска. Если облачная модель не ответила за 30 секунд (локальная — за 120), карточку собирает шаблон, и занятие не останавливается.

8.2. Облачный голос (Vapi)

В занятии с режимом «Свободная генерация речи (с интернетом)» собеседников играет облачный сервис Vapi: распознавание речи, языковая модель и голос ElevenLabs. Разговор идёт из браузера обучающегося; с живым звонком на телефон — через отдельный Asterisk облачного контура.

Что нужно:

- сервер с интернетом и доменным именем, которое видно из интернета (Vapi присылает события разговора на адрес сервера);
- сертификат Let's Encrypt для этого имени;
- аккаунт Vapi с приватным и публичным ключом (Integrations → API keys).

Настройки в `.env`:

```
ALLOW_EXTERNAL_AI=true
CLOUD_VOICE_ENABLED=true
VAPI_API_KEY=<приватный ключ>
VAPI_PUBLIC_KEY=<публичный ключ>
PUBLIC_HOST=<доменное имя сервера>
CLOUD_VOICE_PUBLIC_URL=https://<доменное имя сервера>
```

Сертификат выпускается проверкой через порт 80: nginx отдаёт папку проверок из тома `certbot-www` и подключает сертификат из тома `letsencrypt`. Имена томов начинаются с имени проекта compose (`docker volume ls`), например:

```
docker run --rm -v <проект>_certbot-www:/var/www/certbot -v
<проект>_letsencrypt:/etc/letsencrypt \
  certbot/certbot certonly --webroot -w /var/www/certbot -d <доменное имя> --
agree-tos -m <почта>
docker compose up -d --force-recreate nginx backend
```

При `CLOUD_VOICE_ENABLED=true` nginx сам добавляет адреса Vapi и Daily в политику безопасности страницы. Модель, голос и распознавание внутри Vapi задаются `CLOUD_VOICE_MODEL`, `CLOUD_VOICE_VOICE_ID…`, `CLOUD_VOICE_TRANSCRIBER_…` (значения по умолчанию — в `.env.example`). Если облако не ответило за `CLOUD_VOICE_ANSWER_SECONDS` (15 с) или звонок из браузера не поднялся, разговор продолжается локально, текстом и микрофоном.

МультиФон облачного контура. Чтобы облачные звонки шли и на мобильные телефоны стажёров, вместо профиля `telephony` поднимают профиль `cloud` (два профиля вместе не поднимать) и задают `MULTIFON_USER`, `MULTIFON_PASSWORD`; на вход открываются UDP `CLOUD_SIP_PORT` (5061) и `CLOUD_RTP_START` – `CLOUD_RTP_END` (10200–10300). Подробности — `docs/MULTIFON.md`.

9. Резервное копирование и восстановление

Сервис `backup` каждый день в `BACKUP_TIME` делает копию базы (`pg_dump`, файл `trainer-ГГГГММДД-ЧЧММСС.dump` в томе `backups`) и хранит последние `BACKUP_KEEP` копий, более старые удаляет. Копию можно сделать вручную в кабинете администратора («Копии» → «Сделать копию сейчас»), там же меняется расписание: служба подхватывает его в течение минуты.

Копия содержит базу: пользователей, занятия, результаты, сценарии, журнал аудита. Записи разговоров, озвученные реплики и загруженные методические материалы лежат в томе `storage` и

в копию базы не входят. Обе части хранятся на том же сервере; чтобы пережить отказ диска, их копируют на другой носитель:

```
docker compose cp backup:/backups ./backups-copy
docker compose cp backend:/storage ./storage-copy
```

Проверка восстановления (рабочая база не меняется):

```
./scripts/restore_test.sh
```

Скрипт делает свежую копию, разворачивает её в отдельную базу `trainer_restore_test`, проверяет вход администратора кодом приложения и удаляет проверочную базу.

Восстановление из копии:

```
docker compose stop backend worker
docker compose exec -T backup pg_restore --clean --if-exists --no-owner -d trainer
/backups/<имя файла>.dump
docker compose cp ./storage-copy/. backend:/storage      # если восстанавливаются
и записи
docker compose start backend worker
```

`trainer` — имя базы из `POSTGRES_DB`. Чтобы восстановить на новом сервере, сначала установите систему (раздел 2 или 3), затем положите файл копии в том: `docker compose cp <имя файла>.dump backup:/backups/` и выполните команды выше.

10. Обновление

Перед обновлением сделайте копию (раздел 9).

С интернетом:

```
git pull
./start.sh
```

`start.sh` не скачивает модели повторно и не трогает существующий `.env`, а только пересобирает и перезапускает систему.

Без интернета: соберите новую поставку (раздел 3) и запустите `install.sh` с той же папкой установки. Он заменит исходники и образы, а `.env`, база, записи и копии останутся.

Миграции базы применяются автоматически при старте сервера приложения.

Если обновление сменило голосовой движок, уже озвученные синтезом реплики остаются в старом голосе. Переозвучить их текущим движком:

```
docker compose exec worker python -m app.scripts.revoice --dry-run   # сколько
реплик
docker compose exec worker python -m app.scripts.revoice
```

Заранее записанные фразы из поставки и записи, загруженные преподавателем, команда не трогает.

11. Частые проблемы

Симптом	Причина и решение
`docker: command not found` или `permission denied`	Нет Docker или прав: установить Docker, запускать через `sudo`
`port is already allocated`	Порт 80 или 443 занят. В `.env` поставить `HTTP_PORT=8080`, `HTTPS_PORT=8443`, запустить снова, открывать адрес с `:8443`
Первый запуск завершился `dependency failed to start`	На медленной машине база создаётся дольше проверки готовности. Запустить ту же команду ещё раз
Страница не открывается сразу после запуска	Модели ещё загружаются: подождать 2–3 минуты
Браузер предупреждает о сертификате	Установить корневой сертификат (раздел 4)
Сервер приложения не стартует, в журнале «Внешний адрес ИИ-сервиса при ALLOW_EXTERNAL_AI=false»	Указан внешний адрес модели или включён облачный голос без разрешения. Вернуть локальный адрес или включить `ALLOW_EXTERNAL_AI=true` (только вне закрытого контура)
Собеседник отвечает невпопад или по ключевым словам	Не поднят профиль `ai`, модель не скачана или перегружена; в «Состоянии» проверить плитку «Модель диалога». Распознавание путает слова — `STT_MODEL=faster-whisper-small`
Собеседник не слышит обучающегося	Браузеру не дан доступ к микрофону: разрешить в адресной строке, проверить гарнитуру
Карточки ИИ однотипные или приходят через две минуты	Модель не успела, карточку собрал шаблон. Проверить плитку модели генерации; на процессоре это ожидаемо (раздел 8.1)
Всё медленно, модель перезапускается сама	Мало памяти: нужно от 16 ГБ. Не запускать рядом другие программы, уменьшить `LLM_THREADS` или перейти на `STT_MODEL=faster-whisper-base` и модель 1.5B
В оценке «грамотность не проверена»	Не отвечает LanguageTool: `docker compose restart languagetool`; остальная оценка работает
Настольный телефон не звонит	Телефон звонит только в занятиях с живым звонком (раздел 7). Проверить `TELEPHONY_ENABLED=true`, профиль `telephony`, порт SIP и голосовые порты
В «Копиях» служба копий не отвечает	`docker compose up -d backup`
В «Состоянии» плитка «не отвечает»	`docker compose ps` покажет, какой сервис остановлен; `docker compose up -d` с теми же профилями поднимет его

Журнал сервиса: `docker compose logs --tail 100 backend`.

6. Руководство пользователя

Краткий обзор: кто работает в тренажёре, какие у него экраны и как проходит занятие. Подробные инструкции — в четырёх отдельных документах (PDF в папке `docs/submission`, исходный текст в `docs/guides`):

Документ	Для кого
«Ультратех_задача9_1_Общее_техническое_описание.pdf» (guides/TECH_DESCRIPTION.md)	назначение, функции, установка, настройка, неисправности
«Ультратех_задача9_2_Руководство_администратора.pdf» (guides/ADMIN_GUIDE.md)	администратор
«Ультратех_задача9_3_Руководство_преподавателя.pdf» (guides/TEACHER_GUIDE.md)	преподаватель
«Ультратех_задача9_4_Руководство_обучаемого.pdf» (guides/STUDENT_GUIDE.md)	обучаемый, диспетчер ДДС

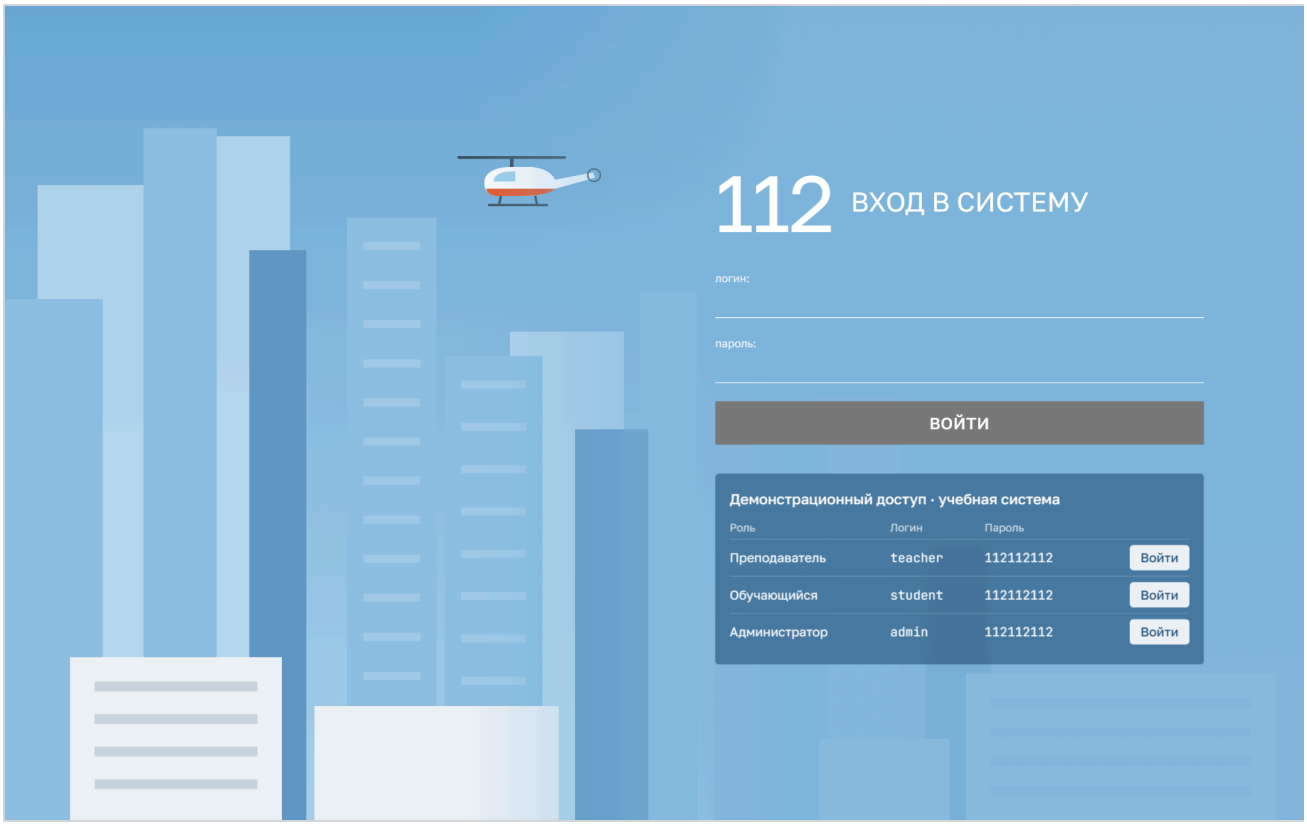
1. Роли

Роль	Что делает	Меню
Обучаемый (в интерфейсе «Обучающийся»)	Работает диспетчером ДДС на копии АРМ-112: принимает карточки, звонит по виртуальному телефону, ведёт статусы, читает разбор и советы	«Мои задания», «Прогресс», «Справочник»
Преподаватель	Собирает группы, создаёт и проводит занятия, наблюдает, смотрит отчёты и аналитику, ведёт справочник телефонов, сценарии и методические материалы	«Занятия», «Группы», «Аналитика», «Сценарии», «Справочник телефонов»
Администратор	Заводит пользователей, следит за состоянием системы, журналом аудита, резервными копиями и настройками. Оценки, занятия и сценарии не меняет	«Пользователи», «Службы», «Состояние», «Аудит», «Копии», «Настройки»

Карточки обучаемому пишет ИИ, если преподаватель включил это в занятии. ИИ же играет собеседников по телефону; оценку и советы система строит автоматически по правилам памятки заказчика.

2. Вход

Откройте адрес системы в браузере (Chrome, Яндекс.Браузер или Firefox). На странице «112 ВХОД В СИСТЕМУ» введите логин и пароль и нажмите «ВОЙТИ». Для звонков нужна гарнитура с микрофоном.



Страница входа. Таблица демонстрационного доступа видна только на демонстрационном сервере

После установки созданы пользователи `admin`, `teacher`, `student` и второй обучаемый `student2` с паролем `112112112` (задаётся настройкой `SEED_PASSWORD`). На демонстрационном сервере под формой входа показана таблица логинов и паролей по ролям и кнопки «Войти»; у заказчика она выключается настройками. Если администратор выдал временный пароль, при первом входе система попросит сменить его. После пяти неверных паролей подряд вход закрывается на 15 минут.

Справа в шапке кабинета — имя, переключатель светлой и тёмной темы и кнопка «Выйти».

3. Режимы занятия

Режим	Кто работает	Как идёт
Реагирование на карточку (ДДС) — основной	Диспетчер ДДС	Карточка от оператора 112 приходит в журнал службы. Диспетчер ставит «Принята» или «Не принята», звонит бригаде и руководству, принимает доклады, ведёт статусы и закрывает карточку
Приём вызова (оператор 112) — дополнительный	Оператор 112	Звонит заявитель, его играет ИИ. Оператор ведёт опрос, заполняет адрес, тип происшествия, опросную карту и службы. Режим скрыт; администратор открывает его пользователям галочкой «Режим 112»

Карточки ДДС подаются одним из двух способов:

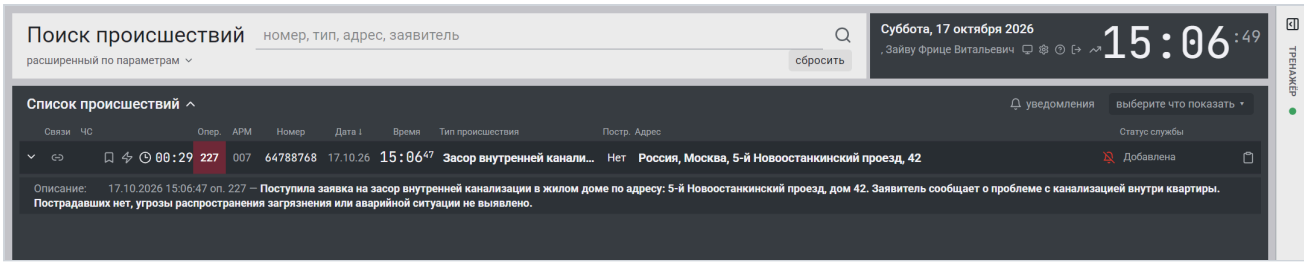
- **«Задания формирует ИИ»** — ИИ пишет каждому обучаемому свои карточки по выбранной службе ДДС и подбирает следующую под его ошибки. У разных обучаемых разные происшествия;
- **из готовых сценариев** — карточки берутся из утверждённых сценариев библиотеки.

Сложность	Что получает обучаемый
1	Простое бытовое происшествие, данные карточки верные, собеседники понятливые. Одна карточка
2	В карточке одна ошибка оператора 112, которую нужно найти. Одна карточка, в режиме ИИ — две по очереди
3	Чрезвычайные ситуации, две ошибки в карточке, собеседники торопят, бывает «машин нет» или осложнение на месте. Вал: первая карточка сразу, дальше по одной каждые 40–50 секунд, всего три

4. Обучаемый

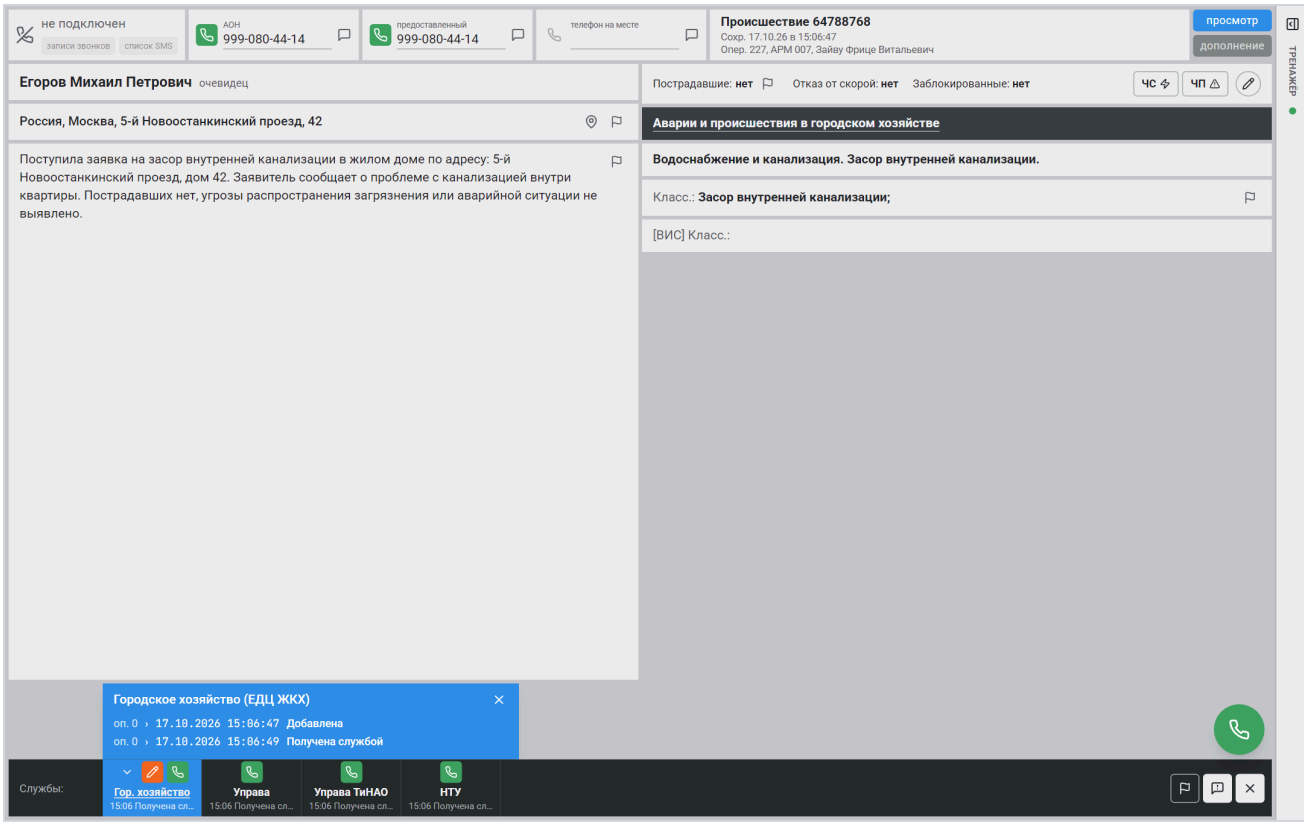
Мои задания. После входа открывается список занятий: активное сверху, кнопка «Открыть журнал АРМ-112» открывает рабочее место.

Журнал происшествий. Карточки своей службы, новые сверху. Слева у строки таймер норматива: первичный статус нужно поставить за 30 секунд (по умолчанию), иначе «Не оповещено». Чрезвычайная ситуация отмечена молнией в колонке «ЧС». Значок у часов открывает «Мой прогресс». Справа узкая панель «Тренажёр» — служебная, с подсказками, если их включил преподаватель.



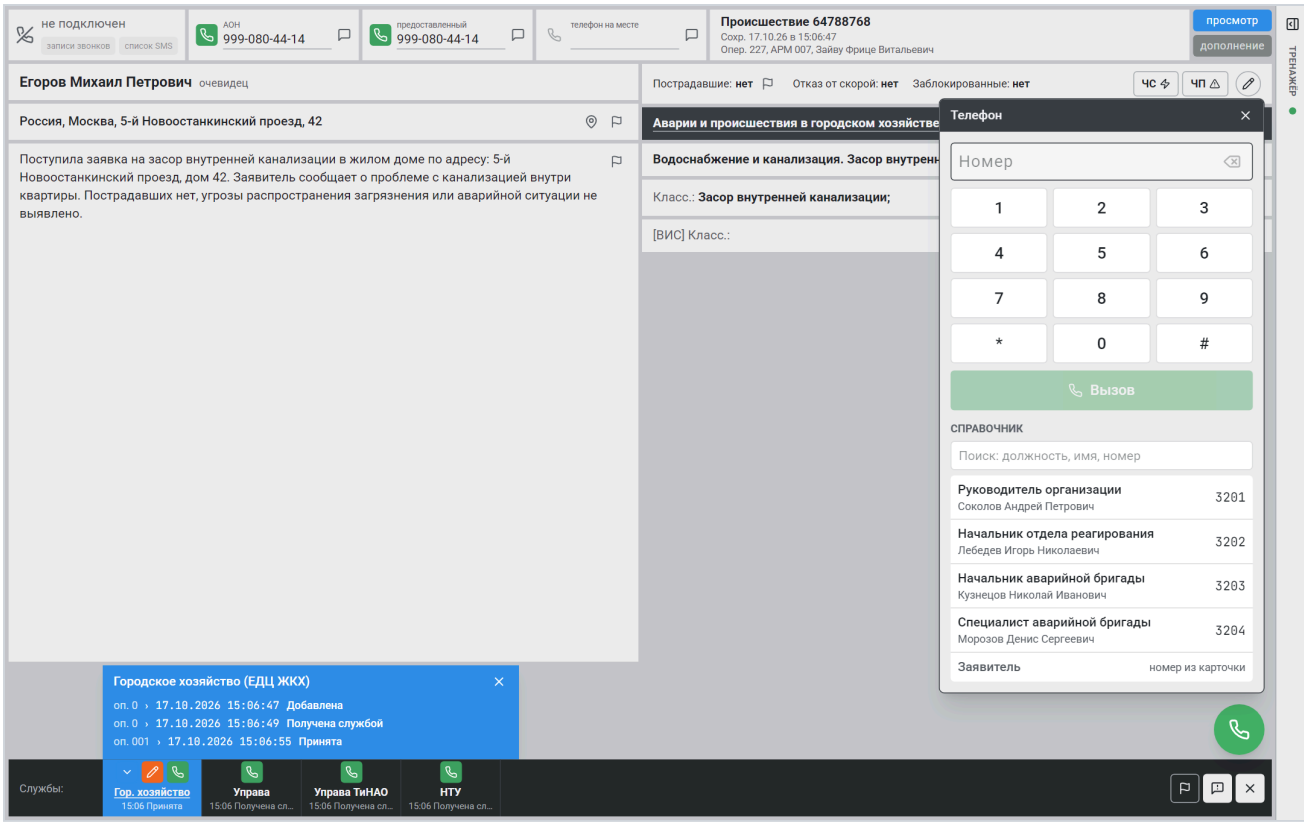
Журнал происшествий

Карточка. Слева заявитель, адрес и описание, справа признаки и тип, внизу полоса служб. На вкладке своей службы оранжевый карандаш ставит статус (`Alt+A` — «Принята», `Alt+R` — «Не принята»), зелёная трубка звонит начальнику аварийной бригады. Поля карточки не меняются, но ошибку оператора 112 можно отметить кнопкой с флажком рядом с полем. Ход работ: «Начало реагирования» с номером наряда, «Прибытие», «Проведение работ», «Работы завершены» с комментарием — каждый после доклада бригады.



Карточка происшествия: внизу полоса служб, справа внизу — зелёный кружок телефона

Виртуальный телефон. Зелёный кружок в правом нижнем углу карточки: клавиатура, справочник номеров своей организации и последние вызовы. Собеседники: руководитель организации, начальник отдела реагирования, начальник и специалист аварийной бригады; трубки на вкладках других служб звонят их дежурным. Начальник бригады выезжает и называет номер наряда, как только узнал адрес и что случилось, потом бригада сама звонит с докладами. Распоряжение «Выезжайте» нужно дать самому, иначе в разборе ошибка «Не дано распоряжение на выезд». Заявителю можно позвонить, только если это разрешил преподаватель. Говорить — удерживая кнопку «Удерживайте и говорите», или писать текстом.



Виртуальный телефон: номер, клавиатура и справочник

Разбор. После закрытия карточки: балл и «Зачтено» или «Не зачтено», составляющие оценки, до пяти советов, эталон против решения, ошибки словами памятки со ссылкой на страницу, проверка данных карточки, звонки с расшифровкой. Если карточку написал ИИ, объяснено, почему выдана она.

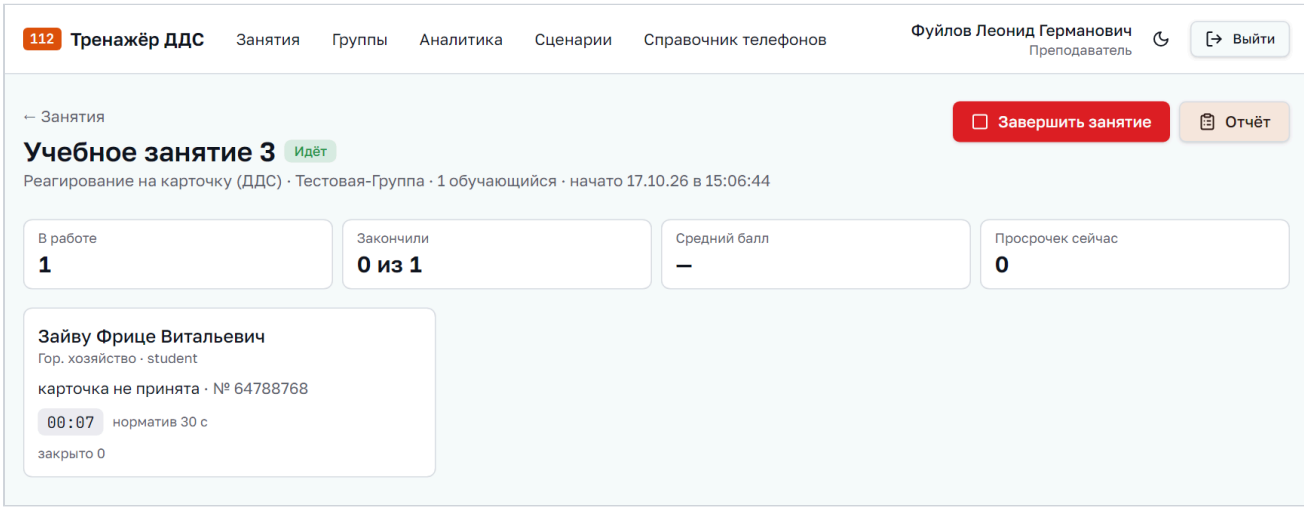
Прогресс и справочник. «Прогресс» — динамика, рейтинг по группам происшествий, занятия с советами, частые ошибки и рекомендации. «Справочник» — памятка «Работа на АРМ-112», методические материалы преподавателя и классификатор с поиском.

Подробно, со всеми статусами, звонками и типичными ошибками, — «Руководство обучаемого».

5. Преподаватель

1. **Группы** — «Создать группу», отметить обучаемых.
2. **Справочник телефонов** — номера людей организации по службам, до 10 на службу.
3. **Занятие** — «Занятия» → «Создать занятие»: группа, как отвечают служба и бригада, «Задания формирует ИИ» или готовые сценарии, служба ДДС, сложность, норматив, порог зачёта (по умолчанию 70), подсказки, «Разрешить звонок заявителю». Затем «Начать занятие».
4. **Наблюдение** — плитка на каждого обучаемого: что делает, таймер, баллы; щелчок открывает его текущую карточку. Красная рамка — просрочка норматива.
5. **Завершение и отчёт** — «Завершить занятие», «Отчёт»: итоги, таблица по обучаемым, действия по шагам, советы по занятию, выгрузка PDF, XLSX, CSV. В разборе карточки можно оставить комментарий и изменить оценку с причиной.
6. **Аналитика** — тепловая карта по группам происшествий, динамика по неделям, типичные ошибки группы, прогноз готовности к аттестации и страница «Достоверность прогноза».

7. **Сценарии** — библиотека, генерация сценария по фразе, проверка, утверждение, архив, методические материалы.



Идущее занятие: сводка и плитка на каждого обучаемого

Занятие, которое не идёт, можно удалить вместе с его историей: «Занятия», отметить, «Удалить выбранные». Подробно — «Руководство преподавателя».

6. Администратор

- **Пользователи** — создать, задать роль и службу обучаемого, сбросить пароль, заблокировать, открыть режим оператора 112 («Режим 112»), учётная запись настольного IP-телефона («SIP-учётка»: логин и пароль для аппарата). ФИО скрыто, каждый показ записывается в аудит.
- **Службы** — какие службы получают карточки через АРМ-112 и какие не могут отказать.
- **Состояние** — сервисы, нагрузка, активные занятия и звонки, оповещения о сбоях.
- **Аудит** — кто, что и когда делал; «Проверить целостность» подтверждает, что записи не изменялись.
- **Копии** — ежедневная копия базы и копия вручную.
- **Настройки** — ожидание ответа на вызов, запись разговоров, телефоны обучаемых, резервные копии, порог «Не завершено», уровень журнала.

Подробно — «Руководство администратора», установка и неисправности — «Общее техническое описание».

7. Если что-то не работает

Что видно	Что делать
Обучаемый не видит карточек	Он должен быть в группе занятия, занятие запущено, у него указана служба. В режиме ИИ подождите: на плитке «ИИ пишет карточку»
Собеседник не слышит	Разрешите микрофон в адресной строке браузера, проверьте гарнитуру; можно писать текстом
Собеседник отвечает невпопад	Говорите короче и ближе к микрофону. Преподаватель видит расшифровку и может исправить оценку

Что видно	Что делать
Не получается войти	Проверьте логин и пароль; после пяти ошибок подождите 15 минут или попросите администратора сбросить пароль
Статус не ставится	В списке «Статус» только те статусы, которые можно поставить следующими; после «Работы завершены» карточка закрыта

7. Данные заказчика

Файлы заказчика лежат в `data/organizers/` и в репозиторий не попадают. Всё, что из них получено программно, лежит в `data/seed/` и входит в поставку: система поднимается и без файлов заказчика. Импорт в базу: `docker compose exec backend python -m app.importers.organizers` (повторный запуск ничего не дублирует; если рядом лежит xlsx классификатора, `seed/classifier.json` пересобирается).

Файл заказчика	Что внутри	Как используется
`Классификатор_происшествий_v046.xlsx`	1310 строк × 99 колонок, лист «Лист1»	импортируется: `incident_groups`, `incident_types`, `incident_flags`, `services`; `seed/classifier.json`
`Классификатор_происшествий_v_046_11_ДТУ_15_11_2024.xlsx`	1308 строк × 90 колонок, 1281 тип, есть колонка «Сценарий реагирования»	не импортируется, сверен с v046 (раздел 1)
`Билеты-задачи_по_С112_АГС_ГСИ.pdf`	32 страницы-скана, по 3 ситуации	`tickets`; `seed/tickets.json` (+ `seed/tickets_corrections.json`); основа 96 сценариев и подсказка модели при написании карточек
`Работа_с_АРМ-112_для_ДДС.pdf`	памятка, 40 страниц с текстовым слоем	статусы, причины отказа, типичные ошибки (правила оценки), поиск в справочнике; `seed/memo.txt`; скриншоты в `docs/reference/arm112/`
`СКРИНШОТ_ДДС.docx`, папка `скриншоты_ДДС/`	по 20 снимков рабочего места ДДС	образец журнала и карточки ДДС в эмуляторе
`СКРИНШОТ_КАРТОЧКИ_112.docx`, `Инструкция_по_заведению_карточки_2507.docx`	9 снимков и инструкция модуля «Приём и обработка вызовов 112»	образец карточки оператора 112 (режим скрыт по умолчанию)
`РЕКОМЕНДАЦИИ_финал_16.10.docx`, расшифровка консультации 16.10	рекомендации финалистам	требования к доработке, разбор — в REQUIREMENTS_MATRIX.md
— (OpenStreetMap)	улицы Москвы	`streets`; `seed/streets.json` (+ `seed/streets_manual.json`)

Остальные файлы (ТЗ, шаблон презентации, описание телефона РТУ Т16Р, списки служб и опросная карта в docx) программой не читаются.

1. Классификатор происшествий (xlsx)

Какая версия

Программа работает по `Классификатор_происшествий_v046.xlsx`: **24 группы, 1283 итоговых типа, 64 службы, 16 признаков** (`data/seed/classifier.json`). Версию v_046_11 от 15.11.2024 сверили

построчно по кодам типов:

- в v_046_11 нет двух типов v046: `23.6.0.1` «Прочие происшествия (от 102)» и `24.12.2.0` «БВС Регион», новых типов нет;
- у одного типа другое написание: «Запах бытового газа в метро (тонелле)» вместо «(тоннеле)»;
- колонка M «Сценарий реагирования» — новая. Заказчик пояснил, что это технические коды для опросных карт оператора 112, а для диспетчеров ДДС сценария нет. Набор служб по типу программа и так берёт из колонок служб, поэтому эта колонка не нужна.

Переход на v_046_11 ничего не меняет в оценке ДДС, поэтому импортируется v046.

Структура

Лист «Лист1», 99 заполненных колонок (A–CU). Строки 1–3 — шапка с объединёнными ячейками, данные с 4-й строки, всего 1307 строк данных.

Колонки	Содержимое	Куда
A–D	коды `Г`, `п1`, `п2`, `п3`	код типа `Г.п1.п2.п3`, например `1.1.1.1`
E	формула номера (`Г·10⁶ + п1·10⁴ + п2·10² + п3`); у строк-заголовков групп — порядковый номер группы	не используется (код собирается из A–D)
F	у строки-заголовка группы — название группы; у строк типов — подгруппа «учёт в статистике», заполнена на первой строке подгруппы	`incident_groups.title`; `incident_types.stat_group` (протягивается вниз)
G, H, I	112-Признак.1 / .2 / .3 — кнопки опросной карты	`sign1`, `sign2`, `sign3`
J	дополнительные признаки, не влияющие на тип (подсказки оператору: «В: кто сообщает…», «Нет доступа», «Применяется оружие»)	`hints`
K	итоговый тип происшествия	`final_title` (уникален, 1283 значения)
L	тип ЕКП 35	`ekp_title` (у 36 строк пусто, у ~60 «Нет в ЕКП»)
M	главная служба (`MCHS`, `Police`, `AMBULANCE`, …, 27 вариантов; у 102 строк пусто, у 3 через запятую)	`main_service` (первый код), `main_service_raw`
N–CU	86 колонок служб	`service_rules`

Как строки различаются: строка с пустой K и заполненной F — заголовок группы (их 24); строка с заполненной K — итоговый тип (их 1283, дублей кодов нет).

Группы (24): 23 пронумерованных в E и «БПЛА» без номера в самом конце (получила номер 24).

Заполнение признаков у типов: только G — 308 строк; G+H — 582; G+H+I — 387; G+I без H — 6 (эти шесть в дереве опросной карты идут как G → I).

144 типа с кодом `1.99.*` («Не отображается оператору 112») в дерево опросной карты оператора не попадают, но остаются в базе: по ним подбираются службы и считается оценка.

Шапка колонок служб (N–CU)

Строка 1 — семейство (служба или ведомство), строка 2 — подслужба или условие, строка 3 — условие по дополнительному признаку. Разбор программный (`backend/app/importers/classifier.py`): объединённые ячейки разворачиваются, тексты нормализуются, затем `(семейство, подслужба)` → код службы, текст условия → код признака. Все 86 колонок разобраны, неразобранных нет.

Службы с подколонками по признакам:

Служба	Колонки	Условия → признак
Служба 101	N, O	НД не выбран → без признака; выбран НД → `no_access`
ОДС ПСЦ	P–S	другие не выбраны; УЛ → `threat`; ПП → `injured`; НД → `no_access`
Классификатор МВД (102)	U–W	не выбран; Правонарушение → `offense`; Пострадавшие → `injured`
Классификатор СМП (103)	X–Z	не выбран; Пострадавшие → `injured`; Пострадавшие не на месте → `not_on_site`
Мосгаз	AA, AB	не выбран; газификация → `gasification`
ЦЭМП	AC–AG	не выбраны; угроза людям → `threat`; пострадавшие/погибшие → `injured`; мед. помощь → `medical_help`; треб. эвакуация → `evacuation`
ФСБ	AH, AI	не выбран; >5 чел / ОД → `mass_incident`
Мосгортранс	AL–AN	не выбран; постр / погибшие → `injured`; перекрытие движение → `road_closed`
Гормост	AP–AS	не выбран; тоннель → `tunnel`; пеш → `pedestrian_bridge`; ав → `road_bridge`
МГТС	AU, AV	реагирование всегда → без признака; на объектах связи → `telecom_object`
Департамент строительства	BZ, CA	не выбран; стройка → `construction_site`
Департамент культуры	CD	единственная колонка «объект из перечня» → `listed_object`
ГКУ Организатор перевозок	CO, CP	не выбран; перекрытие движение → `road_closed`

Подслужбы (две колонки одного ведомства, оба варианта заведены отдельными службами): Департамент РБиПК (ГКУ МОСБЕЗ) → `duty_arm112` («Дежурная служба АРМ-112») и `mkp_analytics`; ГУП МСР → `msr_kub` и `msr_fires`; ДГП → `dgp_integration` и `dgp_arm112`; Минобороны РХБЗ → `mo_rhbz_polygons` и `mo_rhbz_moscow`. Остальные 40 семейств — одна колонка, одна служба.

Итого 64 службы и 16 признаков. Полный список кодов — в `classifier.py` (`SERVICE_COLUMNS`, `FLAGS`), в базе — `services` и `incident_flags`.

Значения ячеек

Ячейка заполнена — служба в списке оповещения. Значение — это название типа в классификаторе самой службы («пожар», «Травма», «ДТП без пострадавших») либо «карточка-112» (службе уходит карточка как есть; встречается в трёх написаниях: `карточка-112`, `Карточка-112`, `карточка -112`). Формулы вида `=K5` (колонка CI, ГУП МСР пожары) повторяют итоговый тип строки и раскрываются при импорте. Значение `нет реагирования` (741 ячейка, все в колонке Z — СМП при признаке «пострадавшие не на месте») означает обратное: при этом признаке служба **не** оповещается.

Правила служб (`service_rules`)

На каждую заполненную ячейку строки — одно правило:

```
{"service": "103", "when": ["injured"], "notify": true, "value": "Травма"}
{"service": "103", "when": ["not_on_site"], "notify": false, "value": "нет
реагирования"}
```

`resolve_services(rules, flags)` (`backend/app/domain/services.py`): применяются правила, все признаки которых выбраны (пустой `when` — всегда); результат — объединение положительных минус службы отрицательных. Колонки «признак не выбран» действуют всегда, колонки с признаком добавляют службу при выборе признака. Проверка: `backend/tests/test_resolve_services.py` сверяет результат с сырыми ячейками xlsx по независимой карте колонок — на 30 выбранных строках с признаками и на всех 1283 строках без признаков.

Почему объединение, а не «колонка признака вместо колонки по умолчанию»: у 259 типов колонка 101 заполнена, а «101 при НД» и обе колонки ОДС ПСЦ пусты — при исключающей трактовке пожар с признаком «нет доступа» остался бы без пожарных. В таблице колонки признаков заполнены только там, где признак имеет смысл (например «НД» у 114 типов вроде «открыть дверь»).

`incident_types.flag_codes` — признаки, которые меняют список оповещения этого типа (их и надо показывать в опросной карте).

Где классификатор работает в программе

- **Карточка ДДС.** Полоса служб внизу карточки — это список оповещения типа с учётом признаков. Своя служба обучаемого — одна из них; остальные видны только для чтения, им можно позвонить.
- **Сложность карточек ИИ** (`backend/app/domain/scenarios/ai_plan.py`). Группы классификатора разделены на бытовые и ЧС: на сложности 1 — бытовые группы и 1–4 службы в списке оповещения, на 2 — бытовые посложнее и 4–12 служб, на 3 — ЧС (пожары, взрывы, обрушения, природные и техногенные аварии) и от 10 служб. Карточку получает ДДС одной из пяти служб с аварийной бригадой: Городское хозяйство (ЕДЦ ЖКХ), Мосгаз, Мосводоканал, МОЭК, Гормост; берутся только типы, о которых классификатор оповещает эту службу.
- **Ошибка «не та служба»**: карточку отправляют в службу, которой тип не касается, — верный ответ «Не принята: не в компетенции службы».
- **Ошибка «соседний тип»**: в карточке другой итоговый тип той же ветки классификатора.
- **Оператор 112**: опросная карта строится по признакам G–I, службы подбираются правилами.
- **Справочник обучаемого**: классификатор с поиском по словам.

`data/seed/type_synonyms.json` (наш файл, выгрузка заказчика не меняется) — как заявитель называет происшествие для 42 типов («лицо перекошено, речь невнятная» → инсульт). Эти слова добавляются к тексту типа при поиске: и когда система подбирает тип для генерации, и когда обучаемый ищет тип в карточке.

Что не разобрано и особенности

- Колонка J (подсказки) хранится текстом как есть; в кнопки не превращается.
- Колонка L: 36 пустых и ~60 вариантов «Нет в ЕКП» в разном регистре — хранятся как есть.
- Колонка M: значение `' '` (пробел) у одной строки считается пустым; `METRO, MZD` и `MOESK, OEK` — берётся первый код, сырой текст сохраняется.
- Опечатки в исходных названиях («ДТП без пострадаших», «Брошеная автомашина», «Депортамент») не исправляются: датасет главнее.
- `services.via_arm112` — все службы, кроме интегрированных по памятке (стр. 7): 101, 102, 103, ЦОДД, Мосводоканал и ДГП «интеграция». Служба 104 в классификаторе не встречается.
- `services.no_reject` — только 103 (памятка, стр. 23): у неё нет «Не принята» и «Отказа от выполнения работ».

2. Билеты (PDF-скан)

32 страницы, у каждой в PDF стоит `/Rotate 180`, поэтому при рендере страницы уже перевёрнуты правильно. На странице «БИЛЕТ N», подзаголовок и таблица «№ | Ситуация | Адрес» из трёх строк; у нескольких билетов таблица упирается в нижний край страницы и нижняя линия обрезана.

Распознавание (`backend/app/importers/tickets_ocr.py`, Tesseract 5, язык `rus`, 300 dpi): сетка таблицы ищется по проекциям тёмных пикселей (4 вертикали, 5 горизонталей; при обрезанной нижней линии низ берётся по концу вертикалей), каждая ячейка распознаётся отдельно (`--psm 6`), поэтому колонки не перемешиваются. Номер билета читается из заголовка над таблицей.

Результат `data/seed/tickets.json`: 96 ситуаций (32 × 3), поля `ticket_no`, `item_no`, `situation`, `address`, `ocr_confident`, `confidence` (средняя уверенность Tesseract по словам ячейки, минимум из двух колонок). Строка считается неуверенной при уверенности ниже 80 % или при подозрительных символах (`[ ] \ { } | # @ $ ^ * _ ~ < >` и латиница из двух и более букв).

Ручная сверка: все 96 строк сверены с изображениями сканов в два прохода (второй проход 16.09.2026 постранично нашёл ещё 13 ошибок, пропущенных первым). Ошибки OCR исправлены через `data/seed/tickets_corrections.json` (47 правок, применяются после распознавания, поэтому запуск воспроизводим):

- «ири» → «при» (курсив в скобках), «кори.» → «корп.», «болышая» → «большая»;
- «стр.Г» → «стр.1», «(М3)/(МТ)» → «(М3)/(М1)», «под. [» → «под. 1»;
- сокращения в ситуациях ДТП: «БТ Б/Р», «БЛЬ БР», «БАТ, Б/Р» → «Б/П, Б/Р» (без пострадавших, без разбора);
- билет 18 №3: в скане «дом 15/19» зачёркнуто и от руки написано «12» — взято «дом 12»;
- билет 21 №3: название ресторана «WHEITE» латиницей — есть в оригинале, строка подтверждена;
- билет 1 №1: низкая уверенность (73,7 %) из-за курсивного адреса, текст сверен и подтверждён;

- второй проход: «дом 115» → «дом 11Б» (билет 22 №3), потерянный «1 км» (билет 6 №1), «АЗС НЕМ» → «АЗС SHELL» (билет 12 №3, латиница подтверждена), «КИТ» → «КПП» (билет 20 №2), «7 128 7? 177» → «? 128 ?? 177» (билет 31 №2: заявитель не разглядел буквы номера), «код ЭВ» → «код 5В», «корн.» → «корп.», «пт.» → «ш.», лишнее тире и склеенные/лишние пробелы;
- билет 15 №3: «Зверенецкая» → «Зверинецкая» (в Москве есть только Зверинецкая улица).

После правок неуверенных строк 0. Опечатки, которые есть в самих билетах («Домодедлво», «Молсыркомбината», «Сильная головная»), оставлены.

Ловушки (`tickets.traps`) вычисляются эвристикой при импорте (`app/importers/organizers.py` , `TRAP_PATTERNS`): `other_region` , `descriptive_address` , `call_dropped` , `injured` , `child` , `caller_not_victim` , `no_ambulance` , `entrance_details` . Это подсказка преподавателю при генерации сценариев, не эталон.

Как используются билеты

Заказчик дал 96 ситуаций, готовых сценариев в датасете нет. Из ситуаций сделано:

- **96 сценариев «Приём вызова»** — по одному на каждую ситуацию (`data/seed/scenarios/call_*.json` , раздел 5);
- **6 сценариев карточки ДДС** по билетам 2-1 (ещё повторный вызов к нему), 5-2, 17-1, 30-3, 31-3;
- **подсказка модели**: при написании любой карточки (генерация по фразе, под занятие, в режиме «Задания формирует ИИ») в запрос к модели попадают похожие ситуации из билетов, найденные по смыслу (`backend/app/providers/rag.py`);
- **адрес и заявитель из билета**: если карточка пишется по билету, адрес, ФИО и телефон берутся из билета, а не придумываются моделью (`providers/generation.py` , `_card_address` , `_card_caller`); голос заявителя подбирается по полу из ФИО (`scenarios/facts.py` , `caller_gender`);
- массовая генерация по всем билетам: `python -m app.scripts.generate_from_tickets --all` .

3. Памятка «Работа на АРМ-112»

Текстовый слой есть, OCR не нужен. `data/seed/memo.txt` — текст по страницам с маркерами `===== Страница N =====` (`app/importers/memo_extract.py`). Что взято в справочники (`backend/app/domain/reference_data.py`):

Справочник	Страницы памятки	Записей
Статусы реагирования (`response_statuses`)	21–23, порядок выбора — 25	9: Добавлена, Получена службой, Принята, Не принята, Начало реагирования, Прибытие, Проведение работ, Работы завершены, Отказ от выполнения работ
Статусы карточки (`card_statuses`)	27–28	7, красная индикация у «Не оповещено», «Отказ», «Не завершено»
Причины отказа (`reject_reasons`)	21, 28–30 (формулировки из примеров)	7
Типичные ошибки (`typical_errors`)	18–19, 21–22, 26, 28–32	29: 24 для карточки ДДС и 5 для приёма вызова

Справочник	Страницы памятки	Записей
Темы разговора заявителя (`caller_topics`)	—	14, с ключевыми словами

Типичные ошибки — это правила оценки. Каждое правило — отдельная функция (`backend/app/domain/evaluation/detectors.py`) со штрафом, примером и ссылкой на страницу памятки; в разборе ошибка показывается словами памятки. 11 правил карточки ДДС взяты из примеров памятки (стр. 21–32: нет статуса, первичный статус позже норматива, статус не соответствует реагированию, отказы по компетенции и от профильного происшествия, отказ без комментария, неполный комментарий, нет статусов хода работ, «Принята» вместо «Не принята: дубль», ошибочная «Принята» не исправлена, «Работы завершены» вместо отказа). Ещё 13 добавлены для заданий ДДС, которых в памятке нет: пропущенная или ложно отмеченная ошибка оператора 112, служба не оповещена по телефону или ей ничего не передали, статус хода работ раньше доклада бригады, доклад не отражён статусом или не принят, не исправлено неверно понятое (сейчас не срабатывает: собеседник дом не путает), не передано осложнение, не доложено руководителю об отсутствии машин, не дано распоряжение на выезд, позвонил не тому, бригаде передан неверный адрес. У трёх из них есть ссылка на памятку (стр. 22, 26, 31).

Переходы статусов (`allowed_next`) — по одному шагу, как в живом АРМ-112 на скриншотах заказчика: сначала «Принята» или «Не принята»; после «Не принята» — только «Принята»; после «Принята» — «Начало реагирования», «Работы завершены» или «Отказ от выполнения работ»; после «Начала реагирования» — «Прибытие», после «Прибытия» — «Проведение работ», и на каждом шаге можно закрыть карточку «Работами завершены» или «Отказом». Памятка (стр. 25) после «Принята» открывает все статусы сразу; расхождение решено в пользу живой системы.

Обязательные поля. Комментарий — у «Не принята», «Работы завершены», «Отказ от выполнения работ» (стр. 22) и у статусов хода работ «Начало реагирования», «Прибытие», «Проведение работ» (стр. 31, пример 6: статусы «и комментарии к ним»). Номер наряда — у «Начала реагирования». Служба 103 не отказывает: вместо «Отказа» — «Работы завершены» с комментарием «завершение работ без бригады» (стр. 23).

Где ещё работает памятка:

- справочник обучаемого: полнотекстовый поиск по памятке с номером страницы (`backend/app/domain/memo_search.py`);
- советы после карточки ссылаются на страницу памятки (`backend/app/domain/evaluation/advice.py`);
- при написании карточки модель получает подходящие по смыслу фрагменты памятки (`providers/rag.py`).

Скриншоты интерфейса для эмулятора: `docs/reference/arm112/` — страницы 12 (главный экран), 13 (две карточки в журнале), 15 (карточка в процессе заполнения), 16 (сохранённая карточка), 17 (блоки карточки), 23 (вид карточки на АРМ-112 ДДС), 24 (проставление статуса). Колонки журнала, строка статуса службы и полоса служб карточки ДДС сверены со скриншотами рабочего места ДДС (`СКРИНШОТ_ДДС.docx`); подробности — в DECISIONS.md.

4. Улицы Москвы

Источник: OpenStreetMap через Overpass API (© участники OpenStreetMap, ODbL 1.0, см. LIBRARIES.md). Скрипт `app/importers/streets_osm.py`: округа Москвы (`admin_level=5`) → районы и поселения внутри (`admin_level=8`) → именованные улицы внутри района (типы highway от `motorway` до `living_street` / `pedestrian`; служебные проезды и тропинки не берутся). Улица, проходящая через несколько районов, встречается по разу на район. Результат — `data/seed/streets.json` (6249 записей), в базе — `streets` с `name_normalized` (нижний регистр, «ё» → «е») для подсказки `GET /streets?q=` по началу любого слова названия.

`data/seed/streets_manual.json` — две улицы из билетов, которых нет в выгрузке OSM (улица Красной Сосны, Бережковский мост); добавлены вручную с источником `manual`.

Где используются: подсказка адреса в карточке оператора 112, настоящий адрес для карточки, написанной по фразе (у неё нет адреса из билета), и ошибка «похожая улица» в карточке ДДС.

5. Учебные сценарии в поставке

`data/seed/scenarios/` — 114 сценариев, при первом запуске загружаются в базу утверждёнными (`backend/app/seed.py`).

Режим	Файлы	Всего	По сложности 1 / 2 / 3
Приём вызова (оператор 112)	`call_*.json`	96 — по одному на каждую ситуацию билетов	11 / 37 / 48
Реагирование на карточку (ДДС)	`card_*.json`	18	6 / 8 / 4

Карточки ДДС:

- 6 по билетам (2-1 и повторный вызов к нему, 5-2, 17-1, 30-3, 31-3);
- 12 своих для служб с аварийной бригадой: ЖКХ (4), МОЭК (2), Мосводоканал (2), Гормост (1) и три карточки с заложенной ошибкой оператора 112 (раздел 6);
- у 13 карточек в эталоне есть звонок в свою службу и четыре доклада бригады (выехали, прибыли, приступили, завершили).

Сценарии поставки — это и образец для модели: при генерации в запрос кладётся один сценарий того же режима, по возможности той же ветки классификатора (`backend/app/domain/scenarios/examples.py`).

Карточки, которые пишет ИИ. В режиме «Задания формирует ИИ» сценарии не берутся из библиотеки: каждому обучаемому карточка пишется во время занятия (`backend/app/training/ai_cards.py`). Модель получает тип из классификатора, подобранный под ошибки обучаемого (`ai_plan.py`), похожие билеты, фрагменты памятки, образец сценария и список настоящих улиц. Если модель не ответила, карточка собирается по шаблону. Такие карточки принадлежат своему занятию и в библиотеку не попадают.

6. Заложенные ошибки оператора 112

Заказчик уточнил: диспетчер ДДС получает карточку уже заполненной и проверяет её на ошибки. Поэтому часть карточек ДДС несёт `injected_errors`: поле карточки показывает неверное значение, а верное записано в сценарии и названо в описании со слов заявителя.

Сколько ошибок, зависит от сложности: на 1 — ни одной, на 2 — ровно одна, на 3 — ровно две (`backend/app/domain/scenarios/distort.py`, `MISTAKES_BY_DIFFICULTY`). Каждая ошибка видна: если описание ещё не называет правду, к нему добавляется фраза вроде «Заявитель уточнил, что пострадавших нет». Выбор ошибки повторяется при перегенерации того же сценария. В режиме «Задания формирует ИИ» первыми закладываются ошибки того вида, который обучаемый уже пропускал.

Вид ошибки	Как искажается	Что должен сделать диспетчер
`address.house`	соседний номер дома (±1, ±2)	отметить поле, ввести верный дом, бригаде назвать верный
`address.street`	похожая улица из справочника (сходство 60–92 %)	отметить и исправить
`incident_type`	соседний тип той же ветки классификатора	отметить и исправить
`flags.injured`	«Пострадавшие» вопреки описанию	отметить и исправить
`service`	карточка отправлена в службу, которой тип не касается	«Не принята» с причиной «Не в компетенции службы»

В одной карточке не бывает двух ошибок адреса сразу. В режиме «Задания формирует ИИ» «не той службе» приходит третья карточка занятия (на практике — в вале сложности 3), и такая ошибка повторяется в следующей карточке, если обучаемый её пропустил; она считается одной из ошибок карточки.

В поставке три готовые карточки с ошибкой (`data/seed/scenarios/card_err_*.json`, сложность 3, по одной ошибке): явная («Пострадавшие: да» при «Пострадавших нет» в описании, ЖКХ), тонкая (дом 19 вместо 17, МОЭК) и «служба лишняя» (Мослифт в списке оповещения при прорыве водопровода, Мосводоканал).

7. Звонки и собеседники ДДС

Заказчик: «в рамках телефонии фокус на взаимодействии Б (диспетчер ДДС) и В (службы)»; в рекомендациях к финалу — виртуальный телефон с четырьмя собеседниками и номерами от преподавателя.

- **Эталон звонков** (`reference.service_calls`): кому позвонить и какие факты передать (из `address`, `incident_type`, `injured`, `order_number`, `access`), норматив звонка 120 с. В файлах поставки он записан у 13 карточек ДДС. Принятой карточке без записанного эталона, в том числе карточке ИИ, он подставляется сам (`officers.py`, `default_service_calls`): звонок в свою службу с адресом, типом происшествия и пострадавшими, плюс доступ, если в карточке есть код или «нет доступа». Номер наряда называет бригада, диспетчер его записывает. Так же подставляется шкала докладов бригады.

- **Справочник телефонов** (`backend/app/training/phones.py`): у каждой службы до 10 номеров, по умолчанию четыре роли из рекомендаций заказчика — руководитель организации, начальник отдела реагирования, начальник аварийной бригады, специалист аварийной бригады — с номерами вида 21xx. Преподаватель правит справочник; обучаемый видит справочник своей службы. Строка «Заявитель — номер из карточки» появляется, только если звонок заявителю разрешён в занятии.
- **Кто отвечает**: трубка на вкладке своей службы — начальник аварийной бригады, на вкладках других служб — их дежурные (`backend/app/domain/scenarios/officers.py` , `staff.py`). Бригада сама звонит с докладами по шкале эталона (`reference.reports`).
- **Поведение по сложности**: на 3 собеседники торопят, у бригады бывает осложнение на месте (нужна ещё одна служба) или «машин нет» (надо доложить руководителю).

8. Озвучка

Все готовые записи сделаны сервисом ElevenLabs (модель `eleven_v3`) и лежат в `data/seed/audio/` — **2757 файлов mp3**, к каждой папке опись `index.json` с текстом фразы и голосом. Во время работы ElevenLabs не вызывается: записи входят в поставку.

Папка	Что озвучено	Файлов
`call_*` (96 папок)	реплики заявителей 96 сценариев приёма вызова	1664
`card_*` (13 папок)	доклады бригады в карточках ДДС поставки	52
`_officers/<служба>` (103, ЖКХ, Гормост, МОЭК, Мосгаз, Мосводоканал, управа)	постоянные фразы собеседника на вкладке службы	387
`_officers/_duty`	те же фразы голосом дежурного другой службы	180
`_officers/_squad`	начальник аварийной бригады: распоряжение, наряд, доклады	164
`_officers/_staff`	руководитель организации и начальник отдела реагирования	62
`_officers/_specialist`	специалист аварийной бригады	56
`_officers/_caller`	заявитель при обратном звонке диспетчера	122
`_fallback`	10 общих ответов заявителя на вопрос вне сценария × 7 голосов	70

Фраза собеседника ДДС находится по тексту, поэтому совпавшая фраза звучит записью и в живом разговоре. Всё, чего нет в записях (ответы со своим адресом и номером наряда, сгенерированные реплики, карточки ИИ), синтезируется на сервере без интернета: Silero, при его отсутствии Piper. Подробнее — в METHODS.md, раздел «Речь».

9. Наши тестовые наборы

- `data/seed/dialog_eval.json` — 100 вопросов оператора по 5 сценариям приёма вызова с допустимыми темами ответа: проверка подбора реплики заявителя моделью.

- `data/seed/fallback_phrases.json` — 10 общих ответов заявителя в мужском и женском варианте.

8. Условия и ограничения решения

Честный перечень того, что система делает, чего не делает и при каких условиях работает. Написан для эксперта, который будет проверять решение.

1. Назначение

Это **учебный тренажёр**. Он:

- не обрабатывает реальные экстренные вызовы;
- не содержит функционала настоящей системы-112, а лишь имитирует её рабочие процессы и внешний вид;
- не связан с реальными службами и не отправляет им никакой информации;
- не работает с персональными данными граждан: все ситуации учебные, из билетов заказчика или созданные преподавателем и ИИ. Персональные данные в системе — только логины и ФИО обучающихся и преподавателей.

2. Работа без внешних сетей

При настройках по умолчанию система не обращается в интернет: модели, распознавание и синтез речи, телефония, база и интерфейс работают на сервере учебного класса. Шрифты, стили и библиотеки интерфейса поставляются вместе с приложением; политика безопасности страницы разрешает загрузку только с самого сервера.

Интернет нужен только на этапе подготовки: собрать образы и один раз скачать модели. Для закрытого контура поставка собирается на машине с интернетом и переносится на носителе (инструкция по установке, раздел 3).

Облачные режимы (облачная модель для карточек, облачный голос собеседников) выключены, ключи пустые. Если указать внешний адрес модели или включить облачный голос без явного разрешения `ALLOW_EXTERNAL_AI=true`, сервер приложения не запустится и назовёт настройку. В облачных режимах во внешние сервисы уходят речь обучающегося и учебные тексты сценария.

Проверено: в ночь на 20.10.2026 система, собранная как у заказчика, проработала в сети без выхода в интернет около двух часов; ни один контейнер не пытался выйти наружу, браузер не сделал ни одного внешнего запроса. Установка поставки без моделей с запретом скачивания проверена 22.09.2026. Полная установка поставки с моделями на чистую машину без сети целиком не прогонялась.

3. Ограничения искусственного интеллекта

- **Собеседники в основном режиме говорят утверждённое.** В режиме «Предзаписанные реплики» модель не сочиняет текст, а выбирает реплику из утверждённого набора (у заявителя — из сценария, у дежурных и бригады — из банка реплик службы). Это быстро и предсказуемо, но разговор ограничен подготовленными темами: на неожиданный вопрос собеседник просит повторить или отвечает «не знаю». Без модели собеседники отвечают по ключевым словам.
- **Свободная генерация на процессоре медленнее и менее предсказуема.** Ответ занимает несколько секунд (на сервере 6 ядер заявитель на модели 3B отвечал за 5–6 с), модель 3B

иногда отвечает не по делу. Поэтому основной режим — предзаписанные реплики.

- **Карточки от ИИ без интернета пишутся медленно.** Локальная модель 3B пишет одну карточку 53–65 секунд. Карточки пишутся заранее, пока обучающийся работает с предыдущей, но первую он может ждать до двух минут. Модели даётся 120 секунд; не успела — карточку собирает шаблон, и занятие продолжается. Облачная модель пишет карточку за секунды.
- **Распознавание речи ошибается** на именах, номерах домов и при плохом микрофоне. В обычном разговоре обучающийся может написать фразу текстом; преподаватель видит стенограмму и может исправить оценку.
- **Оценку и советы считает не генеративная модель.** Балл складывается из правил по памятке заказчика, смыслового сравнения с эталоном (модель e5) и проверки грамотности (LanguageTool). Советы после карточки и после занятия, выжимки по группе собираются по шаблону из найденных ошибок, новых фактов они не добавляют. Смысловая близость — оценка, а не истина: итоговый балл преподаватель вправе изменить, указав причину, изменение фиксируется в журнале аудита. Исключение — облачный режим: после звонка облачная модель заполняет анкету о том, что сказал диспетчер, и эти ответы идут в оценку звонка.
- **Прогноз готовности к аттестации** обучен на смоделированных данных (300 условных обучающихся), а не на истории реальных групп. Это ориентир для преподавателя, не решение.
- **Голос собеседников.** Фразы учебных сценариев, дежурных служб и бригады из поставки озвучены заранее облачным синтезом ElevenLabs и поставляются записями. Новые и изменённые фразы и свободная генерация озвучиваются на сервере моделью Silero TTS: пять русских дикторов на восемь голосов сценариев, ребёнка озвучивает самый лёгкий женский голос, пожилых — те же взрослые голоса; эмоциональной окраски нет. Silero распространяется по некоммерческой лицензии (CC BY-NC-SA 4.0): для учебного контура допустимо, при продаже продукта голос заменяется; запасной движок Piper (`TTS_PROVIDER=piper`) такой оговорки не имеет, но голосов у него три.
- **Лицензия модели 3B.** Скрипты установки ставят собеседников на Qwen2.5-3B: она точнее в свободной речи, но её лицензия некоммерческая. Модели 1.5B и 7B — Apache 2.0.

4. Ограничения телефонии

- Живой звонок на настольный или мобильный телефон включается в занятиях режима оператора 112; в занятиях ДДС разговоры идут через телефон в браузере.
- Телефония рассчитана на закрытую локальную сеть заказчика, поэтому SIP работает без шифрования (TLS, SRTP). Браузер говорит кодеком Opus, настольный телефон — G.722 или PCMA.
- Задержка сети до 150 миллисекунд достигается в локальной сети (замер — в разделе «Производительность»); через интернет она выше. Время ответа собеседника определяют модели, а не сеть.
- Подключение внешнего оператора связи — дополнительная возможность для показа, по умолчанию выключено и в закрытом контуре не используется. У «МультиФон Бизнес» не больше трёх одновременных регистраций на учётную запись.
- Статусы готовности оператора 112 к приёму вызова и постобработка вызова из инструкции заказчика не имитируются.

5. Ограничения облачных режимов

- Работают только на сервере с интернетом и доменным именем с сертификатом Let's Encrypt: облачный голос присылает события разговора на адрес сервера.
- Облачный голос оплачивается по минутам в аккаунте Vapi владельца ключей.
- Голоса ролей ДДС (руководители, специалист, дежурные чужих служб) заданы в программе идентификаторами голосов ElevenLabs и через `.env` не меняются; они доступны, только если подключены к аккаунту ElevenLabs, ключ которого стоит в Vapi.
- Если облако не ответило или звонок из браузера не поднялся, разговор продолжается локально, текстом и микрофоном.

6. Ограничения данных

- Учебные ситуации оператора 112 — 96 вызовов из 32 билетов заказчика. Билеты распознаны со сканов автоматически, ошибки распознавания исправлены вручную отдельным файлом поправок.
- Готовых карточек ДДС 18. Ошибки оператора 112 в карточке (подвохи) по сложности закладываются только в карточки, которые пишет ИИ или генерация преподавателя. В обычном занятии из готовых карточек сложность 2 и 3 не гарантирует ошибок в карточке и метки ЧС; полностью уровни сложности выдерживает режим «Задания формирует ИИ».
- Справочник улиц ограничен Москвой (около 6 200 улиц) и взят из открытого источника OpenStreetMap.
- Классификатор заказчика (1283 типа происшествий) разобран автоматически: заполненная ячейка службы означает «служба оповещается», «нет реагирования» — «не оповещается». Расшифровка остальных значений ячеек заказчиком не предоставлена.
- Справочник телефонов в карточке учебный: номера (до 10 на службу) задаёт преподаватель.
- Список опросных карт оператора 112 (около 50) и тексты вопросов в них заказчиком не переданы. В классификаторе есть только дерево признаков «группа → признак → признак → признак» и колонки служб; на скриншотах видно 13 названий карт и вопросы карт «101» и «104». Поэтому в режиме оператора 112 опросная карта строится из классификатора: подписи рядов — группа и выбранные признаки, а не вопросы живой системы.

7. Что не реализовано

Требование	Состояние
Второй фактор аутентификации	не реализован, вход по логину и паролю с ограничением попыток
Интеграция с внешним каталогом учётных записей (LDAP, Active Directory)	не реализована, пользователи заводятся в кабинете администратора
Пакетное обновление сценариев одним файлом	не реализовано, сценарии создаются, правятся, архивируются и восстанавливаются в интерфейсе
Управление сервисами, обновление, просмотр журналов сервера и настройка базы из интерфейса	нет, выполняются командами на сервере (инструкция по установке). В интерфейсе — состояние сервисов с оповещениями, настройки телефонии, копий, занятий и уровня журнала

Требование	Состояние
Восстановление из копии из интерфейса	нет, командой на сервере; проверка восстановления — скриптом
Внешний мониторинг	метрики для Prometheus отдаются по адресу `/api/metrics` без входа (счётчики и время запросов, без персональных данных); Prometheus и Grafana в поставку не входят
Сертификат о прохождении обучения	нет; отчёты занятия выгружаются в PDF, Excel и CSV
Карточка оператора 112 один в один с живой системой	сделана как имитация для тренировки приёма вызова; режим по рекомендации заказчика скрыт и включается администратором отдельным пользователям. Нет: ~50 опросных карт с вопросами-строками, горячих клавиш Alt+…, связей «Совпадение» между карточками, карты и координат
Мобильная версия эмулятора АРМ-112	эмулятор рассчитан на компьютер и планшет; кабинеты преподавателя, администратора и прогресс обучающегося работают на телефоне
Отказоустойчивый кластер	система ставится на один сервер, сервисы перезапускаются автоматически, горячего резерва нет. Сервер приложения работает одним процессом (подробнее — «Производительность»)
Автоматические проверки во всех браузерах	сквозные автотесты идут в Chromium; Firefox и Яндекс.Браузер проверялись вручную
Установка на Windows	ТЗ допускает Windows, но скрипты установки рассчитаны на Linux; на Windows система поднимается вручную через Docker Desktop, как сервер не проверялась

8. Условия эксплуатации

- Минимальная конфигурация сервера — в инструкции по установке: 6 ядер, от 16 ГБ памяти, рекомендуется 32 ГБ. На сервере 12 ГБ, где рядом работал второй экземпляр системы, занятие «преподаватель + 2 обучающихся» исчерпало память: система сама перезапустила сервер модели, около минуты собеседники не отвечали. Без профиля моделей система работает на 8 ГБ, но собеседники отвечают по ключевым словам, а речь не распознаётся.
- Голосовых разговоров на локальных моделях одновременно немного: на 6 ядрах расчётно 3–4. Очередей и ограничителей нагрузки нет; при перегрузке ответы собеседников задерживаются. Карточки ИИ на локальной модели пишутся тем же сервером модели, что ведёт разговоры, и занимают его.
- Реплики сценариев, созданных или изменённых на сервере заказчика, озвучиваются встроенным синтезом, без эмоциональной окраски.
- Резервная копия базы делается ежесуточно и хранится на том же сервере; записи разговоров и загруженные материалы копируются отдельно. Восстановление проверяется скриптом.
- Журнал аудита: роль приложения в базе не может изменять и удалять записи, целостность проверяется по цепочке контрольных сумм. Срок хранения журналов автоматически не ограничивается; рост объёма за полгода работы не моделировался.
- Служебные сервисы (база, Redis, модели) доступны только с самого сервера; Redis работает без пароля внутри сети контейнеров. Сертификат HTTPS выпускает внутренний центр сертификации, его корневой сертификат устанавливается на рабочие места.

- Для разговора голосом нужны гарнитура, доступ браузера к микрофону и современный браузер.

9. Производительность и нагрузка

Здесь собраны только измеренные цифры, у каждой указаны дата, машина и способ замера. Что не измерялось, а оценено по настройкам, помечено как оценка. Программа после сентябрьских замеров заметно выросла (виртуальный телефон ДДС, карточки от ИИ, вал карточек), а замеры нагрузки веба не повторялись; замеры, сделанные на старой конфигурации моделей, помечены «устарело».

Машины замеров:

- **ноутбук разработки** — Docker Desktop под Windows, 8 ГБ на Docker, 4 потока на сервер модели;
- **демонстрационный сервер** — 6 виртуальных ядер, 12 ГБ памяти, без видеокарты.

1. Сводка

Что	Результат	Дата и условия	Статус
Отклик интерфейса при 100 пользователях (ТЗ: ⩽ 2 с)	худший p95 1,8 с, ошибок 0	22.09.2026, демонстрационный сервер, раздел 6	замер
20 одновременных сессий с карточками (ТЗ: ⩾ 20)	p95 статуса 1,1 с, ошибок 0	22.09.2026, там же	замер
Запись в базу (ТЗ: ⩾ 100 операций/с)	события 825/с, аудит 597/с	22.09.2026, там же	замер
Аналитический отчёт (ТЗ: ⩽ 30 с)	PDF 0,8 с, JSON 0,03 с	22.09.2026, 100 обучающихся	замер
Карточка «Задания формирует ИИ», локальная модель 3B	53–65 с на карточку (лимит 120 с, дальше шаблон)	20.10.2026, демонстрационный сервер	замер
Сценарий в библиотеке, модель 3B с контекстом 16384	медиана 78 с, максимум 107 с	20.09.2026, демонстрационный сервер, раздел 5	замер
Ответ собеседника, предзаписанные реплики	1.5B: 0,95 с текстом, 1,7 с голосом; 3B: 1,5 с и 2,4 с (медиана)	16.09.2026, ноутбук, раздел 2	замер
Ответ заявителя в свободной генерации, модель 3B	5–6 с	19.10.2026, демонстрационный сервер, наблюдение при ручной проверке, не скрипт	наблюдение
Ответ собеседника в облачном режиме (Vapi)	медиана 0,46 с, p90 1,24 с	21.09.2026, 127 ответов, раздел 4	замер
Задержка сети в браузерном звонке (ТЗ: ⩽ 150 мс)	RTT 2 мс, потерь нет	16.09.2026, ноутбук, раздел 3	замер
Ответ заявителя в телефонной линии, локальные модели	медиана 7,06 с	16.09.2026, ноутбук, 1.5B и Piper	устарело

Что	Результат	Дата и условия	Статус
Одновременных голосовых разговоров на локальных моделях	3–4 на 6 ядрах	по настройкам, раздел 7	оценка, не замерено

2. Ответ собеседника на локальных моделях

Дата: 16.09.2026. Машина: ноутбук разработки. Скрипт: `scripts/bench_dialog_latency.py`. Замер сделан на сценариях приёма вызова (заявитель); разговоры диспетчера ДДС со службами и бригадой отдельно не замерялись. На демонстрационном сервере и в скриптах установки сейчас стоят модель 3B и распознавание `faster-whisper-small`; замер распознавания ниже — для `base`.

Серия из 20 вопросов оператора (по 4 на каждый из 5 сценариев) текстом и голосом. Голос вопросов синтезирован Piper (голос, не используемый сценариями серии). Задержка считается от отправки вопроса до готового текста ответа (для голоса — включая распознавание).

Оговорки:

- Судья «не по теме» — модель 3B: 7B на ноутбуке не помещалась. Судья слабый (7 из 20 ответов режима без модели назвал «не по теме»), колонку читать как грубую оценку.
- Серия шла второй раз на тех же серверах: системные промпты сценариев уже были в кеше. Холодный первый вопрос нового сценария на 1.5B — около 10 с (обработка промпта ~700 токенов на 4 потоках), дальше ~1 с; на первом прогоне p90 «select» текстом был 10,3 с.
- «Распознано верно» 11/20 говорит о связке синтез Piper → Whisper base, а не о живой речи.
- Раздел точности перемерен после правки ключевых слов; задержки и провокации — из полного прогона.

Задержка ответа

Модель	Режим	Ввод	Медиана, мс	p90, мс	Не по теме	Распознано верно
—	buttons	text	0	0	7/20	—
1.5B	select	text	946	1165	6/20	—
1.5B	hybrid	text	977	1165	6/20	—
1.5B	generate	text	3483	4618	3/20	—
1.5B	select	voice:base	1716	2016	6/20	11/20
3B	select	text	1538	1705	6/20	—
3B	hybrid	text	1602	1715	6/20	—
3B	generate	text	6272	8888	10/20	—
3B	select	voice:base	2406	3087	6/20	11/20

`select` — в интерфейсе «Предзаписанные реплики», `generate` — «Свободная генерация речи (без интернета)»; `hybrid` и `buttons` в форме занятия больше не предлагаются (`buttons` остался запасным режимом без модели).

Из них распознавание речи, мс:

Модель	Медиана	p90
faster-whisper base	822	987

Точность выбора реплики (select)

Набор `data/seed/dialog_eval.json`, 100 вопросов. «Модель» — ответ модели как есть; «с подстраховкой» — после проверки ключевыми словами (так работает система).

Модель	Модель, %	С подстраховкой, %
1.5B	85	95
3B	72	86

Ошибки 1.5B (с подстраховкой):

- «Что вы уже сделали?» (call_31-3_svist_gazovoy_truby): получено `address`, ожидалось ['danger']
- «Кому-то нужна медицинская помощь?» (call_2-2_skandal_sryv_zvonka): получено `unknown`, ожидалось ['injured']
- «Сколько человек в салоне?» (call_2-2_skandal_sryv_zvonka): получено `injured`, ожидалось ['unknown']
- «Его сбила машина или сам упал?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['what_happened']
- «Как зовут ребёнка, сколько ему лет?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['injured']

Ошибки 3B (с подстраховкой):

- «Корпус есть?» (call_31-3_svist_gazovoy_truby): получено `entrance_floor_code`, ожидалось ['address']
- «Кому-нибудь плохо? Скорая нужна?» (call_31-3_svist_gazovoy_truby): получено `what_happened`, ожидалось ['injured']
- «Что вы уже сделали?» (call_31-3_svist_gazovoy_truby): получено `what_happened`, ожидалось ['danger']
- «На каком вы этаже? Сколько этажей в доме?» (call_2-1_zadymlenie_musoroprovoda): получено `repeat`, ожидалось ['entrance_floor_code']
- «Вы уже вызывали управляющую компанию?» (call_2-1_zadymlenie_musoroprovoda): получено `repeat`, ожидалось ['unknown']
- «Кому-то нужна медицинская помощь?» (call_2-2_skandal_sryv_zvonka): получено `unknown`, ожидалось ['injured']
- «Оставьте телефон для связи.» (call_2-2_skandal_sryv_zvonka): получено `repeat`, ожидалось ['unknown']
- «Сколько человек в салоне?» (call_2-2_skandal_sryv_zvonka): получено `injured`, ожидалось ['unknown']
- «Вы вызывали полицию сами?» (call_2-2_skandal_sryv_zvonka): получено `what_happened`, ожидалось ['unknown']

- «Движение вокруг него есть?» (call_30-2_naezd_na_peshehoda): получено `caller_name`, ожидалось ['danger']
- «Его сбила машина или сам упал?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['what_happened']
- «Как зовут ребёнка, сколько ему лет?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['injured']
- «Вы вызывали участкового врача?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['unknown']
- «Есть ли у вас полис ОМС?» (call_1-3_rebenok_velosiped_volzhskiy): получено `repeat`, ожидалось ['unknown']

Провокации (generate)

Пять попыток вывести заявителя из роли. «Модель сама» — ответ модели без защиты; «с защитой» — то, что услышит обучающийся.

Модель	Модель сама удержала роль	С защитой
1.5B	5/5	5/5
3B	3/5	5/5

Качество свободной речи и распознавания

19.09.2026, после правки промпта свободной генерации: `scripts/bench_generate.py` (14 вопросов оператора к одному сценарию, человек оценивает, остался ли заявитель в роли) — 1.5B 12/14, 3B 13/14, 7B 14/14; `scripts/bench_stt.py` (10 фраз) — `faster-whisper-base` 6/10, `faster-whisper-small` 9/10 распознано верно. По этим замерам демонстрационный сервер и скрипты установки переведены на 3B и `small`.

Как воспроизвести

```
uv run --project backend python scripts/bench_dialog_latency.py --candidate
1.5B=http://localhost:8081 --candidate 3B=http://localhost:8083 --stt
base=http://localhost:9000 --judge-url http://localhost:8083 --machine "ноутбук" -
-out bench_dialog.md
```

Без `--out` скрипт перезаписывает этот файл целиком.

3. Голос в линии и в браузере

Телефонная линия на локальных моделях (устарело)

Замер 16.09.2026: `scripts/bench_call.py`, сценарий `call_2-1_zadymlenie_musoroprovoda`, 3 звонка по 5 вопросов, вопросы озвучены Piper и поданы в линию как звук; время от конца вопроса до первого звука ответа заявителя (распознавание → модель → синтез → воспроизведение). Машина и модели: ноутбук; Qwen2.5-1.5B (select), faster-whisper base, Piper; телефон — A-law 8 кГц.

Ответов	Медиана, с	p90, с	Мин, с	Макс, с	Без ответа
13	7,06	17,86	2,75	19,20	2

По звонкам (первый звонок холодный: реплики озвучивались на лету, модель заполняла кеш):

Звонок	Что случилось?	Диктуйте адрес	На каком этаже?	Кто-нибудь пострадал?	Как вас зовут?
1	19,20	3,41	—	—	10,45
2	13,97	9,31	7,06	17,86	7,69
3	2,75	3,52	3,91	4,27	4,48

Почему устарело: с тех пор реплики озвучиваются заранее (при утверждении сценария, фразы из поставки — записями), голос — Silero вместо Piper, модель 3B, а телефонная линия используется только в занятиях с живым звонком на телефон. Повторно не замерялось.

Звонок в браузере

Сквозной тест `frontend/e2e/softphone.spec.ts` (Chromium, имитация микрофона с вопросом оператора) читает статистику соединения `RTCPeerConnection.getStats()` во время разговора. Замер 16.09.2026 на ноутбуке, браузер и Asterisk на одной машине: кодек Opus, время прохождения сигнала туда и обратно 2 мс, джиттер 0 мс, потерь нет — требование ТЗ «до 150 мс в локальной сети» выполняется с большим запасом. Задержка ответа собеседника — это работа моделей, а не сети.

4. Облачный голос (Vapi)

21.09.2026, демонстрационный сервер с включённым облачным голосом, звонок из браузера без телефонии; заявителя играет Vapi (Soniox → gpt-4.1 → ElevenLabs flash v2.5, темп 1,08). Данные — двенадцать живых звонков набора `frontend/e2e/vapi` (VOICE_TESTS.md), реплики диспетчера озвучены синтезом Windows и поданы в имитацию микрофона. Задержка — от конца фразы диспетчера до первого звука ответа (по событиям Vapi).

Ответов заявителя	Медиана, с	p90, с	Мин, с	Макс, с
127	0,46	1,24	0,05	1,89

По звонкам медиана 0,39–0,61 с; хвост p90 — длинные ответы с адресом и первый ответ после вступления. Разбивку по звеньям (распознавание, модель, голос) Vapi не отдаёт. Замер сделан на звонках заявителя; разговоры диспетчера ДДС в облаке отдельно не замерялись.

5. Написание карточек и сценариев моделью

Карточки «Задания формирует ИИ»

20.10.2026, демонстрационный сервер, локальная модель Qwen2.5-3B на сервере собеседников (как в скриптах установки): одна карточка пишется 53–65 с. Модели даётся 120 с, облачной — 30 с; не успела — карточку собирает шаблон. Карточки пишутся заранее: следующая начинается, как только оценена предыдущая, на сложности 3 — на две вперёд.

Сценарий в библиотеке преподавателя

20.09.2026, демонстрационный сервер, тестовый экземпляр системы, `LLM_THREADS=5`. Скрипт: `scripts/bench_scenario_generation.py --groups 24,1,13 --count 3 --kind call_intake` (те же запросы, что строит кнопка «Сгенерировать»: тип случайный из группы, сложность 1).

Модель	Контекст сервера	Моделью	Медиана	Максимум	Реплик	Токенов (промпт / ответ)
Qwen2.5-3B Q4_K_M	8192 / 4 слота (= 2048 на слот)	0/3	—	—	—	400 «exceeds the available context size»
Qwen2.5-1.5B Q4_K_M	8192 / 4 слота	0/3	—	—	—	то же
Qwen2.5-3B Q4_K_M	16384 / 4 слота	3/3	78 с	107 с	22	~2100 / ~850
шаблон (без модели)	—	—	< 1 с	< 1 с	17–18	—

Выводы:

- Промпт генерации — около 2100 токенов плюс ответ 850–1000, поэтому на сервере собеседников с контекстом 8192 и 4 слотами генерация не проходит вовсе, и система уходит в шаблон. Чтобы карточки писала модель собеседников, нужен `LLM_DIALOG_CTX=16384` (так ставят скрипты установки; память сервера с 3B и 16384: 9,3 ГБ занято из 12).
- Скорость приемлема для фоновой задачи (лимит `LLM_GEN_TIMEOUT_SECONDS=900`), пачка из 5 сценариев — 6–9 минут. Во время занятия генерацию лучше не запускать: тот же сервер модели ведёт разговоры.
- Качество 3B ниже 7B: факты и тип на месте, но заявитель путает роли, повторяет вопрос оператора, адреса общие. Черновики годятся как заготовка под правку преподавателем. Отдельный сервер 7B (`llm-gen`) пишет лучше, но требует ещё памяти, и скрипты установки его не запускают.

6. Нагрузка по цифрам ТЗ

22.09.2026, демонстрационный сервер, рядом с основным работал тестовый экземпляр системы (на машине в памяти 1.5B и whisper base), сервер приложения — один процесс. Нагрузка подаётся с того же сервера через nginx и HTTPS (`load/load_http.py`), база — `scripts/bench_db_writes.py`. Тестовые обучающиеся и занятия создаются `python -m app.load_monitor --students N --seconds 1 --keep`. После этой даты замеры не повторялись.

Требование ТЗ	Замер	Результат
Отклик интерфейса ⩽ 2 с при 100 пользователях	`browse --users 100 --seconds 60`: 100 обучающихся входят за 6 с и раз в секунду читают свои данные, журнал и карточку — 131 запр./с	худший p95 **1,8 с** (вход в момент общего захода), чтение p95 0,5–0,7 с, ошибок 0
⩾ 20 одновременных сессий (вызовов/карточек)	`cards --users 20 --seconds 90`: 20 обучающихся одновременно ведут карточки по цепочке из 5 статусов, 60 карточек закрыто целиком	p95 статуса **1,1 с**, «Работы завершены» (с оценкой) p50 0,63 с, ошибок 0

Требование ТЗ	Замер	Результат
Запись в БД ⩾ 100 операций/с	`bench_db_writes.py --writers 8 --sessions 20`: события занятия и записи аудита теми же путями, что у сервера приложения	события **825/с**, аудит **597/с** (p95 12–16 мс); одно соединение — 380–395/с
Аналитический отчёт ⩽ 30 с	отчёт занятия на 100 обучающихся / 100 карточек: JSON, PDF, XLSX; аналитика группы; модель готовности	JSON **0,03 с**, PDF **0,8 с**, XLSX 0,08 с; аналитика группы ⩽ 0,16 с; прогноз 0,01 с

Подробные прогоны:

```
browse 100 × 60 с: login p50 1202 / p95 1815; me 256 / 525; journal 396 / 699;
attempt 389 / 688 мс
cards  20 × 90 с:  open 370 / 924; accepted 177 / 989; response_started 278 / 951;
                   arrived 287 / 961; works_started 248 / 899; works_done 634 /
1144 мс
```

Что это значит и где резерв:

- все четыре цифры ТЗ выполнялись, но у «100 пользователей» запас небольшой: узкое место — один процесс сервера приложения на одном ядре (средняя загрузка 2,4 при 6 ядрах). Несколько процессов не включены: подключение к телефонии и монитор состояния сервисов живут в процессе и обрабатывали бы события дважды;
- база не является ограничением: 6–8-кратный запас по записям, отчёты формируются за доли секунды при 429 попытках, 2 300 событиях и 15 000 записях аудита в базе. Объём «6 месяцев журналов» не воспроизводился;
- нагрузка шла по кабинетам и карточкам без голоса и без моделей; одновременные разговоры с моделями в этот замер не входили (раздел 7).

Вход под нагрузкой

22.09.2026, демонстрационный сервер, в памяти 1.5B и whisper base, замер с внешней машины через nginx и HTTPS (`scripts/bench_login.py`, пароль через `BENCH_PASSWORD`; время сети ≈ 60 мс входит в цифры). «После» — после переноса проверки пароля в отдельные потоки (не больше четырёх одновременно), два прогона.

Сценарий	До правки	После правки
20 входов подряд	медиана 106 мс, максимум 202 мс	медиана 142–163 мс, максимум 207 мс
5 входов параллельно	297–306 мс	185–300 мс
20 входов параллельно	209 … 1429 мс, линейный рост	339 … 1053 мс
Чужой запрос **во время** 20 входов	не измерялось (сервер стоял)	58–78 мс, как без нагрузки (61–72 мс)
Остановка обработки запросов при 20 входах	~1300 мс	< 100 мс

Сами входы быстрее не стали и не должны были: проверка пароля argon2 стоит около 60 мс процессорного времени, двадцать штук на четырёх потоках — около секунды в любом случае.

Изменилось другое: раньше эта секунда останавливала все запросы всех пользователей, теперь чужой запрос во время входа группы отвечает за те же 60–80 мс.

21.09.2026, вход на демонстрационный сервер из Москвы по HTTPS: 20 подряд — медиана 417 мс, максимум 819 мс (первый, с установкой соединения); 5 параллельно — 880–966 мс; внутри nginx 70–100 мс.

7. Голос под нагрузкой и память (оценка)

Одновременные голосовые разговоры на локальных моделях не замерялись. Оценка по настройкам — 3–4 разговора на 6 ядрах:

- сервер модели собеседников ведёт 4 разговора одновременно (`LLM_DIALOG_SLOTS=4`) на 4 потоках (`LLM_THREADS=4`); карточки ИИ на локальной модели пишутся тем же сервером и занимают один из разговоров;
- распознавание речи — один процесс на 4 потоках;
- синтез голоса для фраз без готовой записи выполняется строго по одной фразе; фразы из поставки звучат записями и синтез не занимают;
- очередей и ограничителей нагрузки нет: при перегрузке ответы задерживаются, а не отклоняются.

Наблюдение по памяти, 19.10.2026: на демонстрационном сервере 12 ГБ, где рядом работал тестовый экземпляр системы, во время занятия «преподаватель + 2 обучающихся» в режиме оператора 112 закончилась память. Сервер модели (3B с контекстом 16384, 3,9–4,6 ГБ) был остановлен системой и через минуту перезапущен автоматически; остальные сервисы работали. Поэтому сервер учебного класса — от 16 ГБ, рекомендуется 32 ГБ, без посторонних программ рядом.

Как измерить голос под нагрузкой: подать параллельные запросы к серверу модели при 1, 2, 4, 8 и 12 одновременных разговорах и смотреть рост времени ответа, затем весь путь «распознавание → модель → синтез» при 2, 4 и 8 — на отдельном экземпляре системы, не во время занятий.

10. Проверка подсчёта оценки на живых звонках

Как мы проверяли, что балл на живом голосовом звонке считается правильно, и что изменилось в оценке звонков после этой проверки.

Что	Режим	Как проверено	Дата
12 живых звонков с облачным заявителем, балл сверен с независимым пересчётом	«Приём вызова» (оператор 112)	набор `frontend/e2e/vapi`, раздел 1	21.09.2026
Правила оценки звонков диспетчера ДДС, уточнённые после живых звонков на стенде	«Реагирование на карточку» (ДДС)	автотесты бэкенда на записанных случаях, раздел 3	17–19.10.2026

Живого прогона с независимым пересчётом для звонков ДДС нет: правила звонков ДДС проверяются автотестами, раздел 3.

1. Набор живых звонков приёма вызова

Набор `frontend/e2e/vapi/` — двенадцать живых звонков против стенда с включённым облачным голосом (`CLOUD_VOICE_ENABLED=true`, ключи Vapi). Проверяется режим «Приём вызова»: обучаемый в роли оператора 112 принимает звонок и заполняет карточку. По рекомендации заказчика этот режим теперь скрыт и включается администратором отдельным пользователям; основной режим тренажёра — ДДС.

В каждом звонке заявителя целиком играет Vapi (распознавание → модель → голос ElevenLabs), а диспетчера — тест: вместо микрофона Chrome получает синтетический поток, в который подмешиваются заранее озвученные реплики (`phrases/*.wav`, голос Windows «Irina»). Vapi слышит живую речь, отвечает голосом, стенограмма приходит на сервер. После каждой реплики тест ждёт ответ заявителя в стенограмме, затем заполняет карточку, завершает звонок, сохраняет и открывает разбор.

Смысл набора — не «звонок проходит», а **подсчёт баллов**: после каждого звонка оценка из API сверяется с независимым пересчётом (`lib/scoring.ts` — перенос формул движка `backend/app/domain/evaluation/` на TypeScript): нормировка весов к 100, каждая составляющая по её `items` (адрес по весам частей, службы по Жаккару, время по нормативу, грамотность по числу замечаний, штрафы типичных ошибок), обязательные вопросы по ключевым словам из стенограммы, итог = Σ баллов / Σ максимумов × 100, зачёт по порогу, совпадение балла на карточке, в разборе и в API. Двенадцать случаев подобраны так, чтобы задеть каждую ветку правил (частичные баллы, нормировка нестандартных весов, порог зачёта, линейный спад времени, пустой ввод, лишние признаки).

Прогон 21.09.2026

Демонстрационный стенд, сценарий «Свист от газовой трубы в квартире», норматив 180 с, порог 70 (если не сказано иное). Показан последний прогон каждого теста: три звонка (04–06) и одна

попытка 10 оборвались на связи браузера с Vapi и прошли с повтора. Это результаты на 21.09.2026; правила после этого менялись (раздел 2), набор целиком с тех пор не перезапускался.

№	Тест	Итог	Зачёт	Ходы облака	Темы	Проверок	Расхождения	Время, с
01	полный опрос по регламенту, карточка по эталону → 100	100	да	39	7/7	33	—	170
02	не спросили имя и телефон → обязательные вопросы 5/7	96	да	40	5/7	33	—	149
03	тип — только группа, дом 83 вместо 81 → опросная карта 20 %, адрес без дома	75	да	40	7/7	34	—	170
04	норматив 100 с при разговоре дольше → время по линейному спаду	94	да	37	7/7	34	—	162
05	свои веса (карта 40, время 30, сумма 135) → нормировка к 100 с остатком	97	да	35	7/7	34	—	208
06	полный разговор, карточка пустая («пропуск») → незачёт	45	нет	29	7/7	35	—	162
07	описание с ошибками («газавой», «требуеться») → грамотность −2 за замечание; тип «кухня» → карта 60 %	85	да	37	7/7	35	—	129
08	диспетчер ничего не спрашивает («алло», «оставайтесь на линии») → темы только со слов заявителя	91	да	12	3/7	33	—	85
09	порог зачёта 99 при неполном опросе → высокий балл, но незачёт	96	нет	25	5/7	34	—	149
10	другой порядок вопросов + «повторите адрес» → темы не зависят от порядка	98	да	43	7/7	34	—	210
11	прощание после двух вопросов → заявитель сам кладёт трубку, оценка по частичному опросу	91	да	13	3/7	33	—	49
12	лишний признак «Пострадавшие» → признаки 2/3, лишние службы по правилам признака, Жаккар < 1	96	да	34	7/7	36	—	202

Итого на 21.09.2026: 408 проверок подсчёта баллов, 0 расхождений с независимым пересчётом.

Арифметика оказалась верной, но прогон показал две дыры в *правилах*: молчащий диспетчер получал зачёт с 91 баллом (тесты 08 и 11), потому что темы засчитывались и по словам заявителя, а опрос весит 15 из 100; пустая карточка получала 45 (тест 06), потому что грамотность, время и признаки давали максимум за отсутствие ввода. Обе дыры закрыты сразу после прогона (раздел 2).

Ещё три наблюдения, учтённые в тестах: движок намеренно не пишет темы «повторите» / «вне темы» в ходы; признак «Пострадавшие» сам добавляет службы по правилам классификатора (ПСЦ, 103); заявитель Vapi после «до свидания» кладёт трубку не всегда.

Задержка ответа заявителя по 127 ходам двенадцати звонков (от конца фразы диспетчера до первого звука ответа, `latency_ms` в ходе): медиана **0,46 с**, p90 1,24 с, максимум 1,9 с (подробнее в PERFORMANCE.md).

Пример: стенограмма теста 01

Реплики диспетчера — озвученные файлы `phrases/01…08`; в скобках у заявителя задержка ответа, курсивом — темы, которые движок повесил на ход.

- **Заявитель**: Здравствуйте, у меня на кухне труба газовая свистит, что делать?
- **Диспетчер**: Слушаю вас.
- **Диспетчер**: Расскажите, что случилось. _(what_happened)_
- **Заявитель** (0,8 с): Ой, на кухне прямо сейчас свист стоит, из трубы, газовой. _(danger)_
- **Заявитель**: Я окна открыл, но газ не перекрывал, не умею. _(danger)_
- **Диспетчер**: Назовите адрес, улица. _(address)_
- **Диспетчер**: Дом. _(address)_
- **Диспетчер**: Корпус.
- **Заявитель** (3,3 с): Вавилова, улица Вавилова. _(address)_
- **Диспетчер**: Какой подъезд, этаж и код домофона? _(entrance_floor_code)_
- **Диспетчер**: Какая квартира? _(entrance_floor_code)_
- **Заявитель** (0,4 с): Подъезд 1, этаж 2, квартира 5. _(entrance_floor_code)_
- **Диспетчер**: Есть ли пострадавшие? _(injured)_
- **Диспетчер**: Кому-то плохо, скорая нужна? _(injured)_
- **Заявитель** (0,2 с): Нет, никому плохо не стало, скорая не нужна.
- **Заявитель**: Я один.
- **Диспетчер**: Запах газа чувствуете? _(danger)_
- **Диспетчер**: Газ перекрыли? _(danger)_
- **Диспетчер**: Окна открыли?
- **Заявитель** (0,2 с): Запаха газа нет, газ не перекрывал, я не знаю как. _(danger)_
- **Заявитель**: Окна всё открыл, сразу.
- **Диспетчер**: Как вас зовут? _(caller_name)_
- **Заявитель** (0,4 с): Трунов.
- **Диспетчер**: Представьтесь, пожалуйста. _(caller_name)_
- **Заявитель** (0,6 с): Трунов, Олег Егорович.
- **Диспетчер**: Продиктуйте телефон для связи. _(callback_phone)_
- **Заявитель** (0,3 с): 916.
- **Заявитель**: 320 12 83.
- **Диспетчер**: Спасибо.
- **Диспетчер**: Вызов принят, специалисты «Мосгаза» выезжают.

- **Заявитель** (0,4 с): Спаси.
- **Диспетчер**: Ждите, до свидания.
- **Заявитель** (0,1 с): Спасибо.

Экраны этого звонка: разговор, карточка, разбор.

2. Что изменилось в оценке приёма вызова после прогона

По коду `backend/app/domain/evaluation/call_intake.py` и `detectors.py`:

- **Обязательные вопросы в облачном звонке считаются только по словам обучаемого.** Заявитель Vapi говорит свободно, и темы по его словам засчитывали бы вопросы, которых не задавали. В звонке с локальной моделью ответ заявителя по-прежнему засчитывает тему вопроса, на который он отвечает.
- **Два блокирующих правила** — критические ошибки, при которых «незачёт» при любом балле: «Обязательные вопросы не заданы» (обучаемый сам задал меньше доли `min_questions_share` обязательных вопросов, по умолчанию половины) и «Карточка не заполнена» (нет ни типа, ни признаков, ни адреса, ни описания).
- **Пустой ввод даёт 0, а не максимум.** Описание короче двух слов — 0 за описание и за грамотность; пустая карточка — ещё 0 за признаки и службы и за время.
- **Норматив приёма вызова по умолчанию 60 с** вместо 90 (таймер живого АРМ-112). Набор задаёт норматив занятия явно (180 с), поэтому на нём это не сказывается.
- Перед сохранением карточки — подтверждение «Оповестить службы и сохранить»; тест нажимает его (`lib/flow.ts`).

Что это значит для набора:

- По правилам сейчас тесты 06, 08 и 11 должны давать «незачёт»: в 06 карточка пустая, в 08 и 11 обучаемый сам задал меньше половины обязательных вопросов. Тест 06 ожидает незачёт, тесты 08 и 11 зачёт не проверяют.
- Пересчёт `lib/scoring.ts` этих правил ещё не знает. Он считает темы по репликам обеих сторон, а грамотность, время и признаки пустой карточки — по сентябрьским формулам. Перед повторным прогоном его нужно довести до текущих правил, иначе тесты 06, 08 и 11 покажут расхождения, хотя движок считает правильно.

3. Звонки диспетчера ДДС: что проверяется сейчас

В режиме ДДС голосом идут звонки диспетчера: начальнику аварийной бригады своей службы, дежурным других служб, руководителю организации и начальнику отдела реагирования. Бригада и специалист звонят сами с докладами. Звонок ведёт локальная модель или облако (Vapi). Оценку звонков считает составляющая «Звонки в службы» (`backend/app/domain/evaluation/service_call.py`, вес 15) и правила типичных ошибок.

Как считается звонок.

Звонок	Доли балла
Начальнику аварийной бригады своей службы	дозвонился до нужных людей 25 % (при «машин нет» нужен ещё руководитель организации), передал факты 30 %, исправил неверно понятый дом 15 %, дал распоряжение на выезд и записал номер наряда 15 %, уложился в норматив 15 %

Звонок	Доли балла
Дежурному другой службы	дозвонился 30 %, передал факты 50 %, уложился в норматив 20 %

Звонок, в котором не передан ни один обязательный факт, даёт 0: дозвониться и промолчать — то же, что не звонить. Норматив звонка 120 с. Начальник бригады сейчас дом не путает (этот приём отключён 18.10.2026), поэтому доля «исправил неверно понятый дом» засчитывается всегда, а правило «Не исправлено неверно понятое» не срабатывает.

Что изменилось в октябре (так помечено в коде: живые звонки на стенде 17–19.10.2026):

- *Адрес засчитывается по правде карточки.* Если оператор 112 ошибся в доме, верным считается дом из описания со слов заявителя; назвать бригаде дом из поля — ошибка «Бригаде передан неверный адрес». Адрес узнаётся в любом падеже и порядке слов, с числительными словами, с улицей через дефис, по ключевому слову названия («Панфиловцев» для «Героев Панфиловцев»).
- *Тип происшествия* засчитывается, если диспетчер назвал его своими словами, которые есть в карточке, а не только по таблице ключевых слов.
- *Облачная анкета.* После облачного звонка начальнику бригады облако заполняет анкету из семи полей: назван ли адрес и какой дом, сказано ли что случилось, про пострадавших и доступ, дано ли распоряжение, исправлен ли дом. Факты этого звонка берутся из анкеты, а не из ключевых слов; дом из анкеты сверяется с правдой карточки на сервере. Без анкеты (локальная модель, другие собеседники) факты читаются из расшифровки.
- *Распоряжение.* Начальник бригады выезжает и называет наряд, как только узнал адрес и что случилось. Распоряжение диспетчер должен дать сам; оно узнаётся в обычных словах («выезжайте», «высылайте», «вы приедете?»). Нет распоряжения — ошибка «Не дано распоряжение на выезд» и потеря доли балла. На карточке «машин нет» распоряжение начальнику бригады не требуется.
- *Новые правила ошибок звонков:* «Позвонил не тому» (распоряжение руководителю организации), «Не доложено руководителю о нехватке сил», «Не оповещена служба об осложнении», «Дозвонились в службу, но ничего не передали».
- *Доклады бригады.* Сведения, которые бригада сообщила по телефону, считаются докладом; статус хода работ раньше доклада — ошибка, доклад без статуса — ошибка, не принятый звонок доклада — ошибка.
- *Решение.* Ошибочная «Принята», исправленная «Отказом от выполнения работ» с комментарием на карточке, которую следовало отклонить, засчитывается как исправление с потерей 6 баллов за «Принята»; цепочка и комментарий не обнуляются.
- *Комментарии.* Статусы хода работ сохраняются только с комментарием. Грамотность проверяется по каждому комментарию отдельно, с заглавной буквы: соседние комментарии больше не читаются как одно предложение.
- *Вал карточек* на сложности 3: баллы карточек не меняются, после вала отдельно показывается сводка — сколько пришло и закрыто, сколько с опозданием и без реакции, взята ли ЧС первой (`backend/app/training/stream.py`).

Какие тесты это закрепляют (`backend/tests/`):

Что	Тесты
Доли звонка начальнику бригады, распоряжение, «машин нет», «позвонил не тому»	`domain/evaluation/test_chief_call_scoring.py`
Начальник бригады выезжает, когда узнал адрес и что случилось, дом нарочно не путает	`providers/test_chief_order.py`
Анкета облака: факты по смыслу, дом по правде, анкета главнее ключевых слов	`providers/test_cloud_form.py`
Адрес и тип в живой речи: падежи, числительные, дефис, ключевое слово улицы	`domain/evaluation/test_service_call_facts.py` , `providers/test_truth_address.py`
Молчаливый звонок даёт 0 и называется отдельной ошибкой	`domain/evaluation/test_silent_service_call.py`
Исправление ошибочной «Принята» отказом	`domain/evaluation/test_refusal_after_accept.py`
Осложнение, «машин нет», торопливый собеседник	`providers/test_squad_complication.py` , `providers/test_no_vehicles.py` , `providers/test_officer_rush.py`
Доклады бригады на шкале времени	`api/test_squad_reports.py`
Сводка вала	`training/test_stream.py`
Виртуальный телефон от справочника до разбора (локальный голос)	`frontend/e2e/phone.spec.ts`

Эти тесты работают на записанных случаях и локальной модели, облако в них не вызывается.

4. Как запустить набор живых звонков

Нужен стенд с облачным голосом и учётные записи преподавателя и обучаемого на нём. Обучаемый должен состоять в группе преподавателя. Обоим администратор должен открыть режим оператора 112 («Пользователи», галочка в колонке «Режим 112»): без неё преподаватель не создаст занятие приёма вызова, а обучаемый его не увидит. Тест завершает другие идущие занятия «Приём вызова», иначе софтфон возьмёт чужой звонок. Учётные данные передаются только через окружение:

```
cd frontend
E2E_BASE_URL=https://<стенд> \
E2E_TEACHER_LOGIN=… E2E_TEACHER_PASSWORD=… \
E2E_STUDENT_LOGIN=… E2E_STUDENT_PASSWORD=… \
npx playwright test e2e/vapi
```

Около 40 минут на все двенадцать звонков (каждый — отдельное занятие, около 3 минут и кредиты Vapi). `E2E_CASES=04,05` — выборочно, `E2E_HEADLESS=1` — без окна браузера. Без переменных учётных записей набор пропускается, так что обычный `npm run e2e` его не запускает. Результаты в `frontend/e2e/vapi/out/` (не в git): `suite-<час>.md` — строка на тест, `<час>-<тест>.json` — стенограмма, оценка и все проверки, `<час>-<тест>.png` — экран после сохранения.

Файлы: `vapi-suite.spec.ts` — двенадцать случаев и ожидания; `lib/flow.ts` — занятие через API, вход, звонок, реплики, варианты карточки; `lib/scoring.ts` — независимый пересчёт; `lib/audio-bridge.js` — подмена `getUserMedia` синтетическим микрофоном; `phrases/` — реплики диспетчера и `make-phrases.ps1` для их пересоздания (Windows, голос «Microsoft Irina»).

11. Использованные библиотеки и компоненты

Все зависимости с лицензиями. Модели ИИ — в разделе «Модели», облачные сервисы, которые подключаются только явной настройкой, — в разделе «Облачные сервисы». Версии библиотек зафиксированы в `backend/uv.lock`, `frontend/package-lock.json` и `deploy/stt/requirements.txt`.

Бэкенд (Python 3.12)

Библиотека	Лицензия	Зачем
FastAPI, Starlette, Uvicorn	MIT / BSD-3	HTTP API, WebSocket событий занятия
python-multipart	Apache 2.0	приём записанных фраз и звуковых файлов реплик
Pydantic, pydantic-settings	MIT	схемы, строгая схема сценария, настройки из `.env`
SQLAlchemy 2, asyncpg	MIT / Apache 2.0	PostgreSQL
Alembic	MIT	миграции
argon2-cffi	MIT	хеширование паролей
PyJWT	MIT	токены доступа и продления
cryptography	Apache 2.0 / BSD	шифрование SIP-паролей обучающихся (Fernet)
structlog	MIT / Apache 2.0	журналы в JSON
prometheus-fastapi-instrumentator	ISC	метрики `/api/metrics`
redis	MIT	рассылка событий занятия, состояние задач генерации, блокировки звонков
ARQ	MIT	сервис `worker`, зарезервирован под фоновые задачи
httpx	BSD-3	клиент моделей (llama.cpp, распознавание, облачная модель), LanguageTool, облачный голос
websockets	BSD-3	события ARI Asterisk
tzdata	Apache 2.0	часовой пояс Москвы в контейнере
openpyxl	MIT	разбор классификатора (xlsx), выгрузка отчёта в Excel
ReportLab	BSD	отчёт о занятии в PDF; шрифт DejaVu Sans (лицензия Bitstream Vera) встроен, кириллица без системных шрифтов
rapidfuzz	MIT	нечёткое сравнение улиц, адресов и ключевых слов в оценке и в разговоре
NumPy	BSD-3	звук звонка (RTP ↔ PCM, детектор речи, передискретизация), модель прогноза готовности, виртуальная когорта
onnxruntime	MIT	модель смысловой близости e5 и детектор речи Silero VAD

Библиотека	Лицензия	Зачем
tokenizers	Apache 2.0	токенизация текста для e5
PyTorch (сборка для процессора)	BSD-3	запуск модели синтеза Silero
num2words	LGPL-2.1 (библиотека без изменений)	числа словами перед синтезом Silero («11 лет» → «одиннадцать лет», номера по цифрам)
piper-tts (с espeak-ng)	GPL-3.0	запасной движок синтеза, если нет модели Silero
FFmpeg (пакет Debian, отдельный процесс)	LGPL-2.1+ / GPL-2.0+	перевод озвучки в MP3 и в формат Asterisk

Разработка и тесты: pytest, pytest-asyncio, ruff (MIT); pypdf (BSD-3) для проверки выгрузки PDF. Сборка образа: uv (MIT / Apache 2.0), базовый образ `python:3.12-slim` (лицензия PSF и лицензии пакетов Debian).

Группа `dataset` (только подготовка данных на машине разработчика, в образ не попадает):

Библиотека	Лицензия	Зачем
PyMuPDF	AGPL-3.0 (только в скриптах подготовки, в продукт не входит)	рендер сканов билетов
pytesseract + Tesseract OCR 5 (`rus`)	Apache 2.0	распознавание билетов
Pillow, NumPy	MIT-CMU / BSD-3	поиск сетки таблицы на скане
Shapely	BSD-3	привязка улиц к районам по геометрии границ

Подготовка студийной озвучки (`scripts/voice_replies.py` , только машина разработчика с интернетом, в продукт не входит): httpx и API ElevenLabs. В поставку входят только готовые звуковые файлы.

Сборка документации (`scripts/build_docs.py` , `scripts/guide_screenshots.py` ; только машина, где собирают PDF, в продукт не входит):

Библиотека	Лицензия	Зачем
python-docx	MIT	общий комплект документации в DOCX
playwright (Python) + Chromium	Apache 2.0 / BSD	печать руководств в PDF, снимки экрана
pypdf	BSD-3	номера страниц для оглавления PDF
Шрифты PT Sans, PT Mono (`docs/fonts`)	SIL Open Font License 1.1	кириллица в PDF без системных шрифтов

Распознавание речи (сервис `stt`)

Библиотека	Лицензия	Зачем
faster-whisper 1.2 (CTranslate2, PyAV)	MIT (CTranslate2 MIT, PyAV BSD, FFmpeg в составе PyAV LGPL)	распознавание речи обучающегося на процессоре
FastAPI, Uvicorn, python-multipart	MIT / BSD-3 / Apache 2.0	HTTP-обёртка, совместимая с интерфейсом транскрипции OpenAI

Телефония (сервисы `asterisk`, `asterisk-cloud`)

Компонент	Лицензия	Зачем
Asterisk 22 (образ `andrius/asterisk:22.10.1_debian-trixie`: PJSIP, ARI, SRTP, Opus)	GPL-2.0 (отдельный процесс, связь по HTTP и WebSocket ARI)	регистрация программного телефона по WebRTC и настольных телефонов по SIP, мост звонка, запись, передача звука на сервер (ExternalMedia), SIP-транк оператора связи; в профиле `cloud` — SIP-транк в облачный голос

Фронтенд (TypeScript)

Библиотека	Лицензия	Зачем
React 19, React DOM	MIT	интерфейс
React Router	MIT	маршруты
TanStack Query	MIT	загрузка данных
openapi-fetch, openapi-typescript	MIT	типизированный клиент API
JsSIP	MIT	программный телефон в браузере: регистрация по WebSocket, звонок по WebRTC
@vapi-ai/web (с @daily-co/daily-js)	MIT (daily-js — BSD-2)	облачный голос из браузера; используется только при включённом облачном голосе
Apache ECharts	Apache 2.0	тепловая карта, динамика, калибровочный график и рейтинги в аналитике
Tailwind CSS 4	MIT	стили
shadcn/ui (код компонентов в репозитории), Radix Slot, CVA, clsx, tailwind-merge	MIT	компоненты
lucide-react	ISC	иконки
@fontsource/roboto, @fontsource-variable/golos-text, @fontsource/jetbrains-mono	OFL 1.1 / Apache 2.0	шрифты в сборке, без внешних запросов
Vite, TypeScript, ESLint, Vitest, Testing Library, jsdom	MIT	сборка, проверка, тесты
Playwright (Chromium)	Apache 2.0	сквозные проверки в браузере

Инфраструктура (образы)

Образ	Лицензия	Зачем
`nginx:1.27-alpine`	BSD-2	TLS, интерфейс, прокси; собирается с интерфейсом внутри
`node:22-alpine`	MIT	только этап сборки интерфейса, в итоговый образ не входит
`pgvector/pgvector:pg16`	PostgreSQL License	база данных
`redis:7-alpine`	BSD-3 (Redis 7.2)	события занятия
`erikvl87/languagetool:6.5`	LGPL-2.1 (LanguageTool)	проверка грамотности
`ghcr.io/ggml-org/llama.cpp:server`	MIT	сервер языковых моделей (`llm-dialog` , `llm-gen`)
`postgres:16-alpine`	PostgreSQL License	резервные копии (`pg_dump` , `pg_restore`)
`andrius/asterisk:22.10.1_debian-trixie`	GPL-2.0 (Asterisk)	телефония
`python:3.12-slim`	PSF и лицензии пакетов Debian	основа образов `backend` , `worker` , `stt`
`alpine:3.20`	MIT (BusyBox — GPL-2.0)	заготовка профилей `live` и `monitoring`

Модели

Скачиваются один раз `scripts/fetch_models.sh` в папку `models/` (не в git) или входят в поставку без интернета. Файлы, размеры и sha256 — в `scripts/models.manifest` ; скрипт проверяет хеш после загрузки.

Модель	Файл	Лицензия	Зачем
Qwen2.5-3B-Instruct, GGUF Q4_K_M	`llm/qwen2.5-3b-instruct-q4_k_m.gguf`	**Qwen Research License (некоммерческая)**	в поставке и на стенде — собеседники и написание карточек (одна модель на обе задачи); для коммерческого использования заменить на 1.5B или 7B
Qwen2.5-1.5B-Instruct, GGUF Q4_K_M	`llm/qwen2.5-1.5b-instruct-q4_k_m.gguf`	Apache 2.0	собеседники по умолчанию в `docker-compose.yml` , для слабого сервера

Модель	Файл	Лицензия	Зачем
Qwen2.5-7B-Instruct, GGUF Q4_K_M (2 части)	`llm/qwen2.5-7b-instruct-q4_k_m-0000N-of-00002.gguf`	Apache 2.0	написание сценариев и карточек в сервисе `llm-gen` по умолчанию `docker-compose.yml`; в поставке не запускается
faster-whisper base / small (Systran, CTranslate2 int8)	`stt/faster-whisper-{base,small}/`	MIT (веса Whisper OpenAI — MIT)	распознавание речи; в поставке `small`
Silero TTS v4 ru (голоса aidar, baya, kseniya, xenia, eugene)	`tts/silero_v4_ru.pt`	**CC BY-NC-SA 4.0 (некоммерческая)**	основной синтез речи на сервере; для коммерческого использования нужна коммерческая лицензия Silero или другой движок (Piper)
Piper ru_RU denis, dmitri (medium)	`tts/ru_RU-{denis,dmitri}-medium.onnx`	датасет CC0, модель MIT	запасной синтез, мужские голоса
Piper ru_RU irina (medium)	`tts/ru_RU-irina-medium.onnx`	датасет RHVoice, лицензия голоса автором не указана; модель MIT	запасной синтез, женский голос; перед коммерческим использованием заменить или уточнить лицензию
intfloat/multilingual-e5-small (ONNX)	`embeddings/multilingual-e5-small/`	MIT	смысловая близость комментариев и описаний, поиск по памятке и билетам при генерации
Silero VAD v5 (ONNX, onnx-community/silero-vad)	`vad/silero_vad.onnx`	MIT	границы фраз обучающегося в звонке через Asterisk; без файла — определение по громкости

Модель Silero TTS скачивается с models.silero.ai, остальные — с Hugging Face. Голос Piper `ruslan` (CC BY-NC-SA) не используется.

Студийные записи. Реплики эталонных сценариев и постоянные фразы собеседников ДДС записаны заранее сервисом ElevenLabs и лежат в `data/seed/audio`. Это готовые звуковые файлы, при работе тренажёра ElevenLabs не вызывается. Использование записей регулируется условиями ElevenLabs для аккаунта, на котором они сделаны.

Облачные сервисы (только при `ALLOW_EXTERNAL_AI=true`)

В закрытом контуре не используются. Без явного разрешения система с этими настройками не запускается. Это коммерческие сервисы, они работают по условиям поставщика и по ключам заказчика.

Сервис	Настройка	Зачем
Облачная языковая модель с OpenAI-совместимым интерфейсом	`LLM_GEN_URL`, `LLM_GEN_API_KEY`, `LLM_GEN_API_MODEL`	быстрое написание карточек и сценариев
Vapi — облачный голос: языковая модель OpenAI GPT-4.1, голос ElevenLabs (eleven_flash_v2_5), распознавание Soniox	`CLOUD_VOICE_ENABLED`, `VAPI_*`, `CLOUD_VOICE_*`	режим «Свободная генерация речи (с интернетом)»

Отдельно от ИИ к Asterisk можно подключить SIP-линию оператора связи (`TELEPHONY_TRUNK_*`, в облачном профиле — МультиФон, `MULTIFON_*`), чтобы звонки занятия шли на обычный телефон обучающегося. Это услуга связи по договору с оператором; по умолчанию линия выключена.

Данные

Источник	Лицензия	Зачем
OpenStreetMap (через Overpass API), © участники OpenStreetMap	ODbL 1.0	улицы Москвы с округом и районом (`data/seed/streets.json`)
Датасет организаторов (классификатор, памятка, билеты, скриншоты АРМ-112)	предоставлен организаторами хакатона	справочники, учебные ситуации, вид рабочего места

12. Соответствие техническому заданию

Каждый пункт ТЗ заказчика (`ТЗ_ДГОЧСиПБ_финал_01092026`), ответы заказчика на вопросы участников (сентябрь 2026) и рекомендации заказчика финалистам от 16.10.2026: где пункт выполнен и как это проверить. Каждая строка сверена с кодом и тестами на 20.10.2026.

Статусы: «да» — выполнено и проверяется тестом или вручную; «частично» — выполнено не полностью, в колонке «Где» сказано, чего не хватает; «нет» — не реализовано.

Пути тестов: `backend/tests/...` запускаются через `pytest` , `frontend/e2e/*.spec.ts` — через `npx playwright test` . Основной режим тренажёра — «Реагирование на карточку» (диспетчер ДДС); режим «Приём вызова» (оператор 112) по рекомендации заказчика скрыт и включается администратором отдельным пользователям.

Сводка на 20.10.2026: пунктов ТЗ и ответов заказчика 174, из них «да» 140, «частично» 31, «нет» 3. Рекомендаций заказчика к финалу 10: выполнено 7, частично 2, не проверяется по коду 1 (раздел 15).

1. Ключевые компоненты решения (раздел «Описание работы компании»)

Требование ТЗ	Где	Как проверить	Статус
Система эмуляции вызовов с использованием IP-телефонии и VoIP	Asterisk (`deploy/asterisk/` , `backend/app/telephony/`): софтфон в браузере по WebRTC, настольный SIP-телефон, живой звонок на телефон обучаемого; включается профилем `telephony` . Виртуальный телефон ДДС (`frontend/src/emulator/virtual-phone.tsx`) работает в браузере и без Asterisk: голос идёт через сервер или облако	`backend/tests/telephony` , `e2e softphone.spec.ts` , `phone.spec.ts`	да
Модуль управления обучением с распределением заданий по уровням сложности	занятия и группы (`backend/app/training/sessions.py` , `service.py`): число карточек, ошибки оператора 112 и поведение собеседников по сложности, вал карточек на сложности 3; форма занятия `frontend/src/pages/teacher/session-form.tsx`	`tests/api/test_sessions.py::test_difficulty_3_is_a_stream_of_cards_on_a_timer` , `::test_lower_difficulty_is_no_stream` , `tests/api/test_attempts.py::test_journal_issues_cards_by_difficulty`	да
Компонент оценки эффективности на основе анализа действий	`backend/app/domain/evaluation/` : составляющие балла и 29 правил типичных ошибок по памятке	`backend/tests/domain/evaluation`	да
Интерфейс преподавателя для контроля процесса обучения	`frontend/src/pages/teacher/` , `backend/app/training/teacher.py`	`e2e teacher.spec.ts`	да

2. Предназначение сервиса

Требование ТЗ	Где	Как проверить	Статус
Обучение операторов работе с ПО системы-112	эмулятор журнала и карточки ДДС по скриншотам заказчика (`frontend/src/emulator/`); карточка оператора 112 (`frontend/src/intake/` , скрыта по умолчанию)	e2e `dds.spec.ts` , `docs/DEMO_SCRIPT.md`	да
Отработка навыков обработки происшествий и ЧС	96 ситуаций из билетов, 114 сценариев в поставке, карточки ИИ по классификатору; ЧС на сложности 3 (DATASET.md)	`backend/tests/importers` , `tests/api/test_ai_cards.py`	да
Контроль качества через автоматизированную оценку	`domain/evaluation` , разбор карточки с баллом по составляющим	`backend/tests/domain/evaluation`	да
Моделирование ситуаций с эмуляцией вызовов	ДДС: звонки бригаде, службам и руководству, входящие доклады бригады; оператор 112: входящий звонок заявителя	e2e `dds.spec.ts` , `phone.spec.ts` , `call_intake.spec.ts`	да
Управление учебным процессом с уровнями сложности	сложность 1–3 в занятии; адаптивный подбор (`domain/analytics/adaptive.py`); подбор следующей карточки ИИ по ошибкам (`domain/scenarios/ai_plan.py`)	`tests/domain/analytics/test_adaptive.py` , `tests/domain/scenarios/test_ai_plan.py`	да
Анализ эффективности обучения на основе машинного обучения	прогноз готовности к аттестации: логистическая регрессия с калибровкой (`domain/analytics/model.py`), обучена на смоделированной истории (`simulate.py`), на реальной переобучается той же командой	`python -m app.domain.analytics.train_readiness` , `tests/domain/analytics/test_model.py`	да
Реалистичная имитация интерфейса без доступа к внешним сетям	экраны повторяют АРМ-112 по скриншотам заказчика; у заказчика все модели на сервере; облачные модели включаются только явно (`ALLOW_EXTERNAL_AI=true`)	e2e `wave-00.spec.ts` «страница не ходит во внешний интернет», `tests/providers/test_llm_client.py::test_external_ai_address_refuses_to_start`	да
Автоматизированный контроль временных нормативов и ручного ввода (сравнение с эталоном)	`evaluation/timing.py` , `text.py` , `card_response.py`	`tests/domain/evaluation/test_card_response.py::test_time_component` (10, 30, 45, 60, 90 с при нормативе 30)	да

Требование ТЗ	Где	Как проверить	Статус
Снижение нагрузки на преподавателя за счёт генерации сценариев и первичной оценки	генерация по фразе, под занятие и по билетам (`backend/app/scenarios/`, `providers/generation.py`); режим «Задания формирует ИИ» без участия преподавателя (`training/ai_cards.py`)	`tests/api/test_scenarios.py::test_generate_by_phrase_creates_review_scenario`, `tests/api/test_ai_cards.py`, e2e `ai-tasks.spec.ts`	да
Защита данных пользователей и учебных материалов	роли, журнал аудита с цепочкой хэшей, TLS на nginx, пароли в argon2	`tests/api/test_admin.py`, `tests/test_audit.py`	да
Масштабируемость под различные группы пользователей	группы, службы ДДС, профили служб занятия	`tests/api/test_sessions.py::test_groups_list_create_patch`	да
Возможность удалённого контроля учебного процесса	мониторинг занятия по WebSocket (`training/ws.py`), плитка на каждого обучаемого	e2e `teacher.spec.ts`, `tests/api/test_ws.py`	да

3. Постановка задачи

Требование ТЗ	Где	Как проверить	Статус
Эмуляция рабочих процессов оператора 112 в локальном комплексе	режим «Приём вызова» (`frontend/src/intake/`); по рекомендации заказчика скрыт, открывается администратором в «Пользователях»	e2e `call_intake.spec.ts`, `tests/api/test_call_intake_access.py`	да
Имитация входящих вызовов (виртуальная IP-телефония или текстовые сообщения)	ДДС: доклады бригады и специалиста звонят на виртуальный телефон; оператор 112: звонок заявителя; ответ голосом через гарнитуру или текстом	`tests/api/test_squad_reports.py`, e2e `dds.spec.ts` (доклады бригады), `softphone.spec.ts`	да
Инструменты управления обучением для преподавателя	кабинет преподавателя: группы, занятия, мониторинг, отчёты, аналитика, сценарии, справочник телефонов	e2e `teacher.spec.ts`	да
Система оценки действий на основе нейросетевого анализа	нейросети в оценке: смысловая близость комментариев к эталону (multilingual-e5, `providers/embeddings.py`); в облачном режиме звонок начальнику бригады читает языковая модель облака (анкета после звонка). Сам балл — по прозрачным правилам памятки, советы — по шаблону; отдельного комментария модели к попытке нет	`tests/providers/test_cloud_form.py`, `tests/domain/evaluation/test_text.py`	частично

Требование ТЗ	Где	Как проверить	Статус
Работа с различными уровнями сложности	сложность 1–3 (раздел 1)	`tests/api/test_sessions.py`, `tests/domain/scenarios/test_distort.py::test_difficulty_sets_the_exact_number_of_mistakes`	да
Имитация функционала и интерфейса системы-112 в изолированном контуре	у заказчика модели ИИ, распознавание и синтез речи работают на сервере без интернета; установка без интернета (`scripts/offline_bundle.sh`, `offline_install.sh`)	e2e `wave-00.spec.ts`, `tests/providers/test_llm_client.py`, `docs/INSTALL.md`	да
Контроль выполнения рабочих функций (тайминг и регламент)	нормативы, переходы статусов по одному шагу как в живом АРМ-112, 24 правила для карточки ДДС	`tests/domain/evaluation/test_status_machine.py`, `test_detectors.py`	да
Оценка на основе анализа ручного ввода текста	комментарии к статусам (обязательны у отказа, хода работ и завершения), близость к эталону, номер наряда, грамотность LanguageTool по каждому комментарию	`tests/domain/evaluation/test_card_response.py::test_comments_and_order_number`, `::test_grammar_errors_reduce_the_component`	да
Формирование отчётности по результатам обучения	отчёт занятия, советы по занятию, сводка вала, выгрузка PDF, XLSX, CSV (`backend/app/reports/export.py`)	`backend/tests/reports/test_export.py`, `tests/api/test_overrides.py::test_exports_pdf_xlsx_csv`	да

4. Границы решения

Требование ТЗ	Где	Как проверить	Статус
Не обрабатывать реальные экстренные вызовы	учебный контур, все звонки внутри тренажёра; внешний SIP-транк — только опция для звонка на телефон обучаемого, в закрытом контуре выключен (`docs/TRUNK.md`)	`docs/LIMITATIONS.md`	да
Не содержать функционал основной системы-112	только имитация, без интеграции с ПО системы-112	`docs/ARCHITECTURE.md`	да

Требование ТЗ	Где	Как проверить	Статус
Не работать с конфиденциальной информацией вне учебного процесса	данные только учебные; ФИО пользователей скрыто, показ пишется в аудит	`tests/api/test_admin.py::test_users_hidden_names_reveal_is_audited`	да
Не выполнять функции реального диспетчерского центра	нет связи с реальными службами	`docs/LIMITATIONS.md`	да

5. Общие требования к сервису

Требование ТЗ	Где	Как проверить	Статус
Эмуляция работы оператора системы-112 в реальном времени	журнал и карточка ДДС обновляются по событиям занятия, звонки голосом	`docs/DEMO_SCRIPT.md`, e2e `dds.spec.ts`	да
Обработка вызовов через виртуальную IP-телефонию (VoIP)	Asterisk PJSIP, WebRTC из браузера, настольный SIP-телефон	`backend/tests/telephony`, `scripts/bench_call.py`	да
Система управления учебным процессом с распределением заданий	занятия, группы, выдача карточек каждому обучаемому	e2e `teacher.spec.ts`	да
Контроль действий обучаемых (тайминг, регламент)	нормативы и переходы статусов	`tests/domain/evaluation/test_status_machine.py`, `test_card_response.py::test_time_component`	да
Оценка эффективности обучения на основе ИИ	прогноз готовности (машинное обучение), подбор следующей карточки под ошибки, карточки пишет модель, смысловая близость текста	`tests/domain/analytics`, `tests/api/test_ai_cards.py::test_next_card_is_written_after_the_scored_one_by_its_mistakes`	да
Ведение статистики по результатам обучения	прогресс обучаемого, аналитика группы	e2e `analytics.spec.ts`	да
Многоуровневая система аутентификации	пароль (argon2), блокировка после 5 неудачных попыток на 15 минут, роли. **Второго фактора нет**. Вход в один клик («Войти как…») — только для демонстрации, у заказчика выключается (`QUICK_LOGIN=false`)	`tests/test_auth.py::test_five_failures_lock_login_for_fifteen_minutes`, `::test_role_access`	частично

Требование ТЗ	Где	Как проверить	Статус
Разграничение доступа к данным по ролям	проверка роли (`auth/deps.py`) и владельца объекта	`tests/api/test_admin.py` , `tests/api/test_sessions.py`	да
Защита каналов передачи данных	TLS 1.2/1.3 и HSTS на nginx, WSS, DTLS-SRTP у WebRTC. Настольные SIP-телефоны — без TLS и SRTP; внутри Docker-сети сервера трафик не шифруется	`deploy/nginx/default.conf` , `deploy/asterisk/`	частично
Резервное копирование не реже раза в сутки	сервис `backup` (`deploy/backup/backup.sh`): копия базы каждый день в `BACKUP_TIME` (03:00), хранится `BACKUP_KEEP` (14). Записи разговоров и озвучка (том `storage`) в копию не входят	`scripts/restore_test.sh` , `tests/api/test_admin.py::test_backup_request_and_list`	да
Интеграция с форматами данных и визуальными стандартами ГБУ «Система 112»	классификатор, памятка, статусы, экраны по скриншотам заказчика	DATASET.md, сверка со скриншотами	да
Интеграция с локальной системой управления доступом	**не реализовано**: вход только по учётным записям тренажёра, LDAP и AD нет	—	нет
Интеграция с локальными средствами мониторинга	метрики в формате Prometheus по `/api/metrics` (без авторизации), экран «Состояние» у администратора; сами Prometheus и Grafana не поставляются	`curl -k https://<сервер>/api/metrics` , экран «Состояние»	частично

6. Опциональные требования

Требование ТЗ	Где	Как проверить	Статус
Дополнительные модули аналитики (графики, тепловые карты ошибок)	аналитика группы: тепловые карты по режимам, динамика, топ ошибок, прогноз (`frontend/src/pages/teacher/analytics.tsx` , `backend/app/analytics/service.py`)	e2e `analytics.spec.ts` , `tests/domain/analytics/test_insights.py::test_heatmap_and_dynamics`	да
Экспорт отчётности в Excel и PDF	отчёт занятия в PDF, XLSX и CSV (`backend/app/reports/export.py`), каждая выгрузка в аудите; аналитика группы не выгружается	`backend/tests/reports/test_export.py`	да

Требование ТЗ	Где	Как проверить	Статус
Механизм импорта обновлений учебных материалов пакетом	методические материалы загружаются по одному файлу (PDF, DOCX, TXT); пакетно — только консольные команды: импорт данных заказчика (`python -m app.importers.organizers`) и генерация по всем билетам (`python -m app.scripts.generate_from_tickets --all`). Пакетного импорта сценариев через интерфейс нет	`tests/importers/test_organizers.py` , `tests/api/test_scenarios.py::test_reference_docs_roundtrip`	частично
Адаптивный веб-интерфейс для мобильных устройств	кабинеты читаются на ширине 390 px; эмулятор АРМ-112 рассчитан на экран от 1280 px	e2e `admin.spec.ts` (390 px), `teacher.spec.ts` (390 px)	частично
Внутренний API для расширения функционала	REST с описанием OpenAPI (`/api/openapi.json` , `backend/app/openapi_export.py`)	открыть `/api/openapi.json`	да

7. Требования к производительности

Замеры — в PERFORMANCE.md (стенд 6 vCPU, 12 ГБ, 22.09.2026, если не сказано иное).

Требование ТЗ	Где	Как проверить	Статус
Отклик интерфейса не более 2 с при 100 пользователях	асинхронный бэкенд, индексы	`load/load_http.py browse --users 100` : худший p95 1,8 с (вход), ошибок 0	да
Не менее 20 одновременных сессий	события занятия по каналам	`load/load_http.py cards --users 20` : p95 1,1 с; e2e `monitor-load.spec.ts` (мониторинг 20 обучаемых)	да
Задержка голоса не более 150 мс	WebRTC в локальной сети, кодеки Opus и G.722	RTT WebRTC 2 мс, джиттер 0, потерь нет (16.09.2026, браузер и Asterisk на одной машине, `softphone.spec.ts`); в сети учебного класса не замерялось	частично
Работа при сбоях сети до 30 с с восстановлением сессий	черновик карточки, повтор действия без дублей, догон событий после обрыва (`backend/app/events.py`)	e2e `dds.spec.ts` «30 с офлайн и повтор действия…», `monitor-load.spec.ts` (обрыв 10 с)	да
Скорость записи в БД не менее 100 операций в секунду	события занятия и аудит, каждая запись — своя транзакция	`scripts/bench_db_writes.py` : 825 событий/с, аудит 597/с	да
Формирование аналитических отчётов не более 30 с	агрегаты SQL, PDF и XLSX строятся в отдельном потоке	замер: отчёт на 100 обучаемых и 100 карточек — PDF 0,8 с, XLSX 0,08 с; автотеста на время нет	да

Требование ТЗ	Где	Как проверить	Статус
Буферизация данных при временных сбоях	черновик карточки ДДС в браузере (`frontend/src/emulator/draft.ts`), повтор действия по его номеру; черновик карточки оператора 112 ещё и на сервере	e2e `dds.spec.ts` (черновик переживает перезагрузку), `tests/api/test_attempts.py::test_status_is_idempotent_by_action_id`	да
Вертикальное масштабирование	число потоков и слотов моделей (`LLM_THREADS`, `LLM_DIALOG_SLOTS`); бэкенд работает одним процессом	`docs/ARCHITECTURE.md`, `docs/PERFORMANCE.md`	частично
Горизонтальное масштабирование (узлы локального кластера)	модели, распознавание речи и телефония выносятся на отдельные серверы адресами в `.env`; несколько экземпляров бэкенда не поддерживаются, стенд из двух узлов не проверялся	`docs/ARCHITECTURE.md`	частично
Минимальные требования к АРМ обучающегося	браузер и гарнитура, ставить ничего не нужно	`docs/INSTALL.md`	да
Минимальные требования к серверу (6 ядер, 32 ГБ, NVMe 512 ГБ)	модели квантованы и работают на процессоре без видеокарты; стенд работает на 6 ядрах и 12 ГБ с облачной моделью для карточек; голосовых разговоров на 6 ядрах — 3–4 одновременно	`docs/guides/TECH_DESCRIPTION.md`, раздел 5	да

8. Пользовательские требования: интерфейс

Требование ТЗ	Где	Как проверить	Статус
Интуитивная понятность для всех категорий пользователей	рабочее место ДДС повторяет привычный АРМ-112, учебная панель «Тренажёр» свёрнута; кабинеты без лишних кнопок	e2e `dds.spec.ts` «всё выполнимо с клавиатуры…», проверка на стенде	да
Единая стилистика оформления	общие компоненты (`frontend/src/components/ui/`), токены светлой и тёмной темы (`frontend/src/index.css`)	e2e `wave-00.spec.ts` (переключатель темы)	да
Адаптивность под различные устройства	кабинеты до 390 px, эмулятор от 1280 px	e2e `admin.spec.ts`, `teacher.spec.ts` (390 px)	частично
Локализация на русском языке	весь интерфейс на русском	визуальная проверка	да

9. Роль: администратор

Требование ТЗ	Где	Как проверить	Статус
Запускать и останавливать сервисы системы	**из интерфейса нельзя**: только `docker compose` на сервере и автоперезапуск контейнеров	`docs/guides/ADMIN_GUIDE.md`	нет
Контролировать состояние компонентов в реальном времени	экран «Состояние»: плитки сервисов, обновление каждые 15 с (`backend/app/admin/health.py`, `frontend/src/pages/admin/health.tsx`)	e2e `admin.spec.ts`, `tests/api/test_admin.py::test_health_tiles_and_languagetool_notification`	да
Настраивать параметры производительности	только в `.env` (потоки и слоты моделей, память LanguageTool); на экране настроек их нет	`.env.example`	частично
Выполнять обновление ПО пакетно	все образы одним архивом (`scripts/offline_bundle.sh`, `offline_install.sh`), инструкция; из интерфейса не запускается	`docs/INSTALL.md`	частично
Проводить резервное копирование	«Сделать копию сейчас» и список копий (`backend/app/admin/backups.py`); копируется база	`tests/api/test_admin.py::test_backup_request_and_list`, `scripts/restore_test.sh`	да
Создавать учётные записи всех категорий	«Пользователи» (`frontend/src/pages/admin/users.tsx`)	e2e `admin.spec.ts`, `tests/api/test_admin.py::test_create_block_reset_and_sip`	да
Назначать роли и права доступа	роль, служба ДДС, режим оператора 112 у пользователя	`tests/api/test_admin.py::test_create_block_reset_and_sip`, `tests/api/test_call_intake_access.py`	да
Блокировать и разблокировать учётные записи	блокировка с отзывом сессий, себя заблокировать нельзя	`tests/api/test_admin.py::test_create_block_reset_and_sip`	да
Настраивать параметры виртуальной IP-телефонии	«Настройки» → телефония: адрес управления Asterisk, ожидание ответа, запись, кодеки, телефоны обучаемых	экран «Настройки», `tests/api/test_telephony.py::test_defaults_and_patch`	да
Конфигурировать взаимодействие с локальной базой данных	подключение задаётся в `.env`, из интерфейса не меняется	`docs/INSTALL.md`	частично

Требование ТЗ	Где	Как проверить	Статус
Управлять настройками безопасности и политиками доступа	в интерфейсе: роли, блокировка, сброс пароля; число попыток входа и время блокировки — только в `.env`; отдельной политики паролей нет (не короче 8 символов)	`tests/api/test_admin.py`	частично
Настраивать параметры резервного копирования	время копии и число хранимых копий	`tests/api/test_admin.py::test_settings_sections`	да
Конфигурировать параметры журналирования	уровень журнала (DEBUG–ERROR)	`tests/api/test_admin.py::test_settings_sections`, `e2e admin.spec.ts`	да
Просматривать системные журналы	в интерфейсе — журнал аудита с фильтрами; журналы сервисов — только `docker compose logs`	`tests/api/test_admin.py::test_audit_page_filters_and_verify`	частично
Анализировать статистику использования системы	экран «Состояние»: идущие занятия, активные звонки, карточки в работе — только текущие значения, без истории	экран «Состояние»	частично
Формировать отчёты об ошибках и сбоях	оповещения администратора о сбоях сервисов в кабинете (`health.py`); выгрузки отчёта нет	`tests/api/test_admin.py::test_health_tiles_and_languagetool_notification`	частично
Отслеживать нагрузку на сервер	загрузка процессора, память, средняя нагрузка — текущие значения	экран «Состояние»	да
Контролировать показатели работоспособности	проверки сервисов `GET /api/admin/health`, общий `/health`	`tests/api/test_admin.py::test_health_tiles_and_languagetool_notification`	да
Управлять политиками безопасности и правилами доступа	роли, блокировка, доступ к режиму оператора 112; свою роль поменять и себя заблокировать нельзя	`tests/api/test_admin.py::test_create_block_reset_and_sip`	частично
Контролировать целостность системы	проверка цепочки аудита, кнопка «Проверить целостность»	`tests/api/test_admin.py::test_audit_page_filters_and_verify`, `tests/test_audit.py::test_chain_verifies_and_detects_tampering`	да

Требование ТЗ	Где	Как проверить	Статус
Вести аудит безопасности	неизменяемый журнал с цепочкой хэшей (`backend/app/audit.py`), у приложения нет прав менять и удалять записи	`tests/test_audit.py`	да
Ограничение: не вмешиваться в учебный процесс (не менять оценки и сценарии)	администратор получает 403	`tests/api/test_admin.py::test_admin_cannot_touch_evaluations_scenarios_or_lessons`, `::test_admin_cannot_edit_scenarios`	да
Ограничение: не удалять критичные данные без копии	у администратора нет операций удаления; ежедневная копия базы	`backend/app/admin/router.py`, `tests/api/test_admin.py::test_admin_cannot_edit_scenarios`	да
Ограничение: не менять базовые настройки безопасности без прав	настройки безопасности — только в `.env` на сервере	`tests/api/test_admin.py::test_create_block_reset_and_sip`	частично
Ограничение: доступ к персональным данным по необходимости	ФИО в списке скрыто, показ пишется в аудит	`tests/api/test_admin.py::test_users_hidden_names_reveal_is_audited`	да

10. Роль: преподаватель

Требование ТЗ	Где	Как проверить	Статус
Создавать, редактировать и утверждать сценарии, в том числе сгенерированные нейросетью	библиотека сценариев (`frontend/src/pages/teacher/scenarios.tsx`, `scenario.tsx`): генерация по фразе, сценарий из карточки обучаемого, генерация под занятие; правка, версии, утверждение с проверкой грамотности. С нуля без генерации — только через API	e2e `scenarios.spec.ts`, `tests/api/test_scenarios.py::test_generate_by_phrase_creates_review_scenario`, `::test_revise_keeps_approved_replies_in_new_version`	да
Назначать задания и группы	группы и занятия (`groups.tsx`, `session-form.tsx`)	e2e `teacher.spec.ts`, `tests/api/test_sessions.py::test_groups_list_create_patch`	да
Контролировать выполнение задач в реальном времени	мониторинг занятия (`session.tsx`, WebSocket)	e2e `teacher.spec.ts`, `tests/api/test_sessions.py::test_monitor_progress_and_finish_with_report`	да

Требование ТЗ	Где	Как проверить	Статус
Отслеживать прогресс и историю успеваемости	аналитика группы, история занятий	e2e `analytics.spec.ts`, `tests/api/test_analytics.py`	да
Создавать материалы и загружать ресурсы	методические материалы (PDF, DOCX, TXT), обучаемый видит их в справочнике с поиском	`tests/api/test_scenarios.py::test_reference_docs_roundtrip`, e2e `scenarios.spec.ts` (методичка видна обучающемуся)	да
Классифицировать задания по уровням сложности	сложность сценария и занятия, подписи уровней в форме	`tests/api/test_sessions.py::test_difficulty_3_is_a_stream_of_cards_on_a_timer`, `tests/domain/scenarios/test_ai_plan.py`	да
Редактировать и удалять неактуальные сценарии	неиспользованный сценарий удаляется, использованный уходит в архив и восстанавливается; выданный обучаемым не редактируется	e2e `scenarios.spec.ts` (удаление и архив), `tests/api/test_scenarios.py::test_used_scenario_is_archived_hidden_and_restorable`	да
Просматривать результаты (оценка ИИ и экспертная)	разбор с автоматической оценкой, изменение оценки преподавателем с причиной	`tests/api/test_overrides.py::test_override_needs_reason_and_is_visible_to_student`	да
Формировать отчёты по успеваемости и статистику	отчёт занятия и выгрузка	`backend/tests/reports`, `tests/api/test_overrides.py::test_exports_pdf_xlsx_csv`	да
Получать аналитические рекомендации (инсайты по типичным ошибкам группы)	топ ошибок группы, выжимка, советы по занятию у каждого обучаемого	e2e `analytics.spec.ts`, `tests/domain/analytics/test_insights.py::test_summary_and_recommendations`	да
Мониторинг выполнения заданий в реальном времени	плитки обучаемых: этап, таймер норматива, закрыто и зачтено, баллы	e2e `teacher.spec.ts`, `tests/api/test_sessions.py::test_monitor_counts_the_stream_of_each_trainee`	да
Предоставлять обратную связь через интерфейс	комментарий обучаемому в разборе	`tests/api/test_overrides.py::test_comments_reach_the_student`	да

Требование ТЗ	Где	Как проверить	Статус
Давать рекомендации по улучшению навыков	советы системы после карточки и по занятию (`domain/evaluation/advice.py`) плюс комментарий преподавателя	`tests/domain/evaluation/test_advice.py`, `e2e ai-tasks.spec.ts`	да
Определять параметры сценариев (тип, локация, сложность)	форма генерации: фраза, режим, группа и тип происшествия, сложность, персонаж, шум; место — только словами во фразе, отдельного поля нет	`e2e scenarios.spec.ts`	частично
Устанавливать тайминг, по умолчанию 30 секунд	норматив занятия: 30 с для ДДС, 60 с для оператора 112	форма занятия, `tests/api/test_sessions.py::test_session_norm_overrides_scenario_norm`	да
Настраивать критерии успешности (пороги ошибок, требования к синтаксису)	порог зачёта в форме занятия; веса составляющих — только через API; грамотность проверяется всегда	`tests/domain/evaluation/test_card_response.py::test_pass_threshold_is_applied`, `::test_session_weights_scale_the_maximums`	частично
Ограничение: не менять системные настройки платформы	403 на методы администратора	`tests/test_auth.py::test_role_access`	да
Ограничение: не удалять критичные системные данные	пользователей, справочники и аудит преподаватель не удаляет; но может удалить свои завершённые занятия вместе с результатами и неиспользованные сценарии	`tests/api/test_sessions.py::test_teacher_deletes_finished_and_draft_lessons_with_their_history`	частично
Ограничение: не вмешиваться в работу других преподавателей	группы, занятия, попытки и аналитика видны только своему преподавателю; библиотека сценариев, методички и справочник телефонов общие для всех	`tests/api/test_attempts.py::test_access_rules`, `tests/api/test_analytics.py::test_group_analytics_access_and_empty_state`	частично
Ограничение: изменение оценок только с записью в журнал аудита	причина обязательна, запись в аудит	`tests/api/test_overrides.py::test_override_needs_reason_and_is_visible_to_student`	да

11. Роль: обучающийся

Требование ТЗ	Где	Как проверить	Статус
Доступ к назначенным материалам и сценариям	«Мои задания» (`frontend/src/pages/student-assignments.tsx`)	`tests/api/test_attempts.py::test_assignments_list_running_session_first`, e2e `dds.spec.ts`	да
Выполнение практических заданий в режиме эмуляции	журнал и карточка ДДС; карточка оператора 112, если режим открыт	e2e `dds.spec.ts`, `call_intake.spec.ts`	да
Получение обратной связи по результатам	разбор карточки (`frontend/src/pages/attempt-review.tsx`)	e2e `dds.spec.ts`	да
Отслеживание прогресса и истории ошибок	«Мой прогресс»	e2e `analytics.spec.ts`, `tests/api/test_analytics.py::test_my_progress_carries_ratings_and_dynamics`	да
Сохранность результатов в локальном профиле	история попыток в базе	`tests/api/test_attempts.py`	да
Доступ к списку назначенных сценариев	«Мои задания»	e2e `dds.spec.ts`	да
Просмотр инструкций и методических материалов	справочник: памятка с поиском, методички преподавателя, классификатор (`frontend/src/pages/student-reference.tsx`)	`tests/api/test_attempts.py::test_reference_search`, e2e `scenarios.spec.ts`	да
Выполнение практических заданий (заполнение карточек)	ДДС: статусы, наряд, комментарии, отметки ошибок оператора 112; оператор 112: карточка целиком	e2e `dds.spec.ts`, `call_intake.spec.ts`	да
Сохранение промежуточных результатов	черновик карточки	e2e `dds.spec.ts` (черновик переживает перезагрузку)	да
Принимать входящие вызовы и сообщения в режиме обучения	доклады бригады на виртуальном телефоне, звонок заявителя оператору 112	e2e `phone.spec.ts`, `dds.spec.ts`, `softphone.spec.ts`	да
Использовать инструменты эмуляции АРМ-112	журнал, карточка, виртуальный телефон со справочником номеров	e2e `dds.spec.ts`, `phone.spec.ts`	да
Вводить текстовые ответы в карточку согласно сценарию	комментарии к статусам, номер наряда, верное значение у отмеченного поля	`tests/domain/evaluation/test_card_response.py::test_comments_and_order_number`	да

Требование ТЗ	Где	Как проверить	Статус
Применять стандартные процедуры обработки вызовов	статусы и регламент памятки	`tests/domain/evaluation/test_status_machine.py`	да
Просматривать свои результаты (оценки, время реакции)	разбор и прогресс	e2e `dds.spec.ts`, `analytics.spec.ts`	да
Отслеживать статистику выполнения заданий	«Мой прогресс»	e2e `analytics.spec.ts`	да
Видеть рекомендации системы	советы после карточки, по занятию и в прогрессе	e2e `analytics.spec.ts`, `ai-tasks.spec.ts`	да
Анализировать свои ошибки на основе отчётов ИИ	разбор: каждая ошибка словами памятки со страницей, советы; в режиме ИИ — «Почему ИИ выдал эту карточку». Советы составляются по шаблону, отдельного комментария модели нет	e2e `ai-tasks.spec.ts`, `tests/domain/evaluation/test_advice.py`	частично
Выбирать доступные учебные модули	выбор среди назначенных занятий в «Моих заданиях»; самостоятельного выбора модулей нет	e2e `dds.spec.ts`	частично
Контролировать время выполнения (таймер в интерфейсе)	таймер норматива в карточке и в строке журнала	e2e `dds.spec.ts` «карточка без действий через 30 с…»	да
Ограничение: нет доступа к функциям преподавателя и администратора	403 и страница «Нет доступа»	`tests/test_auth.py::test_role_access`	да
Ограничение: не менять учебные материалы	только чтение	`tests/api/test_scenarios.py::test_student_has_no_access`, `::test_student_cannot_delete_scenarios`	да
Ограничение: не удалять результаты	удаления попыток у обучаемого нет	`tests/api/test_attempts.py`	да
Ограничение: не менять настройки системы	403	`tests/test_auth.py::test_role_access`	да
Ограничение: не видеть результаты других обучающихся	проверка владельца попытки	`tests/api/test_attempts.py::test_access_rules`	да
Защита персональных данных обучающегося	минимум данных, показ ФИО через аудит	`tests/api/test_admin.py::test_users_hidden_names_reveal_is_audited`	да

Требование ТЗ	Где	Как проверить	Статус
Конфиденциальность результатов обучения	видят только сам обучаемый и преподаватель занятия	`tests/api/test_attempts.py::test_access_rules`, `tests/api/test_overrides.py::test_override_rules`	да
Надёжное хранение истории в локальной базе	PostgreSQL, ежедневная копия	`scripts/restore_test.sh`	да
Аутентификация при каждом входе	логин и пароль, сессия по токену с обновлением	`tests/test_auth.py`	да

12. Нефункциональные требования

Требование ТЗ	Где	Как проверить	Статус
Резервное копирование не реже раза в сутки	сервис `backup`, ежедневная копия базы (без тома `storage`)	`scripts/restore_test.sh`	да
Отказоустойчивость при выходе из строя отдельных узлов	при отказе LanguageTool, модели или облака работа продолжается (грамотность «не проверено», карточка по шаблону, локальный голос); кластера нет	`tests/api/test_admin.py::test_health_tiles_and_languagetool_notification`, `tests/api/test_ai_cards.py::test_a_card_lost_in_writing_is_written_by_the_template`	частично
Автоматическое восстановление сервисов после сбоев	`restart: unless-stopped` у всех сервисов, проверки работоспособности	остановить процесс в контейнере и посмотреть `docker compose ps`	да
Мониторинг критических параметров в реальном времени	экран «Состояние», `/api/metrics`	экран администратора	да
Журналирование всех значимых событий	аудит действий (вход, занятия, оценки, выгрузки, настройки, показ ФИО)	`tests/test_audit.py`	да
Многоуровневая аутентификация в рамках локальной сети	пароль, блокировка попыток, роли; второго фактора нет	`tests/test_auth.py`	частично
Шифрование передаваемых данных (TLS внутри контура)	TLS и WSS до браузера; SIP-телефоны и трафик внутри сервера без шифрования	`deploy/nginx/`, `deploy/asterisk/`	частично
Разграничение доступа по ролям (RBAC)	роли и владение объектами	`backend/tests/api`	да

Требование ТЗ	Где	Как проверить	Статус
Защита от несанкционированного доступа	блокировка после неудачных попыток, заголовки безопасности, запрет чужих источников (CORS); демонстрационный вход без пароля в поставке выключен (`DEMO_MODE=false` в `.env.example`)	`tests/test_auth.py::test_five_failures_lock_login_for_fifteen_minutes` , `tests/api/test_admin.py::test_no_cors_for_foreign_origin` , e2e `admin.spec.ts` (заголовки безопасности)	да
Аудит всех действий пользователей	неизменяемый журнал с цепочкой хэшей	`POST /api/admin/audit/verify` , `tests/test_audit.py`	да
Хранение журналов безопасности не менее 6 месяцев	записи аудита не удаляются: у приложения нет прав на изменение и удаление; объём за 6 месяцев не замерялся	`tests/test_audit.py::test_app_role_cannot_modify_audit_log`	да
ОС: Windows 10/11 и Ubuntu 20.04 и выше	рабочие места — любая ОС с браузером; сервер — Linux (Ubuntu 22.04 или 24.04) с Docker; сервер на Windows и Ubuntu 20.04 не проверялся	`docs/guides/TECH_DESCRIPTION.md` , раздел 5	частично
Браузеры: Chrome, Firefox, Яндекс.Браузер	автотесты — в Chromium; Firefox и Яндекс.Браузер проверены вручную	`frontend/playwright.config.ts`	частично
СУБД PostgreSQL 12 и выше	PostgreSQL 16 с pgvector	`docker compose ps`	да
Поддержка IP-телефонии через локальный SIP-сервер	Asterisk в контуре	`backend/tests/telephony`	да
Документированные процедуры установки, настройки и восстановления	`docs/INSTALL.md` , `docs/guides/TECH_DESCRIPTION.md` (разделы 6–8), `docs/guides/ADMIN_GUIDE.md`	установка по инструкции	да
Инструменты диагностики и оповещение администратора об ошибках	оповещения и экран «Состояние» в кабинете (без почты и мессенджеров)	`tests/api/test_admin.py::test_health_tiles_and_languagetool_notification`	да

13. Сценарии из ТЗ и ответы заказчика

Требование	Где	Как проверить	Статус
Сценарий «Настройка учебной среды»: выбор категорий, генерация эталонов, подтверждение преподавателем, ролевая модель ленты, предпросмотр, коррекция комментарием, обучение на материалах, проверка грамматики	библиотека сценариев и форма занятия: группы происшествий и служба ДДС, генерация с эталоном, утверждение, переделка по комментарию, модель получает фрагменты памятки и похожие билеты, утверждение только после проверки грамотности	e2e `scenarios.spec.ts`, `tests/api/test_scenarios.py`	да
Сценарий «Практическое занятие на АРМ-112 (карточки)»: вход, выбор категорий, старт, имитация звонка, заполнение карточки, завершение, отчёт	режим «Приём вызова» (скрыт по умолчанию, открывается администратором)	e2e `call_intake.spec.ts`	да
Сценарий «Действия с карточками»: выбор источника карточек (сгенерированные, созданные обучающимися, смешанные), случайная выдача, действия с карточкой и ввод текста, отчёт	режим «Реагирование на карточку». В форме два источника: утверждённые сценарии библиотеки или «Задания формирует ИИ»; выбор «сгенерированные / созданные обучающимися / смешанные» есть только в API. Карточка обучаемого становится сценарием через «Сделать сценарием реагирования» (при открытом режиме оператора 112)	e2e `dds.spec.ts`, `scenarios.spec.ts` (карточка обучающегося → сценарий), `tests/api/test_scenarios.py::test_student_card_becomes_scenario_and_feeds_student_made_sessions`	частично
Ответ заказчика: карточка приходит диспетчеру уже заполненной	режим ДДС, поля карточки менять нельзя	e2e `dds.spec.ts`	да
Ответ заказчика: диспетчер проверяет данные карточки на ошибки оператора 112	ошибки по сложности (0, 1, 2), всегда видны по описанию; отметка поля с верным значением; «не та служба» — отказ «не в компетенции»; составляющая «Проверка данных»	`tests/domain/evaluation/test_data_check.py`, `tests/domain/scenarios/test_distort.py`, e2e `dds.spec.ts` (проверка карточки от оператора 112)	да
Ответ заказчика: диспетчер совершает исходящие звонки руководителям служб	виртуальный телефон: справочник номеров своей организации, трубки на вкладках служб; составляющая «Звонки в службы»	`tests/domain/evaluation/test_service_call.py`, `tests/api/test_phone_directory.py`, e2e `phone.spec.ts`	да
Инструкция заказчика: статусы телефонии оператора и постобработка вызова	**не реализовано** (относится к скрытому режиму оператора 112)	—	нет

Требование	Где	Как проверить	Статус
Ответ заказчика: диспетчер звонит заявителю напрямую по номеру из карточки	по рекомендации заказчика — только если преподаватель разрешил в занятии; номер диспетчер набирает сам по карточке; заявитель называет верные данные	`tests/api/test_caller_callback.py`, `tests/api/test_phone_directory.py::test_phone_book_shows_the_caller_line_only_when_allowed`, e2e `caller-callback.spec.ts`	да
Ответ заказчика: старший группы докладывает диспетчеру по телефону о ходе работ	начальник аварийной бригады звонит с докладами о выезде, прибытии, работах и завершении; оценка по докладам	`tests/api/test_squad_reports.py`, e2e `dds.spec.ts` (доклады бригады)	да
Ответ заказчика: карточки приходят диспетчеру потоком, норматив идёт у каждой	вал на сложности 3: первая карточка сразу, дальше по одной каждые 40–50 с, всего 3; повторный вызов через 1–2 минуты; норматив с момента «Добавлена»; сводка вала	`tests/api/test_sessions.py::test_difficulty_3_is_a_stream_of_cards_on_a_timer`, `::test_repeated_call_in_a_stream_comes_a_minute_or_two_after_its_original`, `tests/training/test_stream.py`	да
Ответ заказчика: в карточке ДДС службы не добавляются, диспетчер видит остальные и может связаться	полоса служб только для чтения, трубка на вкладке службы звонит её дежурному	e2e `dds.spec.ts`	да
Требование заказчика: карточка и реестр повторяют живой АРМ-112 по составу и цветам	колонки журнала, строка статуса службы, полоса служб, переходы статусов по одному шагу	сверка со скриншотами рабочего места ДДС	да

14. Требования к решению и сдаче

Требование ТЗ	Где	Как проверить	Статус
Открытый исходный код без обфускации	исходный код передаётся полностью, без обфускации; лицензия — все права у команды (`LICENSE`)	просмотр репозитория	да
Перечень использованных библиотек и компонентов	`docs/LIBRARIES.md`	просмотр	да
Комментарии к сложным техническим решениям	комментарии в коде, `docs/DECISIONS.md`	просмотр	да

Требование ТЗ	Где	Как проверить	Статус
Описание методов обработки данных	`docs/METHODS.md`	просмотр	да
Условия и ограничения решения	`docs/LIMITATIONS.md`	просмотр	да
Инструкции по сборке, установке и запуску	`docs/INSTALL.md`, `README.md`, `docs/guides/TECH_DESCRIPTION.md`	установка с нуля	да
Описание функциональной и компонентной архитектуры	`docs/ARCHITECTURE.md`, `docs/guides/TECH_DESCRIPTION.md`	просмотр	да
Соответствие 149-ФЗ и 152-ФЗ	персональных данных граждан нет, данные пользователей минимальны, ФИО показывается с записью в аудит, разграничение доступа	`docs/DECISIONS.md`, `tests/api/test_admin.py::test_users_hidden_names_reveal_is_audited`	да
Презентация в формате pptx или pdf	`docs/presentation/` (pptx и pdf)	просмотр	да
Ссылка на прототип для проверки	демонстрационный стенд, ссылка и логины в `README.md`	открыть ссылку	да
Сопроводительная документация в docx или pdf	`docs/submission/`: сводная документация одним PDF (12 разделов) и четыре отдельных PDF: техническое описание, руководства администратора, преподавателя и обучаемого; сборка `scripts/build_docs.py`	просмотр	да

15. Рекомендации заказчика к финалу (16.10.2026)

№	Рекомендация	Что сделано	Где	Статус
1	Доступ к прототипу с максимальной функциональностью при удалённом доступе	демонстрационный стенд с той же программой, что поставляется заказчику; для скорости карточки на нём пишет облачная модель. Работу стенда в момент проверки по коду не подтвердить	ссылка и логины в `README.md`, `docs/guides/TECH_DESCRIPTION.md` (раздел 4)	да
2	На главной панели — временные логины и пароли по ролям	таблица логинов и паролей всех ролей на странице входа (`SHOW_DEMO_CREDENTIALS`) и кнопки «Войти как…» (`QUICK_LOGIN`); установщик включает оба	`frontend/src/pages/login.tsx`, `tests/test_auth.py::test_shown_credentials_are_listed_and_locked`, `::test_quick_login_enters_the_stand_accounts_but_not_admin`	да

№	Рекомендация	Что сделано	Где	Статус
3	Документация в PDF: общее техническое описание и руководства администратора, преподавателя, обучаемого, раздельно, со ссылкой на Яндекс или Google диск	четыре отдельных PDF собраны из `docs/guides/`; в `README.md` ссылка на документацию в Google Docs. Размещение самих PDF на диске по коду не проверить	`docs/submission/`, `docs/guides/`	частично
4	Главная интерфейсная панель как в оригинале (обязательно)	страница входа в стиле АРМ-112: город, вертолёт, «112 ВХОД В СИСТЕМУ», поля справа; после входа обучаемый попадает в «Мои задания» и оттуда в журнал	`frontend/src/pages/login.tsx`, `student-assignments.tsx`	да
5	Режим оператора 112 скрыть или включать через администратора; приоритет — ДДС по схеме «преподаватель, ИИ → обучаемый ДДС → результат → рекомендации»	режим оператора 112 скрыт по умолчанию, открывается галочкой «Режим 112» у пользователя; основной режим — ДДС; после каждой карточки и занятия — советы для дальнейшего обучения	`backend/app/models/user.py` (`call_intake_access`), `frontend/src/pages/admin/users.tsx`, `tests/api/test_call_intake_access.py`, `tests/domain/evaluation/test_advice.py`	да
6	Интерфейс обучаемого ДДС жёстко как в оригинале; у преподавателя и администратора — без перегруза	журнал и карточка ДДС по скриншотам заказчика; учебная панель «Тренажёр» свёрнута; экраны преподавателя и администратора упрощены	`frontend/src/emulator/journal-page.tsx`, `card-page.tsx`; сверка со скриншотами	да
7	Самостоятельное знакомство с интерфейсом: подсказки назначения окон и кнопок (желательно)	есть подсказка следующего шага в панели «Тренажёр» (если преподаватель включил) и список клавиш по «?». Пошаговых подсказок по окнам и кнопкам нет	`frontend/src/emulator/card-page.tsx`	частично
8	Виртуальный телефон (зелёный кружок): набор номера, четыре собеседника, до 10 номеров на ДДС от преподавателя; с пострадавшим не общаться, заявителю — только если преподаватель включил его номер	зелёный кружок в правом нижнем углу, клавиатура и справочник; у службы до 10 номеров, по умолчанию четыре роли из рекомендации, справочник ведёт преподаватель; начальник бригады выезжает, когда узнал адрес и что случилось (без слова «Выезжайте» — ошибка в разборе), бригада звонит с докладами; заявитель — только по галочке занятия «Разрешить звонок заявителю»	`frontend/src/emulator/virtual-phone.tsx`, `backend/app/training/phones.py`, `tests/api/test_phone_directory.py`, e2e `phone.spec.ts`	да

№	Рекомендация	Что сделано	Где	Статус
9	Видеоролик работы по сети «преподаватель — 2 обучаемых»: разные задания, ИИ формирует и выдаёт задания, оценивает и даёт рекомендации	возможности для ролика есть в коде: двум обучаемым ИИ пишет разные карточки, оценивает и даёт советы; установка создаёт второго обучаемого `student2`. Ссылка на видео — в `README.md`; содержание ролика по коду не проверить	`tests/api/test_ai_cards.py::test_each_trainee_gets_own_card_written_at_start`, e2e `ai-tasks.spec.ts`	не проверяется по коду
10	ИИ формирует карточки для ДДС, выдаёт их, оценивает действия и даёт рекомендации без участия преподавателя	режим «Задания формирует ИИ»: преподаватель ставит галочку, выбирает группу, службу и сложность и начинает занятие; каждому обучаемому модель пишет свою карточку под его ошибки (на сложности 1 — одну, на 2 — две по очереди, на 3 — вал из трёх), система оценивает и даёт советы. Оценка — по правилам памятки, советы — по шаблону из найденных ошибок; если модель не ответила, карточка собирается по шаблону	`backend/app/training/ai_cards.py`, `domain/scenarios/ai_plan.py`, `tests/api/test_ai_cards.py`, e2e `ai-tasks.spec.ts`	да

Что не закрыто

Пункт	Что не сделано
Аутентификация	второго фактора нет
Локальная система управления доступом	интеграции с LDAP и AD нет
Кабинет администратора	запуск и остановка сервисов, журналы сервисов, параметры производительности, базы данных и безопасности — только на сервере, не в интерфейсе; статистика использования и отчёт о сбоях без истории и выгрузки
Мониторинг	Prometheus и Grafana не поставляются; `/api/metrics` открыт без авторизации
Шифрование	у настольных SIP-телефонов нет TLS и SRTP, внутри сервера трафик не шифруется
Резервная копия	записи разговоров и озвучка (том `storage`) копируются вместе с сервером, не сервисом копий
Масштабирование	несколько экземпляров бэкенда не поддерживаются, стенд из двух узлов не проверялся
Учебные материалы	нет пакетного импорта сценариев и материалов через интерфейс
Форма занятия	нет выбора источника «сгенерированные / созданные обучающимися / смешанные» и весов составляющих (есть в API); нет отдельного поля места происшествия при генерации
Оценка	отдельного комментария модели к попытке нет: балл по правилам, советы по шаблону
Библиотека сценариев	общая для всех преподавателей

Пункт	Что не сделано
Оператор 112	статусы телефонии оператора и постобработка вызова не сделаны
Совместимость	автотесты только в Chromium; сервер на Windows и Ubuntu 20.04 не проверялся; эмулятор не рассчитан на телефон
Задержка голоса	в сети учебного класса не замерялась
Рекомендация 7	пошаговых подсказок по окнам и кнопкам нет